# OGC CDB Version 2 - Part 1

***Core Standard***

**Open Geospatial Consortium**

Submission Date: 2023-05-22

Approval Date: 2023-12-19

Publication Date: 2026-01-06

External identifier of this OGC® document: http://www.opengis.net/doc/IS/CDB-core/2.0

Internal reference number of this OGC® document: 23-034

Version: 2.0

Category: OGC® Implementation Standard

Editor: Carl Reed, PhD

# OGC CDB Version 2 - Part 1: Core Standard

**Copyright notice**

Copyright © 2026 Open Geospatial Consortium

To obtain additional rights of use, visit http://www.ogc.org/legal/

**Warning**

This document is an OGC Member approved international standard. This document is available on a royalty free, non-discriminatory basis. Recipients of this document are invited to submit, with their comments, notification of any relevant patent rights of which they are aware and to provide supporting documentation.

| |
|---|
| Document type: OGC® Standard |
| Document subtype: |
| Document stage: Approved |
| Document language: English |

License Agreement

Use of this document is subject to the license agreement at https://www.ogc.org/license

Suggested additions, changes and comments on this document are welcome and encouraged. Such suggestions may be submitted using the online change request form on OGC web site: http://ogc.standardstracker.org/

# Table of Contents


1. Introduction: CDB 2.X Key Concepts and Terminology . . . 11
   1.1. Background . . . 11
   1.2. Key Concepts . . . 11
      1.2.1. **Analysis Ready Data (ARD)** . . . 11
      1.2.2. **Authoritative data** . . . 11
      1.2.3. **Authoritative data source** . . . 12
      1.2.4. **Data store** . . . 12
      1.2.5. **Extension** . . . 12
      1.2.6. **Foundation data** . . . 12
      1.2.7. **Profile** . . . 12
      1.2.8. **Standardization target** . . . 13
2. Conformance . . . 14
3. References . . . 15
4. Conventions . . . 16
   4.1. Standards Document Terms . . . 16
   4.2. Identifiers . . . 17
   4.3. Use of HTTPS . . . 17
   4.4. HTTP URIs . . . 17
   4.5. CDB XML Schema Definitions . . . 18
      4.5.1. The CDB Namespace . . . 20
      4.5.2. Accessing the CDB global metadata files, controlled vocabularies, and enumerations. . . . 20
   4.6. CDB Metadata, Controlled Vocabulary, and Metadata Files . . . 21
      4.6.1. What is a Controlled Vocabulary? . . . 21
      4.6.2. What is an Enumeration? . . . 21
   4.7. A note CDB Directory File Naming and Structure . . . 22
5. Scope and Introduction - Informative . . . 23
   5.1. Scope . . . 23
   5.2. Introduction - High level CDB 2.0 Architecture . . . 23
      5.2.1. What is a datastore? . . . 24
      5.2.2. Foundation Standards . . . 25
      5.2.3. CDB 2 Core . . . 25
      5.2.4. Application Profiles . . . 25
   5.3. Metadata and the CDB Core . . . 26
   5.4. Overview of the CDB Storage Structure . . . 26
6. CDB 2.0 Core Requirements Classes . . . 28
7. Additional Parts - Future Work . . . 30
   7.1. Attribution . . . 30
      7.1.1. Special consideration . . . 31

7.1.2. Core Attributes Requirements Class . . . 31
7.1.2.1. Attribute Model . . . 31
7.1.2.2. URI - External loation of attribution schema . . . 31
7.1.2.3. Attribute Model Content . . . 32
7.2. Coverages . . . 32
7.2.1. Terms and definitions relevant to this module . . . 33
7.2.2. Coverages Requirements Class . . . 34
7.2.3. Requirements . . . 35
7.2.3.1. When to use coverage rule . . . 35
7.2.3.2. Consistent with Topic 06 . . . 35
7.2.3.3. Conformance with this Clause . . . 36
7.2.4. CRS Consistency . . . 36
7.2.5. Resource Metadata . . . 36
7.2.6. domainSet . . . 36
7.2.6.1. Units of Measure . . . 37
7.2.6.2. Using the Scale and Offset Values . . . 38
7.2.6.3. Grid Cell/Pixel Encoding . . . 38
7.2.6.4. field_name . . . 38
7.2.6.5. quantity_definition . . . 38
7.2.7. Tiling Requirements . . . 39
7.2.8. Tiling Extension . . . 39
7.3. Coordinate Reference System (CRS) Requirements Module . . . 39
7.3.1. CDB CRS Core Requirements Class and Requirements . . . 40
7.3.1.1. Abstract Specification (ISO Standard) . . . 40
7.3.1.2. CDB one CRS per data store instance . . . 41
7.3.1.3. CRS Type . . . 41
7.3.1.4. CRS Metadata . . . 42
7.3.1.5. Units of Measure of the CRS . . . 43
7.3.1.6. Data Store Epoch . . . 43
7.3.1.7. Vertical CRS - Optional Class . . . 44
7.3.1.8. Default vertical CRS units . . . 45
7.3.1.9. WKT for CRS Example . . . 45
7.4. Resource Path and File Naming Requirements Module . . . 46
7.4.1. CDB Resource Path/Naming Requirements Class . . . 46
7.4.2. Do not uses spaces in paths or names . . . 47
7.4.3. Do not use unicode . . . 47
7.4.4. Language Requirement and Recommendation on English . . . 48
7.4.5. Empty Folder Guidance . . . 48
7.4.6. Style Guide Requirement . . . 48
7.4.7. String case . . . 48
7.4.8. File name extensions . . . 49

7.5. File Hierarchy Structure . . . 50
7.5.1. CDB File Requirements Class . . . 50
7.5.2. Hierarchial file system . . . 51
7.5.3. Root location of a CDB datastore . . . 51
7.5.4. Hierarchy of file system/folder structure . . . 51
7.5.5. Name of root directory/folder . . . 51
7.5.6. Recommended Name of root directory/folder (CDB Landing Page) . . . 52
7.5.7. Global Metadata . . . 52
7.6. Geometry . . . 52
7.6.1. The Geometry Model . . . 52
7.6.2. Geometry Requirements Class . . . 57
7.6.2.1. Geometry Model . . . 57
7.6.2.2. Geometry types . . . 58
7.6.3. Geometry with a Z coordinate. . . . 58
7.6.4. Consistent SRS/CRS in a Geometry Collection . . . 59
7.7. Links . . . 59
7.7.1. Note on templated links . . . 61
7.8. Media Types . . . 61
7.8.1. Media Type for CDB Datastores . . . 61
7.9. CDB Global and Local Resource/Dataset Metadata . . . 62
7.9.1. What is metadata? . . . 62
7.9.2. CDB and Metadata . . . 62
7.9.3. CDB Global Datastore and Resource/Dataset Metadata . . . 63
7.9.3.1. Global Metadata . . . 64
7.9.3.2. Specify the metadata standard being used. . . . 64
7.9.3.3. Link/Path to Global Metadata . . . 65
7.9.3.4. Language for metadata . . . 65
7.9.3.5. Metadata encoding consistency . . . 65
7.9.3.6. Date and Time Metadata . . . 66
7.9.3.7. Temporal Intervals . . . 66
7.9.4. Global metadata for UoM . . . 66
7.9.4.1. Global Metadata elements . . . 67
7.9.4.2. Resource Metadata (aka dataset metadata) . . . 69
7.10. Tiling Requirements Module (Abstract) . . . 71
7.10.1. Tiling, LoD, and Layers Logical Model . . . 71
7.10.2. CDB Core Tiling Requirements Class and Requirements. . . . 72
7.10.2.1. When to tile rule . . . 73
7.10.2.2. Consistent with Topic 22 . . . 73
7.10.2.3. Conformance with this Clause . . . 74
7.10.2.4. Tiling Scheme Restrictions . . . 74
7.10.2.5. TilingScheme Definition . . . 75

7.10.2.6. TileSet Metadata . . . 75
7.10.2.7. Tiling/Lod Recommended Approach . . . 76
7.11. Tiling Extension (TE) - CDB Global Grid . . . 77
7.11.1. CDB Tiling and the OGC Two Dimensional Tile Matrix Set and Tile Set Metadata Standard (Informative) . . . 77
7.11.2. CDB 2.0 CDB1GlobalGridTiling Extension overview (Informative) . . . 78
7.11.3. Tiling Extension Requirements Class . . . 79
7.11.3.1. Conformance with this Clause . . . 79
7.11.3.2. Conformance with OGC Tile Matrix Standard . . . 80
7.11.3.3. Conformance with EPSG:4326 Coordinate Reference System . . . 80
7.11.3.4. Units of measure rule for tile orgins, tile boundaries, and so forth . . . 80
7.11.3.5. Level of Detail (LoD) level 0 rule . . . 80
7.11.3.6. LoD Hierarchy/Tessellation rule . . . 81
7.12. Tiling Extension (TCE) - GNOSISGlobalGrid . . . 81
7.12.1. CDB Tiling and the OGC Two Dimensional Tile Matrix Set and Tile Set Metadata Standard (Informative) . . . 81
7.12.2. CDB 2.0 GNOSISGlobalGrid Tiling Extension overview (Informative) . . . 82
7.12.3. Tiling Extension Requirements Class . . . 83
7.12.3.1. Conformance with this Clause . . . 84
7.12.3.2. Conformance with OGC Tile Matrix Standard and GNOSISGlobalGrid definition . . . 84
7.12.3.3. Conformance with EPSG:4326 Coordinate Reference System . . . 84
7.12.3.4. Units of measure rule for tile orgins, tile boundaries, and so forth . . . 84
7.12.3.5. Level of Detail (LoD) level 0 rule . . . 85
7.12.4. Additional Information . . . 85
7.13. Topology Core Requirements Module . . . 85
7.13.1. Topology concepts and terminology . . . 85
7.13.2. Topology, Geometry, and Features . . . 88
7.13.3. Crosswalk with 1.x Terminology . . . 88
7.13.4. Topology Core Requirements . . . 89
7.13.4.1. Conformant with ISO 19107:2017 (aka OGC Abstract Specification Topic 1) . . . 89
7.13.4.2. Unique Node Identifiers . . . 89
7.13.4.3. Unique Edge Identifiers . . . 90
7.13.4.4. Directed Nodes . . . 90
7.13.4.5. Directed Edges . . . 90
7.13.4.6. Faces - Recommendation . . . 90
7.13.4.7. Clipping edges at tile boundaries. . . . 91
7.13.5. Face Topology Primitive - Optional . . . 91
7.13.5.1. Relation of topological face to a polygon. . . . 91
7.13.5.2. Conformance with ISO 19107:2017 . . . 92
7.13.5.3. Unique Face ID . . . 92
7.13.5.4. Structure of a Face primitive . . . 93

7.13.5.5. Face winding order . . . 93
7.13.5.6. Islands/Holes . . . 93
7.13.6. An example logical model for topologically structured vector data . . . 93
7.14. Versioning . . . 93
7.14.1. Versioning Requirements Class . . . 94
7.14.2. A CDB datastore implementation shall support versioning . . . 94
7.14.3. Collection of changes . . . 95
7.14.4. Versioning metadata . . . 95
7.14.5. CRUD . . . 95
7.14.6. Minimum update capability - File (Asset) replacement . . . 96
7.14.7. Transitory (ephemeral) Assets . . . 96
Annex A: Abstract Test Suite (Normative) . . . 98
A.1. Introduction . . . 98
A.2. Conformance Class Core . . . 98
Annex B: Annex CDB Design Principles . . . 99
B.1. Detailed Design Principles and Objectives . . . 100
B.1.1. Design Objectives for CDB X Tiling, LoDs and Layers . . . 100
B.2. Design Objecctices for Metadata and Provenance . . . 100
B.3. Design principles from SOFWERX Sprint - CDB 2.0 Design Goals and Objectives . . . 101
B.4. Additional 2.0 Design Requirements . . . 101
Annex C: Annex C Description of the High Level Architecture . . . 102
C.1. Three tiered architecture . . . 102
C.1.1. Tier 1 . . . 102
C.1.1.1. Foundation Conceptual Models and Abstract Foundation Standards . . . 102
C.1.1.2. Tier 1 Logical Model . . . 103
C.1.1.3. Tier 1 CDB Metadata, Controlled Vocabularies, and Metadata Files . . . 104
C.1.2. Metadata . . . 104
C.1.2.1. Tier 1 Implementation Standards . . . 104
C.1.3. Tier 2 - Implementable requirements and conformance classes . . . 105
C.1.3.1. Tier Two Tiling Example . . . 105
C.1.4. Tier 3 . . . 106
C.1.4.1. Tier 3 Content Requirement Example . . . 106
C.2. CDB Application Profiles . . . 107
Annex D: Revision History . . . 109
Annex E: Bibliography . . . 110

## i. Abstract

CDB Version 2.0 is a major revision to the CDB Standard. The CDB Version 2.0: Core Standard specifies requirements (rules) defining a standardized model and structure for a single, versionable, virtual representation of the earth. The full CDB Standard is comprised of a fundamental core, Parts (extensions to the core), and profiles. The core consists of a suite of mandatory and optional requirements classes and conformance classes that are components in any extension, profile, and implementation. Core requirements can be profiled (restricted). An overarching design principle is that a CDB data store is focused on the ability to store, manage, and access extremely large volumes of geographic content and related assets such as model definitions in as deterministic manner as possible.

A CDB compliant structured data store provides a geospatial content and model repository that is plug-and-play interoperable between applications within a domain and potentially between domains. In the CDB context, a domain represents an information community, such as gaming, tactical analysis, or simulation, that has a common set of requirements for data content including access.

## ii. Keywords

The following are keywords to be used by search engines and document catalogues.

ogcdoc, OGC document, CDB, simulation, synthetic environment, virtual, gaming, 3D

## iii. Preface

This major revision to the OGC CDB Standard is the result of numerous OGC interoperability experiments, "sprints", SWG discussions, and general input and feedback from the OGC Membership, the CDB User base, and the broader Simulation Community. Further, there was a requirement for alignment of the CDB standard with the OGC/ISO standards baseline. In CDB Version 1.0, effort was invested to begin aligning terminology and concepts with the OGC Standards baseline, specifically in the coordinate reference system discussions and requirements. For CDB version 1.1, the primary effort was focused on resolving several Change Requests and adding guidance on incorporating metadata into a CDB data store. Version 1.2 provided requirements and best practices for using the OGC GeoPackage Standard to specify containers for CDB content. The CDB 2.0 major revision completes that trend.

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights. The Open Geospatial Consortium shall not be held responsible for identifying any or all such patent rights.

Recipients of this document are requested to submit, with their comments, notification of any relevant patent claims or other intellectual property rights of which they may be aware that might be infringed by any implementation of the standard set forth in this document, and to provide supporting documentation.

## iv. Submitting organizations

The following organizations submitted this Document to the Open Geospatial Consortium (OGC):

- CAE Inc.
- Carl Reed, OGC Individual Member
- FlightSafety

Organization name(s)

## v. Contributors and Endorsing Members

| Name | Affiliation | Role |
|---|---|---|
| Carl Reed | Carl Reed & Associates | Editor |
| David Graham | CAE | Editor and Contributor |
| William Aycock | The Joint staff | Contribtior |
| Kevin Bentley | Cognitics | Contributor |
| Paul Foley | KaDSci LLC | Contributor |
| Ryan Franz | FlightSafety | Contributor |
| Jérôme Jacovella-St-Louis | Ecere | Contributor |
| Chris Little | UK Met Office | Contributor |
| Greg Peele | Geometric Progress | Contributor |

Please submit your comments using the following email address: requests@list.opengeospatial.org. The link provided above should include a standard template in the message body. If the preloaded message body does not work properly using your mail client, please refer to the official template for the message body. Please direct comments to the editors named above.

## vi. Security Considerations

Security considerations for implementations of a CDB 2.0 compliant datastore are in the domain of the implementing application, deployment platform, operating system and networking environment. The CDB 2.0 Standard does not place any constraints on application, platform, operating system level, or network security.

## vi. Future work

The CDB community understands that additional standardization work is required to define and document additional Parts, extensions, profiles, and content appropriate to targeted applications, new use cases, and application in new domains.

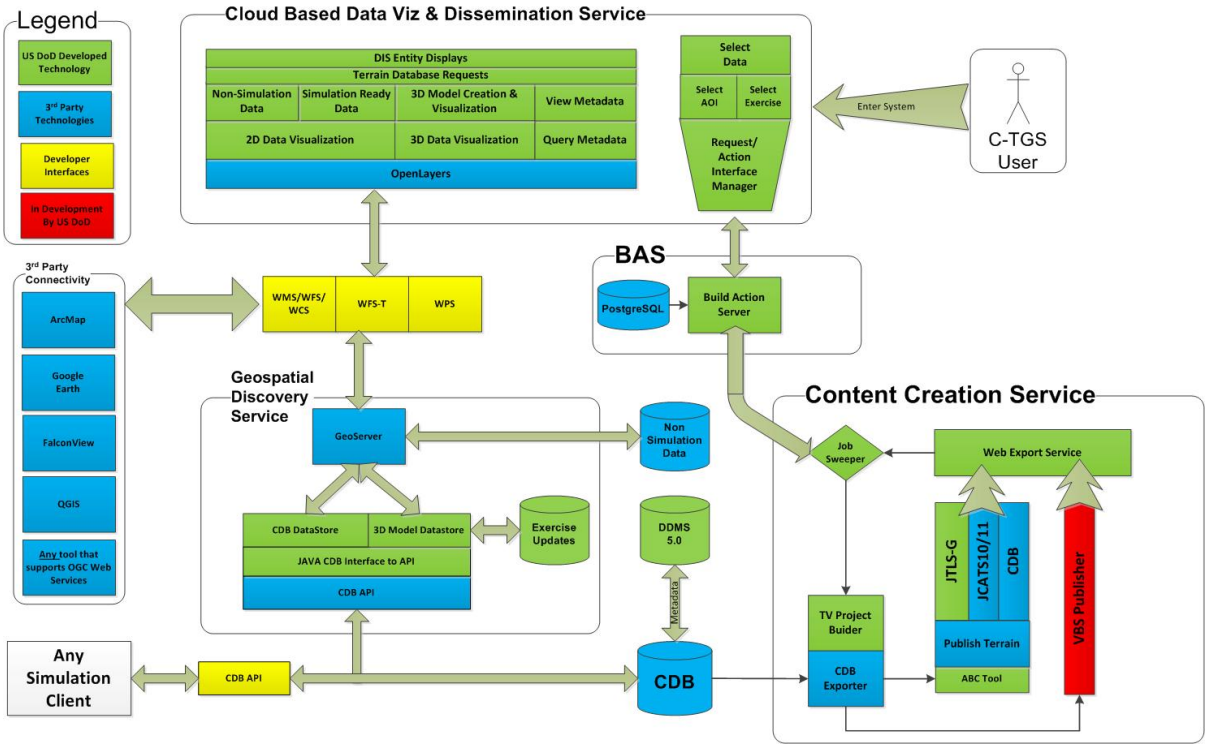
The requirements for a CDB data store are focused on the ability to store, manage, and access extremely large volumes of geographic content and related model definitions.

## vii. A note on using a CDB Data Store with OGC Standards

The U.S. Department of Defense (DoD) developed the Joint Training Data Services (JTDS) to support rapid scenario generation for the U.S. DoD M&S training community [1]. The recently developed (2014) and deployed JTDS Common Terrain Generation Service (C-TGS) is a cloud-based service for

the war fighter. C-TGS provides a robust capability to search, discover and retrieve Modeling and Simulation (M&S) terrain data. JTDS based the C-TGS standard data structure on OGC-approved formats served by OGC Web Services. The goal for the Terrain Generation Service (TGS): Eliminate the current paradigm where every simulation uses a different proprietary format. Current data efforts are fraught with simulation issues and errors, and data mining, correlation and processing to support multiple simulation formats is manpower intensive.

The TGS Solution: Have a repository and user interface that provides access to a single source of data through a persistent web service AND set and enforce a standard data structure based on industry standard Open Geospatial Consortium (OGC) approved formats, to normalize terrain data across Force Development simulations.




The above figure, DoD C-TGS Architecture (Chambers and Sherman, 2014), represents the implementation architecture for C-TGS. Implementation software represents a mix of commercial/proprietary and open source. At the heart of the C-TGS are two data stores: A CDB structured data store and a 3D model data store. All applications at the Visualization and Dissemination level interface the data stores via OGC service interfaces.

Connecting OGC Web Services to an open, simulation-optimized terrain data store, such as a CDB structured data store, results in high-end performance for serving and rendering geospatial content, the ability to dynamically modify terrain and support for all levels of simulation. a high-end simulation system using CDB can dynamically update the terrain (such as a bomb crater) and have that dynamically updated terrain instantly served by the Geospatial Discovery Service to any OGC compliant system, such as Esri® ArcMap, QGIS, or a mobile web page. The underlying architecture and framework of the C-TGS provide a mechanism to move high-end simulation to the point where data sharing, crowd-sourcing, and dynamic terrain are in-sync with GIS tools and simulation system edits of the terrain repository.

# Chapter 1. Introduction: CDB 2.X Key Concepts and Terminology

## 1.1. Background

This section details key concepts and terminology in the CDB 2.0 Standard and beyond.

## 1.2. Key Concepts

The following key concepts and definitions are critical to understanding how to read the CDB 2.X Standard, understanding the CDB architecture, and how to define implementable application profiles.

### 1.2.1. Analysis Ready Data (ARD)

Data that have been processed to a minimum set of requirements and organized into a form that allows immediate use with a minimum of additional user effort for further interoperability both through time and with other datasets. (source: CEOS http://ceos.org/document_management/Meetings/Plenary/30/Documents/5.5_CEOS-CARD4L-Description_v.22.docx). ARD is an abstract concept that can only be implemented in the context of a common domain of application. The CDB 1.x standard specifies how to structure, attribute, and curate geospatial (2d and 3d) data to make that data ready for the simulation domain of application.

Other characteristics of ARD include the following.

- There is an effort made by the producer to reduce the need for pre-processing by integrating the necessary correction(s) in the product production chain. These corrections are based on the rules (requirements) for generating ARD.
- Data are made available in easy-to-use formats. These are commonly known and well documented formats that are supported in multiple software products.
- The product is well documented (metadata) at the product level as well as other levels (such as a layer).

In terms of the architecture, ARD fits well with the requirements specified in an application profile. An application profile of the CDB Core would provide requirements for data content and structure for a given domain, such as simulation or tactical.

### 1.2.2. Authoritative data

Officially recognized data that can be certified and is provided by an authoritative source.

(SOURCE: Authority and Authoritative Sources: Clarification of Terms and Concepts for Cadastral Data. FGDC/NIST 2008)

### 1.2.3. Authoritative data source

An information technology (IT) term used by system designers to identify a system process that assures the veracity of data sources. These IT processes should be followed by all geospatial data providers. The data may be original or it may come from one or more external sources all of which are validated for quality and accuracy.

(SOURCE: Authority and Authoritative Sources: Clarification of Terms and Concepts for Cadastral Data. FGDC/NIST 2008)

### 1.2.4. Data store

A data store is a repository for persistently storing and managing collections of data which include not just repositories such as databases, but also simpler store types such as simple files, metadata, models, etc.

### 1.2.5. Extension

Requirements class which has a direct dependency on another requirements class.

| | |
|---|---|
| **NOTE** | Here extension is defined on requirements class so that their implementation may be software extensions in a manner analogous to the extension relation between the requirements classes. A common mechanism to extend the functionality of a specification is to define extensions, which may be either local or encompass other standards. Specifications should use extensions where feasible, but should never hinder them. |

### 1.2.6. Foundation data

Authoritative data and models that have been curated and stored in a CDB repository. Foundation data uses known and agreed to controlled vocabularies for attribution, has been curated based on the requirements as stated in the CDB standard or some other authoritative source, and supported by required metadata.

For example, as per the CDB 1.3 Standard (and industry best practice), a road network (RoadNetwork dataset in CDB 1.3) could be considered as foundation data. In CDB 1.3, there are a set of requirements associated with how the roads data are structured and attributed and stored in a CDB repository. The specification of any foundation data layer (such as RoadNetwork) at the logical model level can be 100% independent from the storage environment, end user use case(s), and so on while at the same time also having the requirements based on industry knowledge and use case requirements.

Any CDB 2.X foundation data requirements should not mention the actual storage structure, enabling software, storage formats, tiling structure, and so forth.

### 1.2.7. Profile

Specification or standard consisting of a set of references to one or more base standards and/or other profiles, and the identification of any chosen conformance test classes, conforming subsets,

options and parameters of those base standards, or profiles necessary to accomplish a particular function. [ISO/IEC TR 10000-1] In the usage of this standard, a profile will be a set of or extensions to requirements classes or conformance classes (either preexisting or locally defined) of the base standards.

In the CDB 2.0 Standard and later, the requirements and any related extensions are based on the use cases and interoperability requirements of a specific domain or information community. Some example Profiles are could be for tactical training, for gaming, and for flight simulation. All of these and any other defined profiles shall implement and be consistent with conformance classes as defined in the CDB Core Standard.

### 1.2.8. Standardization target

entity to which some requirements of a standard apply. (The Specification Model — A Standard for Modular specifications)

For the CDB Standard, the standardization target for the core would be any extension, application profile and implementation of a CDB data store conformant with the core. So, for example, the standardization target for an extension would be the core conformance class. Specifically, every requirements class in a standard shall have a standardization target type which is a subtype of that of the core and shall have the core as a direct dependency.

| | |
|---|---|
| **NOTE** | The standardization target is the entity which may receive a certificate of conformance for a requirements class. |

[1] This section is based on presentations and a paper on the JTDS C-TGS activity. Specifically, “Cloud Terrain Generation and Visualization Using Open Geospatial Standards, Chambers and Freeman, 2014. From proceedings of the Interservice/Industry Training, Simulation, and Education Conference.

# Chapter 2. Conformance

This CDB 2.0 Core Standard defines a minimal Abstract Test Suite in Annex A.

The standardization target type for all Core requirements modules and requirements is a `CDB Datastore`.

Conformance with this Standard shall be checked using all the relevant tests specified in Annex A (normative) of this document. The framework, concepts, and methodology for testing, and the criteria to be achieved to claim conformance are specified in the OGC Compliance Testing Policies and Procedures and the OGC Compliance Testing web site.

The CDB 2.0 Core is comprised of a set of requirements modules. Each requirements module specifies requirements and recommendations that an implementation of a CDB Application Profile may have to implement. The single conformance class in this Standard specifies the minimum set of requirements classes that SHALL be implemented in any CDB 2.0 and later CDB datastore.

The following are the Mandatory Requirements classes. Any (application) profile or profile with extensions of the CDB 2.0 and later core requirements classes SHALL specify conformance classes for these mandatory requirements classes.

*Table 1. Mandatory Requirements/Conformance Classes*

| **Requirements class** | **Suggested URI** |
|---|---|
| CRS | http://www.opengis.net/spec/CDB/2.0/conf/<name>/crs |
| File Naming | http://www.opengis.net/spec/CDB/2.0/conf/<name>file-naming |
| File Structure | http://www.opengis.net/spec/CDB/2.0/conf/<name>file-structure |
| Links | http://www.opengis.net/spec/CDB/2.0/conf/<name>/links |
| Metadata | http://www.opengis.net/spec/CDB/2.0/conf/<name>/metadata |

**NOTE** The `<name>` refers to the name of the application profile (profile or profile with extensions). For example, if the application profile specifies the requirements for a CDB datastore based on the OGC GeoPackage Standard, then `<name>` could be `gpkg`.

**NOTE** The CDB 2.0 Core Standard does not mandate a specific encoding or format for representing any content.

# Chapter 3. References

Normative References


- Internet Engineering Task Force (IETF). RFC 3339: **Date and Time on the Internet: Timestamps** [online]. Edited by G. Klyne, C. Newman. 2002 [viewed 2020-03-16]. Available at http://tools.ietf.org/rfc/rfc3339.txt
- Berners-Lee, T., Fielding, R., Masinter, L: **IETF RFC 3986, Uniform Resource Identifier (URI): Generic Syntax**, http://tools.ietf.org/rfc/rfc3986.txt
- Internet Engineering Task Force (IETF). RFC 8288: **Web Linking** [online]. Edited by M. Nottingham. 2017 [viewed 2020-03-16]. Available at http://tools.ietf.org/rfc/rfc8288.txt
- ISO 8601-1:2019, **Date and time - Representations for information interchange - Part 1: Basic rules**
- ISO/OGC: ISO 19111:2019 and OGC 18-005r4 **OGC Abstract Specification Topic 2: Referencing by coordinates**, 2019 http://docs.opengeospatial.org/as/18-005r4/18-005r4.html
- ISO/TC 211: ISO 19115-1:2014 **Geographic information — Metadata — Part 1: Fundamentals**, 2014 https://www.iso.org/standard/53798.html
- ISO/TC 211: ISO/WD 19123-1 **Geographic information — Schema for coverage geometry and functions — Part 1: Fundamentals**, 2005 https://www.iso.org/standard/70743.html
- Whiteside, A., Greenwood, J.: **OGC Web Services Common Standard**, version 2.0, OGC 06-121r9
- OGC: OGC 12-128r15, **GeoPackage Encoding Standard - with Corrigendum**, 2018 https://www.opengeospatial.org/standards/geopackage
- Herring, J.: OGC 06-104r4,**OpenGIS® Implementation Standard for Geographic information - Simple feature access - Part 2: SQL option**, http://portal.opengeospatial.org/files/?artifact_id=25354
- van den Brink, L., Portele, C., Vretanos, P.: OGC 10-100r3, **Geography Markup Language (GML) Simple Features Profile**, http://portal.opengeospatial.org/files/?artifact_id=42729
- Reed, C. 19-014r3, **OGC Abstract Specification Topic 22 - Core Tiling Conceptual and Logical Models for 2D Euclidean Space**, 2020-10-12, https://docs.ogc.org/as/19-014r3/19-014r3.html

# Chapter 4. Conventions

This section provides details and examples for any conventions used in the document. Examples of conventions are symbols, abbreviations, use of XML or JSON schema, or special notes regarding how to read the document.

## 4.1. Standards Document Terms

Many of the terms used in the CDB Standard, such as `requirement` and `requirements class`, are used to ensure that any OGC Standards document is specified as clearly and unambiguously as possible. For example, a `requirement` is an "expression in the content of a document conveying criteria to be fulfilled if compliance with the document is to be claimed and from which no deviation is permitted". The standards terms are defined in the OGC Specification Model — A Standard for Modular specifications [OGC-08-131r3]

As many readers of this Standard may not be familiar with the OGC Modular Specification, some key terms and definitions from the Mod Spec used in the CDB Standard are provided here.

`conformance test`: set of conformance tests that must be applied to be considered conformant. Typically each stated requirements class will have a related conformance class.

`core requirements class`: one of more unique requirements classes that must be satisfied by any conformant standardization targets associated to the standard. The CDB Core Standard specifies a related suite of core requirements classes group into requirements modules.

`direct dependency` (of a requirements class): another requirements class (the dependency) whose requirements are defined to also be requirements of this requirements class. An example in the CDB Core is that the Geometry Requirements Module has a dependency on the OGC Simple Features Standard which itself has a dependency on the ISO/OGC 19107, Geographic information  Spatial schema.

`extension` (of a requirements class): requirements class which has a direct dependency on another requirements class that may or may not be part of the core. In the CDB Standard, an extension is defined on requirements class so that their implementation may be software extensions in a manner analogous to the extension relation between the requirements classes.

`profile`: In the usage of this standard, a profile will be a set of requirements classes or conformance classes (either preexisting or locally defined) of the base standards. This is typically referred to as a restricted subset.

`recommendation`: expression in the content of a document conveying that among several possibilities one is recommended as particularly suitable, without mentioning or excluding others, or that a certain course of action is preferred but not necessarily required, or that (in the negative form) a certain possibility or course of action is deprecated but not prohibited.

`requirement`: expression in the content of a standard conveying criteria to be fulfilled if compliance with the standard is to be claimed and from which no deviation is permitted.

`requirements class`: aggregate of all requirement modules that must all be satisfied to satisfy a

conformance test class.

`requirements module`: aggregate of requirements and recommendations of a standard against a single standardization target type.

`standardization target`: entity to which some requirements of a standard apply. In CDB, the standardization target is a `CDB Compliant Data Store`.

## 4.2. Identifiers

The normative provisions in this standard are denoted by the URI namespace

```
http://www.opengis.net/spec/cdb/2.0/core
```

All requirements that appear in this document are denoted by partial URIs which are relative to the namespace shown above.

For the sake of brevity, the use of “req” in a requirement URI denotes:

```
http://www.opengis.net/spec/cdb/2.0
```

An example might be:

/req/core/crs

All conformance tests that appear in this document are denoted by partial URIs which are relative to the namespace shown above.

For the sake of brevity, the use of “conf” in a requirement URI denotes:

```
http://www.opengis.net/spec/cdb/2.0/core
```

## 4.3. Use of HTTPS

For simplicity, this document in general only refers to the HTTP protocol. This is not meant to exclude the use of HTTPS and simply is a shorthand notation for "HTTP or HTTPS." In fact, most servers are expected to use HTTPS, not HTTP.

## 4.4. HTTP URIs

This document does not restrict the lexical space of URIs used in the API beyond the requirements of the HTTP and URI Syntax IETF RFCs. If URIs include reserved characters that are delimiters in the URI subcomponent, these have to be percent-encoded. See Clause 2 of RFC 3986 for details.

# 4.5. CDB XML Schema Definitions

Historically, the CDB standard made extensive use of XML. XML was used to:

- Describe CDB metadata,
- Encode controlled vocabularies and enumerations,
- Encode global metadata,
- Add attributes,
- Add information to 3d models, such as an OpenFlight model,
- To describe base and composite materials,
- And other key information.

In the CDB 2.0 Core modules, specific encoding technologies such as XML or JSON are not specified. The Core requirements are agnostic to any particular encoding. This is because metadata, vocabularies, enumerations, and so forth could be encoded in any number of technologies including database tables. As such, CDB application profiles restrict and define which encoding technology is used.

Regardless of the encoding technology, CDB 2.0 (and later) still provides a number of controlled vocabularies and enumerations. These were integral to the success of earlier CDB versions and are provided in version 2.x as optional but recommended. In CDB 1.2 and earlier, these files were located in the `\CDB\Metadata\Schema` subdirectory of a CDB datastore.

In CDB 2.x, these vocabularies may be referenced by URI, stored in a database table, or stored in some file in a file system (including the Cloud). The traditional XML metadata and controlled vocabulary schema and files are provided as part of the CDB 2.x Standard. They are identical to those provided in CDB 1.3.

The schemas provided with the CDB Standard that can be used as is, replaced, or extended are:

| **Vocabulary/Schema Name** | O/C/M | **Description** |
|---|---|---|
| BaseMaterialTable.xsd | C | Defines the format of the CDB Base Material Table |
| Materials.xml | C | The CDB Base Material Table. This file contains the base material names for a CDB datastore. |
| CompositeMaterialTable.xsd | C | Defines the format of a CDB Composite Material Table (CMT) - a collection of substrates each made of base materials. |
| Configuration.xsd | O | Defines the configuration of one logical CDB |

| | | |
|---|---|---|
| Configuration.xml | O | An example is provided with the CDB standard of what could be in this file. |
| Datasets.xsd | M | XML Schema to define and validate the content of Datasets.xml |
| Datasets.xml | M | Dataset type enumeration |
| Defaults.xsd | O | XML Schema to define and validate the content of Defaults.xml |
| Defaults.xml | O | Defaults values for key attributes or data layers in a CDB datastore. |
| DIS_Country_codes.xsd | C | Schema to define and validate the content of the country code controlled vocabulary |
| DIS_Country_codes.xml | C | ISO COuntry Codes as used in CDB version 1.2 |
| FeatureDataDictionary.xsd | M | Defines the format of the CDB Feature Data Dictionary. |
| FeatureDataDictionary.xml | M | DIGEST Feature Data Dictionary |
| Globalgeometadata.<ext> | O | Global geospatial metadata |
| Lights .xsd | C | Schema to define and validate the content of the CDB Light Names Hierarchy |
| Lights.xml | C | Lights Hierarchy as defined and use in CDB version 1.3. |
| LightsTuning.xsd | O | Schema to define and validate the content of a CDB Light Tuning enumeration. |
| LightsTuning.xml | O | Example of the type of parameters that can be specified for light tuning visualization and modelling. |
| Globalgeometadata.<ext> | O | Global geospatial metadata |
| Model_Components.xsd | C | Schema to define and validate CDB Model Components. |
| Model_Components.xml | C | Sample list of Model Components provided with CDB 1.3 |

| | | |
|---|---|---|
| Moving_Model_Codes.xsd | C | Schema to define and validate the content of the moving model code enumeration. |
| Moving_Model_Codes.xml | C | Moving model codes as provided in CDB 1.3 |
| Model_Metadata.xsd | O | Schema defines the metadata associated with 3D models. |
| Vector_Attributes.xsd | O | Schema to define and validate CDB, Geomatics, and Vendor Vector Attributes as defined in Clause 10 of CDB Volume 1 |
| Version.xsd | M | All CDB Versions are identified by a file called *Version.xml*. This schema defines the format of this file. |

| | |
|---|---|
| **NOTE** | These files/schema have a download location of - **Need download location once we know**. |
| **NOTE** | Type: CV = Controlled Vocabulary, M = Metadata, E = Enumeration |
| **NOTE** | M/O: [M = Mandatory, O = Optional, C = Conditional] Conditional indicates that if a specific optional requirements module, such as Lights, is specified in an application profile than that specific controlled vocabulary or enumeration SHALL be used (although modified and.or extended as required) |
| **NOTE** | <ext> could be xml for XML, jsn for JSON, and other extensions based on the encoding technology used for the geospatial metadata |

Each of these files is described in more detail later in either a specific core requirements module, such as for Lights, or in a specific application profile.

How these schemas are provided and accessed in a physical implementation of a CDB datastore is specified in the requirements clauses of a CDB Application Profile. However, the CDB 2.x Standard specify requirements and recommendations for naming, accessing, and processing these schemas.

### 4.5.1. The CDB Namespace

The CDB standard makes use of namespaces to isolate the definitions of required schemas. The name of these namespaces is built around the base CDB URL.

### 4.5.2. Accessing the CDB global metadata files, controlled vocabularies, and enumerations.

The following is in reference to how global metadata, vocabularies, and enumerations are provided

as part of the CDB Standard on the OGC website. These schemas can be used as the source to populate the global metadata folder in a CDB datastore.

The target namespace of all CDB schemas for vocabularies and enumerations follow this pattern:

"http://opengis.net/cdb/Version/Name

Where the Name is identical to the filename or URI portion of the file containing the schema and Version is the version number of the schema.

To illustrate how a target namespace/URI is composed, here is the target namespace for the Lights vocabulary provided as XML schema:

"http://opengis.net/cdb/2.0/Lights.xml"

## 4.6. CDB Metadata, Controlled Vocabulary, and Metadata Files

There are a number of CDB files that are stored and referenced from the CDB metadata folder. First, some terms are defined

In addition to "traditional metadata" for describing geospatial and other resources, CDB implementations make extensive use of one or more additional controlled vocabularies and enumerations. In CDB 1.1 and earlier, all of these datasets were referred to as metadata. From CDB version 1.2 and later, a distinction is made between metadata about a resource, such as a description, and controlled vocabularies/enumerations.

### 4.6.1. What is a Controlled Vocabulary?

Controlled vocabularies provide a way to organize knowledge for subsequent retrieval and use. They are used in subject indexing schemes, subject headings, thesauri, taxonomies and other forms of knowledge organization systems. Controlled vocabulary schemes mandate the use of predefined, authorized terms that have been preselected by the designers of the schemes, in contrast to natural language vocabularies, which have no such restriction. The use of controlled vocabularies in standards such as CDB can significantly increase interoperability and consistent understanding of the semantics. Controlled vocabularies typically are managed through formal processes and official governance.

### 4.6.2. What is an Enumeration?

In computer programming, an enumerated type (also called an enumeration) is a data type consisting of a set of named values called elements, members, or enumerators of the type. The enumerator names are usually identifiers that behave as constants in the language. Similarly, in a database enumerated (enum) types are data types that comprise a static, ordered set of values. They are equivalent to the enum types supported in a number of programming languages. An example of an enum type might be the days of the week, or a set of status values for a piece of data.

# 4.7. A note CDB Directory File Naming and Structure

A CDB datastore directory and folder structure is defined by a combination of requirements defined:

- In the File Naming Requirements Class and the File Structure Requirements Class. All content in a single physical CDB datastore instance shall be accessible via the root location structured as sub-directories, folders, or some other hierarchial structure.
- Additional structure and naming requirements as defined in any CDB 2.0 application profiles or profiles with extensions.

Please note that these requirements can be extended to meet any specific domain or community specific requirements.

The following file extensions are used:

| File Format | Minimal Version Number | Extension |
|---|---|---|
| TIFF | 6.0 | *.tif |
| SGI Image | 1.0 | *.rgb |
| JPEG 2000 | 1.0 | *.jp2 |
| OpenFlight | 16.0 | *.flt |
| Shapefile | Esri White Paper, July 98 | *.shp, *.shx |
| dBASE | III+ | *.dbf, *.dbt |
| XML | 1.0 and later | *.xml, *.xsd |
| ZIP | 6.3.1 and later | *.zip |
| GeoPackage | 1.1 and later | *.gpkg |

# Chapter 5. Scope and Introduction - Informative

## 5.1. Scope

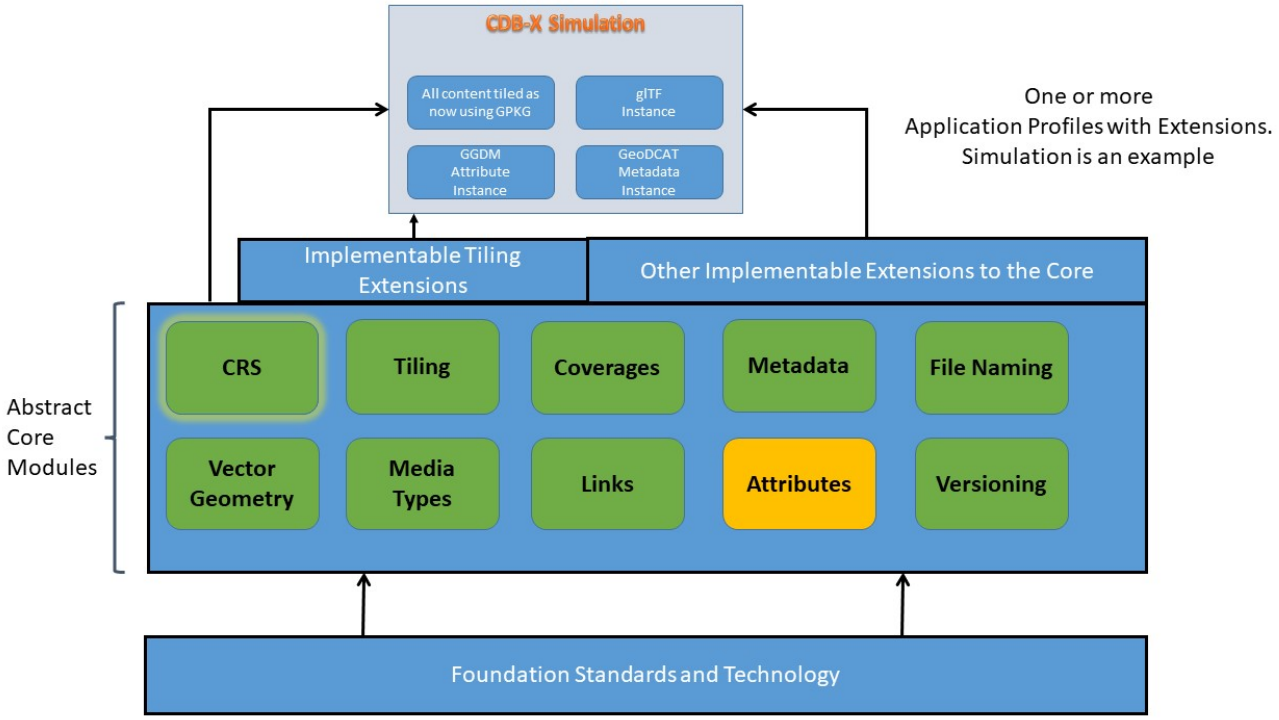
The CDB 2.0 Core Standard defines the mandatory and optional requirements, recommendations, and conformance classes that must be profiled or profiled and extended in any CDB 2.0 and later compliant application profile or implementation. In the CDB 2.0 Standard, requirements and recommendations are organized as aggregates in requirements modules. Some requirements modules, such as Topology, are optional. However if an application profile mandates the use of the Topology requirements module and related conformance classes, then that application profile shall be conformant with the Core Topology Requirements Module.

**NOTE** The CDB 2.0 Core is abstract and cannot be directly implemented. Therefore, application profiles need to be specified that restrict or restrict and extend requirements that can actually be implemented in an operational environment. For example, the CRS Requirements Module does not specify what CRS should be used for a given CDB datastore. An application profile would clearly state which CRS, such as EPSG 4326, is to be used for a CDB datastore instance. More details are provided below.

## 5.2. Introduction - High level CDB 2.0 Architecture

The following figure portrays, at a high level, the CDB 2.0 architecture.

**NOTE** This diagram is abstract and does not incorporate all aspects and requirements classes defined in the CDB 2.0 Core.




The OGC CDB 2.0 Standard is comprised of a fundamental core and multiple Parts, each Part being a separate standard. This part, the "Core," specifies the core capabilities and is restricted to being able to define a CDB datastore where geometries and coverages represented in a given CRS represented can be structured, referenced, and stored. Additional capabilities that address more advanced requirements will be specified in additional Parts. Examples include support for Materials, Lights, and textures.

**NOTE** The Core as specified cannot be directly implemented. The specification of profiles and profiles with extensions are required to provide a requirements framework that can be implemented. That said, the fundamental core does provide and specify two tiling extensions that can be implemented.

### 5.2.1. What is a datastore?

The following is from Technopedia.

A `datastore` is a repository for storing, managing and distributing datasets typically at an enterprise level. `Datastore` is a broad term that incorporates all types of data that is produced, stored and used by an organization. The term references data that is at rest and used by one or more data-driven applications, services, or individuals.

A datastore may include data from end user database applications, files or documents, or the random data property of an organization or an information system. Data may be structured, unstructured, or in another electronic format.

Depending on the organization, a datastore may be classified as an application-specific datastore, operational datastore or centralized datastore. Moreover, a datastore may be designed and implemented by using purpose-built software or through typical database applications.

### 5.2.2. Foundation Standards

The CDB 2.0 Standard is grounded in a number of OGC Abstract Specifications, ISO TC 211 91xxx series International Standards, and standards from other Standards Development Organization such as the W3C and the IETF. The OGC Abstract Specification and ISO 19xxx Standards provide the conceptual foundation for many OGC standards development activities. They also provide a conceptual (abstract) foundation for many other geospatial standards development activities in the IETF, W3C, OASIS, and a large number of government organizations. Open interfaces and protocols can be designed, built, and referenced against the Abstract Specification, thus enabling interoperability between different platforms, applications, and different spatial processing systems. The Abstract Specification provides a reference model for the development of standards that can be implemented.

### 5.2.3. CDB 2 Core

The CDB Core consists of core requirements modules and corresponding conformance classes that any extension, application profile, or implementation of the CDB Standard shall be conformant with. For example, any implementation of an Application Profile (aka Profile) shall inherit and be dependent on the conformance classes as defined and implemented in the Core. In the above diagram, several requirements modules are mandatory. Any application profile (including extensions) shall conform with the requirements stated in those modules. However, if an optional core module is specified in an application profile (including extensions) then that profile shall conform with the requirements stated in those modules.

The CDB Fundamental Core is comprised of a set of abstract requirements modules and related conformance modules. These requirements modules are defined by an aggregate of requirements and recommendations that are abstract in the sense that they cannot be directly implemented. Instead, they need to be profiled or profiled with extensions in order to be implemented. However, any profile, profile with extensions, or implementation of any given requirements class shall be conformant with that core requirements class.

Another key design concept of the Core is that each specified requirements module - as defined by one or more requirements classes - has no dependencies on any other core requirements module. For example the CRS Core Requirements Module has no dependencies on Geometry and visa-versa. The only exception is the metadata core module. More on that below.

| **NOTE** | There are very few interdependencies between the core modules. The only exceptions are related to metadata and coordinate reference systems. |
|---|---|

### 5.2.4. Application Profiles

The CDB 2.0 core is not implementable. While the core consists of requirements classes and related conformance classes, profiles that restrict any given requirement or requirements module are necessary to create a set of requirements and conformance classes that can be implemented. In CDB 2.0, these are called *application profiles*.

For example, CDB 2.0 Core specifies a Coordinate Reference System (CRS) Requirements Module]. This requirements Module does not mandate which CRS is to be used for a given implementation

instance or profile. Instead, that restriction would be specified in a profile or profile with extensions. The CRS requirements module also specifies requirements such as that the ISO/OGC Well Known Text for CRS (WKT for CRS) shall be used to encode the CRS metadata for storage in a CDB datastore.

## 5.3. Metadata and the CDB Core

The majority of the requirements modules specified in the CDB Core are metadata centric. For example the requirements modules for tiling, feature attribution, or coordinate reference systems are focused on metadata related to the actual tiling structure or the CRS of a given CDB datastore. There are also requirements modules specifically focused on is considered more traditional metadata such as metadata for a given resource (dataset). Additional Parts to the CDB Standard will specify requirements modules related to specific controlled vocabularies such as Lights and Navigation Aids. Again, however, in a broad sense a controlled vocabulary is also metadata.

The determinism design principle is one reason for the focus on metadata in the CDB Core. Metadata elements defined in an application profile of the Core, such as for tiling, feature attribution, and the CRS, provide a well known and documented deterministic structure for both service and client access.

Note: The CDB Core does not address specific metadata required for a given resource or dataset type. For example, the Core does not provide requirements or guidance on all imagery metadata other than a few key metadata elements required for discovery.

## 5.4. Overview of the CDB Storage Structure

The CDB 2.0 specified storage structure supports efficient searching, retrieval and storage of any information contained within a CDB structured data store. Storage structure aspects include descriptions of each information field used within CDB conformant files, including data types and data type descriptions. The CDB Standard relies on three important concepts to organize a CDB compliant datastore.

- Tiles: The tiling structure specifies how to geographically divide the world into geodetic tiles (each tile bounded by latitude and longitude), each tile containing a specific set of features (such as terrain altimetry, vectors) and models (such as 3d models and radar cross section models), which are in turn represented by individual datasets. The datasets define the basic storage unit used in a CDB data store. The geographic granularity is at the tile level while the information granularity is at the dataset level. As a result, the CDB storage structure permits flexible and efficient updates due to the different levels of granularity with which the information can be stored or retrieved
- Layers: The CDB model is also logically organized as distinct layers of information. The layers are notionally independent from each other (i.e., changes in one layer do not impose changes in other layers).
- Levels-of-Detail (LODs): The use of LOD representations is critical for high performance. Most client-devices can readily take advantage of an LOD structure because, in many cases, less detail/information is necessary at increasing distances from the user perspective. As a result, many client-devices can reduce (by 100-fold or more) the required bandwidth to access data in

real-time. The availability of levels-of-detail permits client-devices to deal with data stores having big-data levels of content. The CDB storage model supports a LOD hierarchy consisting of up to 34 LODs. The CDB Standard requires that each geographic area be reduced into a LOD hierarchy in accordance to the availability of source data.

The CDB Standard does not define or enforce an operating system or file system.

# Chapter 6. CDB 2.0 Core Requirements Classes

This clause specifies the requirements classes that comprise the abstract core of the CDB-2.0 Standard. The core requirements classes are unique requirements classes that must be satisfied by any conformant standardization targets associated with the CDB 2.0 and later Standard. Standardization targets may be extensions to this Standard, application profiles based on the core, and implementations of the Standard.

Some of the requirements classes are mandatory (M) for any application profile and implementation. Other requirements classes are optional (O), but any application profile or implementation using the optional requirements classes must be conformant with those classes.

| Requirements Class | M/O | Description |
|---|---|---|
| Attribution | O | Requirements for encoding and storing attributes in a CDB datastore. |
| Coverages | O | Requirements for specifying coverages in a CDB datastore. |
| Coordinate Reference System (CRS) Requirements Module | M | Requirements for the coordinate reference system of a CDB datastore. |
| Resource Path and File Naming Requirements Module | M | Requirements for naming assets in a CDB datastore. |
| File Hierarchy Structure | M | Requirements for the file structure in a CDB datastore. |
| Geometry | O | Requirements for specifying geometry in a CDB datastore. |
| Links | M | Specifies requirements for how links are structured. |
| Media Types | O | Specifies enumerations of CDB 2.0 Media Types. |
| CDB Global and Local Resource/Dataset Metadata | M | Specifies requirements classes and requirements for global and resource metadata used in a CDB datastore. |

| Requirements Class | M/O | Description |
|---|---|---|
| Tiling Requirements Module (Abstract) | O | Specifies requirements classes and requirements for tiling a CDB datastore at the abstract level. |
| Tiling Extension (TE) - CDB Global Grid | O | Requirements module for implementing the CDBGlobalGrid tiling extension. |
| Tiling Extension (TCE) - GNOSISGlobalGrid | O | Requirements module for implementing the GNOSISGlobalGrid tiling extension. |
| Topology Core Requirements Module | O | Specifies requirements for topology primitives used in a CDB datastore. |
| Versioning | O | Specifies requirements for versioning used in a CDB datastore implementation. |

# Chapter 7. Additional Parts - Future Work

The OGC CDB 2.0 Standard consists of a fundamental core of mandatory and optional requirements classes and a number of additional Parts. These additional Parts define the requirements classes for such features as Materials and Textures. These Parts have been identified as future work. Many are in draft form but work will not be completed until the fundamental core is approved as an official OGC Standard.

| Module | O/M | Description |
|---|---|---|
| 3D Models | O | Requirements for specifying 3D Models in a CDB datastore. |
| Lights | O | Requirements for the structure of a lights controlled vocabulary and how new Light types and related metadata can be added to the Lights controlled vocabulary. |
| Generation of Materials | O | Specifies core requirements for physical materials. |
| Textures | O | Specifies the requirements for textures in a CDB datastore. |

## 7.1. Attribution

In the OGC CDB 2.0 and later Standards, each feature can be described by a set of attributes. Attributes are typically nonspatial information about a geographic feature in a GIS, usually stored in a table and linked to the feature by a unique identifier. For example, attributes of a river might include its name, length, and sediment load at a gauging station.

An attribution schema is used to clearly define the characteristics, structure, and internal relationships for attributes for a feature collection. Attribution schemas can be very simple (e.g. and ID and a name) or quite complex (e.g. NSG Application Schema (NAS)). Attribution schemas are the method to handle different types of attributes. CDB attribution schemas files are encoded using either the Extensible Markup Language (XML) or JSON formats. Quite often, intnerational standards exist for classification schemes and their related attribute schemas. Examples are:

- The Land Cover Classification System (LCCS) developed by FAO to provide a consistent framework for the classification and mapping of land cover.
- Classification of Wetlands and Deepwater Habitats of the United States
- NSG Application Schema (NAS)

### 7.1.1. Special consideration

A CDB 2.0 compliant datastore can be implemented using the Attribution rules and requirements as specified in the CDB 1.3 Standard. An example of this approach is provided in the CDB Version 2.0 GeoPackage Profile for CDB 1.x Datastores Standard.

### 7.1.2. Core Attributes Requirements Class

If the CDB 1.3 approach to attribution is not desired, then the following requirements class should be used.

The following is the CDB 2.0 Attributes Requirements Class.

| **Requirements Class - Attribution** | |
|---|---|
| /req/core/attributes | |
| Target type | Operations |
| Dependency | Classification scheme or controlled vocabulary |
| Requirement Attr1 | /req/core/attribute-model |
| Requirement PAttr1 | /per/core/attribute-schema-uri |
| Requirement Attr2 | /req/core/attribute-model-content |

#### 7.1.2.1. Attribute Model

An attribute or classification scheme needs to specified. This scheme will be used for all feature data in a given CDB datastore.

| **Requirement Attr1** | **/req/core/attribute-model** |
|---|---|
| A | Any CDB implementation, profile, extension, or application profile specifying and/or implementing attribution for features *SHALL* specify an attribute model. Examples are provided above. |
| B | The attribute model information *SHALL* be stored in the `global_metadata` folder as defined in CDB File Hierarchy Structure Core Requirements Class |
| C | The name of the attribute model schema file *SHALL* be `vector_attributes`.<ext> where <ext> is either `xml` or `json`. |

#### 7.1.2.2. URI - External loation of attribution schema

The following is a permission that a URI may be used to specify the location of the attribution schema.

| **Permission PAttr1** | **/per/core/attribute-model** |
|---|---|

| | |
|---|---|
| A | The `vector_attributes` file *MAY* contain a URI that links to the external location of the attribute schema. |

#### 7.1.2.3. Attribute Model Content

A minimal amount of information needs to be contained in the attribution schema.

| **Requirement Attr2** | **/req/core/attribute-model-content** |
|---|---|
| A | Any CDB attribute schema *SHALL* at a minimum specify elements (fields) as defined below. |
| B | Each attribute in the model *SHALL* have a unique identifier. |
| C | Each attribute in the model *SHALL* have a name. |
| D | Each attribute in the model *SHALL* have a description. |

An example of the mandatory information elements is:

| ID | Name | Description |
|---|---|---|
| 1 | StreetName | Name of a street as an alphanumeric string |
| 2 | StreetType | Type street as an alphanumeric string (Interstate, Arterial, . . .) |
| 3 | StreetWidth | Width of street in feet as an integer number) |

## 7.2. Coverages

In the CDB 1.x series of Standards, raster datasets, elevation models, and other raster/gridded data sets were an integral component of any CDB datastore. As such, the use of these data types is of high importance in any CDB 2.0 and later datastore.

The standard term for these data types is `coverage`. A coverage is a feature that acts as a function to return values from its range for any direct position within its spatiotemporal domain, as defined in OGC Abstract Topic 6: Schema for coverage geometry and functions

Coverages are represented by some binary or ASCII serialization, specified by some data (encoding) format. Arguably the most popular type of coverage is that of a regular gridded coverage. Gridded coverages have a grid as their domain set describing the direct positions in multi-dimensional coordinate space, depending on the type of grid. For example, satellite imagery and elevation models are typically modeled as a gridded coverage.

Coverages can be encoded in any suitable format such as:

1. The OGC/ISO Geography Markup Language (GML),
2. The OGC CoverageJSON Community Standard,
3. The OGC GeoTIFF Standard,

4. TIFF,
5. PNG
6. The OGC NetCDF Standard.

Coverage content can also be stored in a database oriented datastore, such as defined in the OGC GeoPackage Tiled Gridded Coverage Standard.

Coverages can be partitioned, such as for a time-interleaved representation. Coverages are independent from service definitions and, therefore, can be accessed through a variety of OGC services types, such as the Web Coverage Service (WCS) Standard or an implementation instance of the OGC API - Coverages Standard. The coverage structure can serve a wide range of coverage application domains, thereby contributing to harmonization and interoperability between and across these domains.

The CDB 2.0 Coverages core requirements module is abstract and optional. However, any extension, application profile or implementation that requires use of coverage data SHALL (Requirement 1 below) comply with the requirements stated in this module.

### 7.2.1. Terms and definitions relevant to this module

For the purposes of this document, the following additional terms and definitions apply. The definitions for “height” and “depth” are meant to be general and are derived from ISO 19111. However, the reader should note that there ma be domain specific variations or enhancements of the definitions used in this standard.

**Continuous coverage**

> **coverage** that returns different values for the same feature attribute at different **direct positions** within a single spatial object, temporal object or spatiotemporal object in its domain. [ISO 19123]

| | |
|---|---|
| **NOTE** | A continuous (grid) coverage has values not only at the direct positions themselves, but also at any location between the direct positions. In other words, an application can apply interpolation methods to obtain values between direct positions. |

**Coordinate Reference System (CRS)**

> A coordinate system that is related to the real world by a datum. [ISO 19111]

**Coverage**

> A coverage is a function that describe characteristics of real-world phenomena that vary over space and/or time. Typical examples are temperature, elevation and precipitation. A coverage is typically represented as a data structure containing a set of such values, each associated with one of the elements in a spatial, temporal or spatiotemporal domain. Typical spatial domains are point sets (e.g. sensor locations), curve sets (e.g. contour lines), grids (e.g. orthoimages, elevation models), etc. A property whose value varies as a function of time may be represented as a temporal coverage or time-series [*ISO-19109*].

| | |
|---|---|
| **NOTE** | A feature that acts as a function to return values from its range for any direct position within its spatiotemporal domain, as defined in OGC Abstract Topic 6 [ISO |

19123].

**Discrete Coverage**

Coverage that returns the same feature attribute values for every direct position within any single spatial object, temporal object, or spatiotemporal object in its domain [ISO 19123/OGC Topic 6]

**Depth**

Distance of a point from a chosen reference surface measured upward along a line perpendicular to that surface. [ISO 19111]

**NOTE**

Note 1: The line direction may be straight, or be dependent on the Earth's gravity field or other physical phenomena.
Note 2: A depth above the vertical reference surface will have a negative value.

**Direct Position**

The position described by a single set of **coordinates** within a **coordinate reference system**. [ISO 19123]

**Elevation**

Synonym for "height"

**Grid**

Grid network composed of two or more sets of curves in which the members of each set intersect the members of the other sets in an algorithmic way. NOTE: The curves partition a space into grid cells. [ISO 19123]

**Grid point**

Point located at the intersection of two or more curves in a grid. [ISO 19123:2005]

**Height**

Distance of a point from a chosen reference surface measured upward along a line perpendicular to that surface. [ISO 19111] Note 1 to entry: A height below the reference surface will have a negative value, which would embrace both gravity-related heights and ellipsoidal heights.

**Irregular grid**

A grid whose grid lines have individual distances along each grid axis.

**Regular grid**

Grid whose grid lines have a constant distance along each grid axis

### 7.2.2. Coverages Requirements Class

| **Requirements Class - Coverages** |
| --- |
| /req/core/coverages- |

| | |
|---|---|
| Target type | Operations |
| Dependency | ISO 19123/OGC Abstract Spec Topic 6 |
| Dependency | cdb-core-crs-requirements-class.adoc[CDB Core CRS Requirements Module] |
| Dependency | CDB Core Metadata Requirements Module |
| Requirement Coverages1 | /req/core/coverages-rule |
| Requirement Coverages2 | /req/core/coverage-topic6 |
| Requirement Coverages3 | /req/core/coverages-conformance |
| Requirement Coverages4 | /req/core/coverages-crs |
| Requirement Coverages5 | /req/core/coverages-min-metadata |
| Requirement Coverages6 | /req/core/coverages-domainSet |
| Recommendation Coverages7 | /req/core/coverages-tiling |
| Recommendation Coverages8 | /req/core/coverages |

## 7.2.3. Requirements

### 7.2.3.1. When to use coverage rule

If this Coverages Core Requiments Module is used, then the following requirement applies.

| **Requirement Coverages1** | **/req/core/coverage-rule** |
|---|---|
| A | Any coverage (such as elevation) content *SHALL* be encoded according to the requirements specified in this CDB 2 abstract coverage requirements module. |

### 7.2.3.2. Consistent with Topic 06

This Coverages requirements module has a dependency to OGC Abstract Specification Topic 6: Schema for coverage geometry and functions. This means that all terms and definitions and coverage models related to coverages in the CDB 2.0 Standard shall comply with Topic 6.

| **Requirement Coverages2** | **/req/core/coverage-topic06** |
|---|---|

| | |
|---|---|
| A | Any CDB implementation, profile, extension, or application profile that implements or references coverages *SHALL* be consistent with and conform to the terms and definitions and logical models as defined in OGC Abstract Specification Topic 6: Schema for coverage geometry and functions. |

#### 7.2.3.3. Conformance with this Clause

| **Requirement Coverages3** | **/req/core/coverage-conformance** |
|---|---|
| A | Any CDB implementation, profile, extension, or application profile that implements or references coverages *SHALL* be consistent with and conform to all other requirements specified in this Coverages Abstract Requirements Module. |

### 7.2.4. CRS Consistency

The CRS for any Coverage content in a CDB datastore has to be the same as specified by the global CRS metadata defined as per the CRS Requirements module.

| **Requirement Coverages4** | **/req/core/coverage-crs** |
|---|---|
| A | Any CDB implementation, profile, extension, or application profile that implements or references coverages *SHALL* be consistent with and conform to all other requirements specified in CDB Core Coordinate Reference System Module. |

At a minimum any Coverage content must have a minimum set of resource (dataset) metadata as defined in the CDB Core Metadata Module.

### 7.2.5. Resource Metadata

| **Requirement Coverages5** | **/req/core/coverage-min-metadata** |
|---|---|
| A | Any given coverage instance in a CDB datastore *SHALL* have resource metadata (dataset) elements as specified in the CDB Core Metadata Requirements Module, Resource Metadata sub-clause. |

### 7.2.6. domainSet

| **Requirement Coverages6** | **/req/core/coverage-domainSet** |
|---|---|
| | Any given coverage instance in a CDB datastore *SHALL* have the following domainSet metadata elements. |

| | |
|---|---|
| A | UoM - Units of Measure for values in the grid cell/pixel coverage. The UoM for any given coverage instance *SHALL* be the same. |
| B | precision - The smallest value that has meaning for a given coverage instance. Default is 1. |
| C | scale - Scale as a multiple relative to the unit of measure. Default is 1. |
| D | offset - The offset to the 0 value. Default is 0. |
| E | data_null - The value that indicates NULL. |
| F | grid_cell_encoding - Specifies how a value is assigned to a grid cell (pixel) in a given coverage instance. Default is value-is-center. See below for more information. If value-is-corner is specified, then an additional metadata element _SHALL be specifed stating which corner the value is related to. Possible values are lower-left-corner, upper-left-corner, lower-right-corner, or upper-right-corner. |
| G | field_type - Uniquely identifies the type of Coverage Data. The default is Height aka elevation. |
| H | quantity_definition - Description of the values contained in the Coverage instance. |

**NOTE** If elements that have defaults are not specified then the default values are assumed.

The following provides additional information on the domainSet metadata elements.

#### 7.2.6.1. Units of Measure

The default is the units of measure as defined in the vertical CRS metadata, such as for heights (elevations). However, for many types of grid coverage data there will not be a vertical CRS defined in EPSG or other registries. Examples of such phenomenon are temperature, pressure, and wind speed. The formal (normative) definition for these phenomenon should be specified based on some well know accepted registry, ontology, or other community accepted units of measurement definitions. Currently, UCUM is used as the normative UoM reference in the OGC standards baseline. Some examples from UCUM are:

```
Cel - Celsius, degree  temperature
```

```
[degF]  degree Fahrenheit  temperature
```

```
mbar - millibar  pressure
```

```
B[uV] - microvolt, bel  electric potential level
```

There are other normative sources for UoM definitions, such as the World Meteorological codes registry and the NASA QUDT semantic definitions.

An additional consideration is that a CDB data store may be used in a disconnected environment. Therefore, the UoM specification should not be a URN or URI to some external resource. Consequently, the UoM code used should also have a specific field name and quantity definition.

This field is mandatory.

#### 7.2.6.2. Using the Scale and Offset Values

Integer values MAY be scaled and offset in order to make more efficient use of 16-bit integer space available in some image storage formats such as .png. The scale and offset MAY be applied to the entire coverage and/or the individual tile. The scale and offset do not apply to the data_null value.

#### 7.2.6.3. Grid Cell/Pixel Encoding

There is a small set of possible ways in how a value is actually assigned to a given grid cell or pixel. For example, a value could be assigned to the center of a cell, a corner of a cell, and so forth. Additionally, a value can be assigned to the entire cell. The OGC standards baseline currently states the use of pixel-is-point: GMLJP2 follows the definition of grids in GML 3.2.1 [OGC 07-036] clause 19.2.2: “When a grid point is used to represent a sample space (e.g. image pixel), the grid point represents the center of the sample space (see ISO 19123:2005, 8.2.2)”.

For the purposes of grid coverage GeoPackage extension, the following values are allowed.

```
value-is-center: This is the default. Assume the value is the center of the
cell/pixel.
```

```
value-is-area  Assume the entire grid cell/pixel has the same value.
```

```
value-is-corner  A typical use case is for a mesh of elevation values as specified in
the OGC CDB 1.x Standard.
```

As other sampling methods are identified, the list of enumeration types can be expanded.

#### 7.2.6.4. field_name

Each “field” attribute in a given coverage instance is identified by a name that is unique to the coverage instance. For example, if the field_name is “temperature”, then the entire coverage instance is “temperature”. Therefore, in a CDB data store, there is only one field attribute and one instance.

#### 7.2.6.5. quantity_definition

Associated with the field name is a definition of that field/attribute. In SWE Common this is defined in the Quantity class. In the above example, navigating to http://mmisw.org/ont/cf/parameter/

air_temperature provides a very specific definition of what is meant by “temperature”.

Please note that in a disconnected environment, the use of URNs and URIs is discouraged. Instead, use the text field to provide the quantity definition for the UoM being used. For example, for the above example the Marine Metadata registry provides the following: “Air temperature is the bulk temperature of the air, not the surface (skin) temperature”.

This metadata element/column is required if the values contained in a grid coverage are anything other than height (elevation). This is a text string that describes the field (type). This could be an http uri to an ontology or enumeration, such as “http://sweet.jpl.nasa.gov/2.0/spaceExtent.owl#Height"

### 7.2.7. Tiling Requirements

While there is no a specific CDB requirement that a coverage be tiled, for the vast majority of use cases coverages are tiled. This requirement is driven by performance requirements. There, tiling coverages in a CDB datastore is recommended. As such, if a coverage is tiled then the following requirements are mandatory.

| **Recommendation Coverages7** | **/req/core/coverage-tiling-abstract** |
|---|---|
| A | Any Coverages dataset in a CDB datastore that is tiled *SHALL* be tiled in a manner consistent with and conform to all requirements specified in CDB Core Tiling Requirements Module (Abstract). |

### 7.2.8. Tiling Extension

| **Recommendation Coverages8** | **/req/core/coverage-tiling-extension** |
|---|---|
| A | Any Coverages dataset in a CDB datastore that is tiled *SHALL* implement the tiling rules as defined in a single tiling extension module. Currently available tiling extension requirements modules are defined in CDB Core 2.0 Tiling Requirements Module (Abstract) Recommendation Tiling 1. |

## 7.3. Coordinate Reference System (CRS) Requirements Module

This clause defines the CDB Core CRS Requirements Module. Any CRS may be used - not just WGS-84 - to define the CRS for a CDB conformant data store. Specific CRS types and restrictions are defined in extensions and profiles of this core requirements class. The storage CRS for a CDB data store is the CRS identifier that may be used to retrieve objects, models, coverages, and/or features from that data store without the need to apply a CRS transformation. Any CRS transformations would occur in the client or other down stream applications. Note that if the coordinate epoch is unknown that coordinates referenced to a dynamic CRS are ambiguous. Please see the Epoch requirement below for additional information.

The example below is how CRS could be expressed in a CDB application profiles. Very specific statements are made that restrict the more general core requirements classes. For example, the simulation application profile might state:

Coordinates in a CDB datastore *SHALL* be expressed using WGS-84 (World Geodetic System 1984), equivalent to EPSG (European Petroleum Survey Group) code 4326 (2 dimensions) and EPSG code 4979 (3 dimensions). If a geographic location also has a height or depth, the height or depth *SHALL* be expressed relative to the WGS-84 reference.

This is a restriction of a CDB Core requirement.

### 7.3.1. CDB CRS Core Requirements Class and Requirements

The following is the CDB Core CRS requirements class.

| **Requirements Class - Coordinate Reference System (CRS) Specification and Representation.** | |
|---|---|
| /req/core/data-representation | |
| Target type | Operations |
| Dependency | OGC Abstract Specification Topic 2: Referencing by coordinates (ISO 19111:2019) |
| Dependency | EPSG Geodetic Parameter Dataset |
| Dependency | Geographic information — Well-known text representation of coordinate reference systems |
| Dependency | ISO 19115: Geographic Information - Metadata |
| Requirement CRS2 | /req/core/crs/crs-topic2 |
| Requirement CRS3 | /req/core/crs/crsStorage |
| Requirement CRS4 | /req/core/crs/storageCrs-valid-value |
| Requirement CRS5 | /req/core/crs/crsMetadata |
| Requirement CRS6 | /req/core/crs/uom |
| Requirement CRS7 | /req/core/crs/crsEpoch |

#### 7.3.1.1. Abstract Specification (ISO Standard)

The following requirement states that any and all CRS requirements derive from the OGC (and ISO) Topic 2 Abstract Specification for spatial referencing by coordinates.

| | |
|---|---|
| **Requirement CRS2** | **/req/core/crs/crs-topic2**<br>Any CDB implementation, profile, extension, or application profile *SHALL* be consistent with and conform to the terms and definitions and logical models as defined in OGC Abstract Specification Topic 2: Referencing by coordinates (ISO 19111:2019) AND *SHALL* comply with all other requirements specified in the CRS Requirements Class. |

#### 7.3.1.2. CDB one CRS per data store instance

The following requirement states that any CDB data store compliant with this standard specifies only one CRS. This requirement when combined with Requirement 5 (CRS Metadata) ensures that rigorous coordinate transformations as stated in OGC Topic 2 can be successfully performed without loss of data.

| | |
|---|---|
| **Requirement CRS3** | **/req/core/crs/crsStorage**<br>Only one CRS *SHALL* be specified for a given CDB data store. Further, all coordinate tuples in a coordinate set shall be referenced to the same coordinate reference system.<br><br>NOTE: While specific datasets may be local in nature, the structure of a CDB data store is designed to be global (full earth or planet). |

#### 7.3.1.3. CRS Type

The following requirement specifies that the CRS definition has to be non-projected and earth centric.

| | |
|---|---|
| **Requirement CRS4** | **/req/core/crs/storageCrs-valid-value**<br>Only non-projected (geodetic or geographic) coordinate reference systems *SHALL* be used.<br><br>NOTE: In simple terms: A geographic coordinate system defines where the data is located on the earth. A projected coordinate "tells" how data is to be drawn on a flat surface, such as on a paper map or a computer screen. |

| | |
|---|---|
| **Recommendation** | **/rec/core/crs/crs-definition**<br>Coordinates in CDB *Should* be expressed using WGS-84 (World Geodetic System 1984), equivalent to EPSG (European Petroleum Survey Group) code 4326 (2 dimensions) and EPSG code 4979 (3 dimensions). If a geographic location also has a height or depth, the height or depth *Should* be expressed relative to the WGS-84 reference ellipsoid.<br><br>NOTE: The above recommendation is for backwards compatibility with previous versions of the CDB standard. |

#### 7.3.1.4. CRS Metadata

The following requirement specifies that CRS metadata is to be encoded and stored in the single CDB global metadata file in a CDB data store. This information would be stored once as the same CRS is used for an entire CDB data store. In CDB 1.x, such metadata is in the metadata folder.

| | |
|---|---|
| **Requirement CRS5** | **/req/2.0/core/crs/crsMetadata**<br>Any CDB data store implementation *SHALL* encode and store CRS metadata in the CDB global metadata folder (this was \CDB\Metadata\ in version 1.x). The encoding *SHALL* be consistent with and conform to Well Known Text Version 2 (WKT-2) for CRS. |

The following is an example using WKT-2 for CRS for EPSG 4326 (WGS-84, 2D) :

```
COMPOUNDCRS ["I3S Compound CRS",
GEODCRS["WGS 84",
  DATUM["World Geodetic System 1984",
    ELLIPSOID["WGS 84",6378137,298.257223563,LENGTHUNIT["metre",1.0]]],
  CS[ellipsoidal,2],
    AXIS["latitude",north,ORDER[1]],
    AXIS["longitude",east,ORDER[2]],
    ANGLEUNIT["degree",0.01745329252],
  ID["EPSG",4326]]
VERTCRS["EGM2008 height",
  VDATUM["EGM2008 geoid"],
  CS[vertical,1],
    AXIS["gravity-related height (H)",up],
    LENGTHUNIT["metre",1.0],
  ID["EPSG",3855]]]
```

and

A static geographic CRS with prime meridian other than IRM and ellipsoidal 2D coordinate system in grads (Using Version 2 of the standard):

```
GEOGCRS["NTF (Paris)",
  DATUM["Nouvelle Triangulation Francaise",
```

```
    ELLIPSOID["Clarke 1880 (IGN)",6378249.2,293.4660213]
  ],

  PRIMEM["Paris",2.5969213],
  CS[ellipsoidal,2],
    AXIS["latitude",north,ORDER[1]],
    AXIS["longitude",east,ORDER[2]],
    ANGLEUNIT["grad",0.015707963267949],
  REMARK["Nouvelle Triangulation Française"]

]
```

#### 7.3.1.5. Units of Measure of the CRS

The following requirement is optional and allows for the specification of the Units of Measure (UoM) for a CRS if either:
1.) The UoM is not specified in the CRS metadata or
2.) the UoM to be used is not what is specified in the CRS metadata.

For example, if the CRS is EPSG 4326, the UoM in the provided CRS metadata is decimal degrees.

| **Requirement CRS6** | **/req/2.0/core/crs/uom**<br><br>All CRS Units of Measure (UoM) in a CDB conformant data store *SHALL* be the same for all geospatial coordinates for the entire data store regardless of feature type or geometry type.<br>UoMs for coordinates may be specified using the OGC WKT for CRS `unit` parameter in the CRS global metadata (WKT for CRS encoding). |
|---|---|

| **NOTE** | The CRS UoM requirement does not apply to models that may be in local (engineering) coordinate systems. |
|---|---|

#### 7.3.1.6. Data Store Epoch

The earth is not static. Plate tecktonics and other geological and geomorphological processes cause locations (and features) to change over time. This is called *displacement*. For example, some locations in California can move 3 to 4 inches a year as the Pacific plate moves north. Previous versions of the CDB Standard assumed a static datum. However, to accommodate the dynamic nature of the earth, CDB 2.0 and later recommends the use of dynamic datums.

CDB 2.0 and later mandates the use of a reference epoch for any CRS that is dynamic. In this document “Epoch” is a point in time. For example, WGS84 is a dynamic reference frame. At the beginning of each year, the NGA updates station coordinates of the seventeen sites. The coordinates are adjusted to an epoch at the half year mark to account for plate tectonic motion.

There are two levels of epoch metadata that should be provided in a CDB data store: The epoch at the time the data store is created and the coordinate epoch for when any image or vector data was

collected. The former is a global metadata element and should be used when there is no epoch metadata available for a given data set in the data store. The latter is specified at the data set level and provides the epoch of the coordinates in a given CDB data set.

| **Requirement CRS7** | **/req/core/crs/crsEpoch** |
|---|---|
| A | Any CDB 2.0 and later data store that implements a dynamic datum (reference frame) *SHALL* specify an epoch as part of the CRS metadata. |
| B | The epoch *SHALL* be specified as a decimal year in the Gregorian calendar, with yyyy.00 being midnight at the start of the 1st January of year “yyyy”. |

| **NOTE** | In a dynamic CRS, coordinates of a point on the surface of the Earth may change with time. To be unambiguous the coordinates must always be qualified with the epoch at which they are valid. This is often expressed in the form “<CRS_name> at epoch T”, “<CRS_name> epoch T” or “<CRS_name>@T”. |
|---|---|

EXAMPLES

- ITRF2008 at epoch 2017.53
- ITRF2008 epoch 2017.53
- WGS 84 (G1762) @ 2017.53

By providing an epoch along with a coordinate epoch, the necessary metadata is available to perform deformation transformations to not only calculate the current location but also to predict future locations.

#### 7.3.1.7. Vertical CRS - Optional Class

This clause defines an optional Requirements for defining a Vertical Coordinate Reference System (VCRS) for a CDB data store. The VCRS may either be ellipsoidal (height defined with respect to a reference ellipsoid) or gravity-related height (height defined with respect to a reference geoid/gravity surface). This approach supports the use of CDB 2.0 and later with a VCRS across a diverse range of fields and applications where the definition of height is of importance.

This is an optional requirements class. However, any application profile, extension, or implementation that specifies the use of a VCRS *SHALL* be conformant with this requirements class.

| **Requirements Class - Vertical Coordinate Reference System (VCRS) Specification and Representation.** | |
|---|---|
| /req/core/data-representation | |
| Target type | Operations |
| Dependency | OGC Abstract Specification Topic 2: Referencing by coordinates (ISO 19111:2019) |
| Dependency | EPSG Geodetic Parameter Dataset |

| | |
|---|---|
| Dependency | Geographic information — Well-known text representation of coordinate reference systems |
| Requirement VCRS 1 | /req/core/crs/vcrs-topic2 |
| Requirement VCRS 2 | /req/core/crs/vcrs-encoding |
| Requirement VCRS 3 | /req/core/crs/vcrs-units |

The following requirement states that any and all Vertical CRS requirements derive from the OGC (and ISO) Topic 2 Abstract Specification for spatial referencing by coordinates.

| **Requirement VCRS 1** | **/req/core/crs/vcrs-topic2** |
|---|---|
| A | Any CDB implementation, profile, extension, or application profile specifying a VCRS *SHALL* be consistent with and conform to the terms and definitions and logical models as defined in OGC Abstract Specification Topic 2: Referencing by coordinates (ISO 19111:2019) |
| B | AND *SHALL* also comply with all other requirements specified in the CRS Requirements Class. |

The following requirement states that ISO 19125 Geographic information — Well-known text representation of coordinate reference systems *SHALL* be used for encoding VCRS data.

| **Requirement VCRS 2** | **/req/core/crs/vcrs-wkts-crs** |
|---|---|
| A | Any CDB implementation, profile, extension, or application profile specifying a VCRS *SHALL* encode VCRS as defined in Geographic information — Well-known text representation of coordinate reference systems. |

#### 7.3.1.8. Default vertical CRS units

| **Requirement VCRS 3** | **/req/core/crs/vcrs-units** |
|---|---|
| A | If vertical extent units are not stated they *SHALL* be assumed to be meters. |

#### 7.3.1.9. WKT for CRS Example

The following is an example of using WKT for a description of a compound WGS 84, EPSG 4326 and EPSG 3855.

```
COMPOUNDCRS ["CDB Compound CRS",
```

```
GEODCRS["WGS 84",
  DATUM["World Geodetic System 1984",
    ELLIPSOID["WGS 84",6378137,298.257223563,LENGTHUNIT["metre",1.0]]],
  CS[ellipsoidal,2],
    AXIS["latitude",north,ORDER[1]],
    AXIS["longitude",east,ORDER[2]],
    ANGLEUNIT["degree",0.01745329252],
  ID["EPSG",4326]]
VERTCRS["EGM2008 height",
  VDATUM["EGM2008 geoid"],
  CS[vertical,1],
    AXIS["gravity-related height (H)",up],
    LENGTHUNIT["metre",1.0],
  ID["EPSG",3855]]
```

# 7.4. Resource Path and File Naming Requirements Module

The CDB 1.x series of Standards specified very strict and deterministic rules for generating a path to a resource or for generating a name for a resource.

**NOTE** There exists a certain degree of ambiguity of whether the term "asset" or the term "resource" should be used in this Standard. The gaming industry uses the term "asset" for textures, geometries, and so forth. The Web and Internet worlds use the term "resource" for the same items. One can split semantic hairs but given the fact that the OGC typically uses the word "resource", CDB 2.0 and later uses the term "resource". From IETF RFC 2396: A resource can be anything that has identity.

A key aspect of providing a performant yet deterministic CDB data store are the resource naming conventions. The following requirements and recommentations are based on current CDB 1.x best practices as well as style guides for a number of widely used gaming engines such as Unity and Unreal.

## 7.4.1. CDB Resource Path/Naming Requirements Class

| **Requirements Class - Resource Path/Name Structure** | |
|---|---|
| /req/core/naming-system | |
| Target type | Operations |
| Dependency | Target file system. |
| Dependency | Resource File Conventions |
| Requirement Name1 | /req/core/name-spaces |
| Recommendation Name1 | /req/core/name-unicode |

| | |
|---|---|
| Requirement Name2 | /req/core/name-language |
| Recommendation Name2 | /req/core/name-empty-folders |
| Requirement Name3 | /req/core/name-ap-guide |
| Requirement Name4 | /req/core/name-case |
| Requirement Name5 | /req/core/name-ap-guide |
| Requirement Name6 | /req/core/name-case |
| Requirement Name7 | /req/core/name-extensions |

### 7.4.2. Do not uses spaces in paths or names

| **Requirement Name1** | **/req/core/name-spaces** |
|---|---|
| A | Spaces *SHALL* not be used in any CDB resource path, folder name, or resource name. |

### 7.4.3. Do not use unicode

| **Recommendation Name1** | **/req/core/name-unicode** |
|---|---|
| A | Unicode characters or other symbols *SHOULD* not be used in any CDB resource path, folder name, or resource name. |
| B | The following characters *SHALL NOT* be used in any CDB resource path, folder name, or resource name.<br><br># pound % percent & ampersand { left curly bracket } right curly bracket<br>\ back slash < left angle bracket > right angle bracket * asterisk<br>? question mark / forward slash blank spaces $ dollar sign ! exclamation point<br>' single quotes " double quotes : colon @ at sign |

| | |
|---|---|
| **NOTE** | While Unicode strings can be used in most operating and file storage systems for resource paths, folder names, and file names be aware that conversions from UTF8 and UTF16 can cause performance issues as well as content size issues. Further, to ensure maximum system portability of a CDB datastore, only ASCII characters |

should be used in any CDB resource path, folder name, or resource name.

### 7.4.4. Language Requirement and Recommendation on English

| Requirement Name3 | /req/core/name-language |
|---|---|
| A | All resource names, paths, and folders *SHALL* be in the same language. |
| B | This language *SHOULD* be in English. |

### 7.4.5. Empty Folder Guidance

| Recommendation Name4 | /req/core/name-empty-folders |
|---|---|
| A | Empty folders or resources devoid of content *SHOULD* be avoided. |

**NOTE** This is a performance recommendation.

### 7.4.6. Style Guide Requirement

| Requirement Name5 | /req/core/name-ap-guide |
|---|---|
| A | Any CDB application profile based on the CDB Core Requirements Modules and Classes *SHALL* define a style guide of requirements for consistent naming and path generation for any resource and folder. |

### 7.4.7. String case

| Requirement Name6 | /req/core/name-case |
|---|---|
| A | Case - Any CDB implementation and/or application profile *SHALL* use the same case rule for an entire data store. These are:<br><br>PascalCase: Pascal case combines words by capitalizing all words (even the first word) and removing the space.<br>camelCase: Camel case combines words by capitalizing all words following the first word and removing the space<br>Snake_case: Snake case combines words by replacing each space with an underscore (_) and, in the all caps version, all letters are capitalized<br>kebab-case: Kebab case combines words by replacing each space with a dash (-) |

### 7.4.8. File name extensions

CDB 1.x series of standards specified a consistent set of file extensions for any resource containing content. For example, every Shapefile in a CDB datastore has the .shp file extension. The use of these file extensions, along with others, continue to be used and specified in CDB 2.0 and later. The table below the next requirement provides a list of file types/encodings used in CDB version 1.x as well as additional file types and extensions that may be used in CDB 2.0 and later implementations.

**NOTE** In the CDB 1.x Standards, the standard specified rules (requirements) and guidance for the proper use of each of the formats/encodings used in the 1.x series of Standards. For example, in Volume 2 CDB Core: Model and Physical Structure Annexes C and H describe the use of the JPEG 2000 file format in a CDB data store. These rules and guidance were provided to enhance interoperability and "determinism" in a CDB datastore. For formats/encodings in the table below that are not in CDB 1.x, such rules and guidelines shall be specified in any application profile as required.

| **Requirement Name7** | **/req/core/name-extensions** |
|---|---|
| A | Any CDB implementation and/or application profile *SHALL* use the file extensions as specified in the table below. |
| B | For file types or resource types not listed in the table then industry standard extensions *SHALL* be used. |

| **File Format** | **Minimal Version Number** | **Extension** |
|---|---|---|
| Bitmap Image | Microsoft | *.bmp |
| dBASE | III+ | *.dbf, *.dbt |
| GeoPackage | 1.1 and later | *.gpkg |
| glTF JSON/ASCII | 2.0 and later | *.gltf |
| glTF Binary | 2.0 and later | *.glb |
| JPEG 2000 | 1.0 and later | *.jp2 |
| JSON | Any | *.json |
| OpenFlight | 16.0 and later | *.flt |
| Shapefile | Esri White Paper, July 98 | *.shp, *.shx |
| SGI Image | 1.0 | *.rgb |
| SGI Image + Alpha | 1.0 | *.rgba |
| TIFF | 6.0 | *.tif |
| XML | 1.0 and later | *.xml, *.xsd |
| ZIP | 6.3.1 and later | *.zip |

**NOTE** Additional file extensions, such as for compressed textures may be defined in other

CDB 2.0 and later requirements modules.

## 7.5. File Hierarchy Structure

The CDB Standard assumes that some form of a hierarchical file management system that supports the concept of folders is available for CDB implementations. For example, in a file system in the cloud this may be a hierarchical storage system that provides shared access to file data. Users can create, delete, modify, read, and write files and can organize them logically in directory trees for intuitive access. This concept is true for the majority of widely used cloud platforms. Similarily, all widely used operating systems, such as MS Windows, UNIX, HADOOP, and LINUX, provide hierarchical file systems and the concept of folders.

**NOTE** The CDB file, directory, and folder structure is defined by a combination of folder hierarchy and metadata files.

**NOTE** As a CDB datastore grows it becomes unwieldy to store in different environments and transfer between services. To use or transfer a CDB in its original format would be to use or transfer massive amounts of small files, amounting to very large costs and potential for lossy data. There is a recommendation to use the ISO 9660 chunking format which may be beneficial. Using this standard, a CDB datastore is separated into transferrable-sized (gigabytes each) chunks. The alternative would be to package the many small files in a "traditional CDB datastore" into a much smaller number of GeoPackages.

### 7.5.1. CDB File Requirements Class

| **Requirements Class - File/Folder Structure** | |
|---|---|
| /req/core/file-system | |
| Target type | Operations |
| Dependency | Target file system. |
| Dependency | Resource Naming Conventions |
| Requirement File1 | /req/core/file-structure |
| Requirement File2 | /req/core/file-cdb-root-location |
| Requirement File3 | /req/core/file-hierarchy |
| Permission PFile1 | /req/core/file-hierarchy-links |
| Requirement File5 | /req/core/file-hierarchy-root |

| | |
|---|---|
| Recommendation RFile1 | /rec/core/file-hierarchy-root-name |
| Requirement File6 | /req/core/file-root-global-metadata |

### 7.5.2. Hierarchial file system

| **Requirement File1** | **/req/core/file-structure** |
|---|---|
| A | Any platform on which a CDB data store is to be implemented *SHALL* support the concepts of both folders and hierarchical file systems. |

### 7.5.3. Root location of a CDB datastore

| **Requirement File2** | **/req/core/file-cdb-root-location** |
|---|---|
| A | Each physical instance of a CDB data store *SHALL* have a `root` location. |
| B | All content in a single physical CDB datastore instance shall be accessible via the root location structured as sub-directories, folders, or some other hierarchial structure. |

NOTE 1: In the CDB 1.x Standard, the root directory/folder was identified as \CDB

NOTE 2: In CDB 2.0 and later, the concept of the root location is identical to the OGC API concept of `landing page`.

### 7.5.4. Hierarchy of file system/folder structure

| **Requirement File3** | **/req/core/file-hierarchy** |
|---|---|
| A | All of the files stored within a CDB data store *SHALL* be accessible under the root directory or within a subdirectory under the root directory. |

| **Permission PFile1** | **/req/core/file-hierarchy-link** |
|---|---|
| A | Subfolders (or locations) within a CDB data store *MAY* be accessible under the root directory or within a subdirectory under the root directory as links to the physical resource (dataset). |

### 7.5.5. Name of root directory/folder

| **Requirement File5** | **/req/core/file-hierarchy-root** |
|---|---|
| A | The root of the file/folder hierarchy in a CDB data store *SHALL* begin with `/`. |

| | |
|---|---|
| **NOTE** | The above requirement is consistent with the OGC API Landing Page concept and related requirements. The purpose of the landing page (root) is to provide clients with a starting point for using accessing a CDB datastore. Any resource in a CDB datastore can be accessed by following paths or links starting from the landing page or root. |

### 7.5.6. Recommended Name of root directory/folder (CDB Landing Page)

| **Recommendation RFile1** | **/rec/core/file-hierarchy-root-name** |
|---|---|
| A | The recommended root of the file/folder hierarchy in a CDB data store *SHOULD* be /cdb. |

### 7.5.7. Global Metadata

| **Requirement File6** | **/req/core/file-root-global-metadata** |
|---|---|
| A | All global metadata, controlled vocabularies, and enumerations in a CDB data store *SHALL* be accessible from the CDB datastore root node (landing page) in a folder labeled `global_metadata`. |

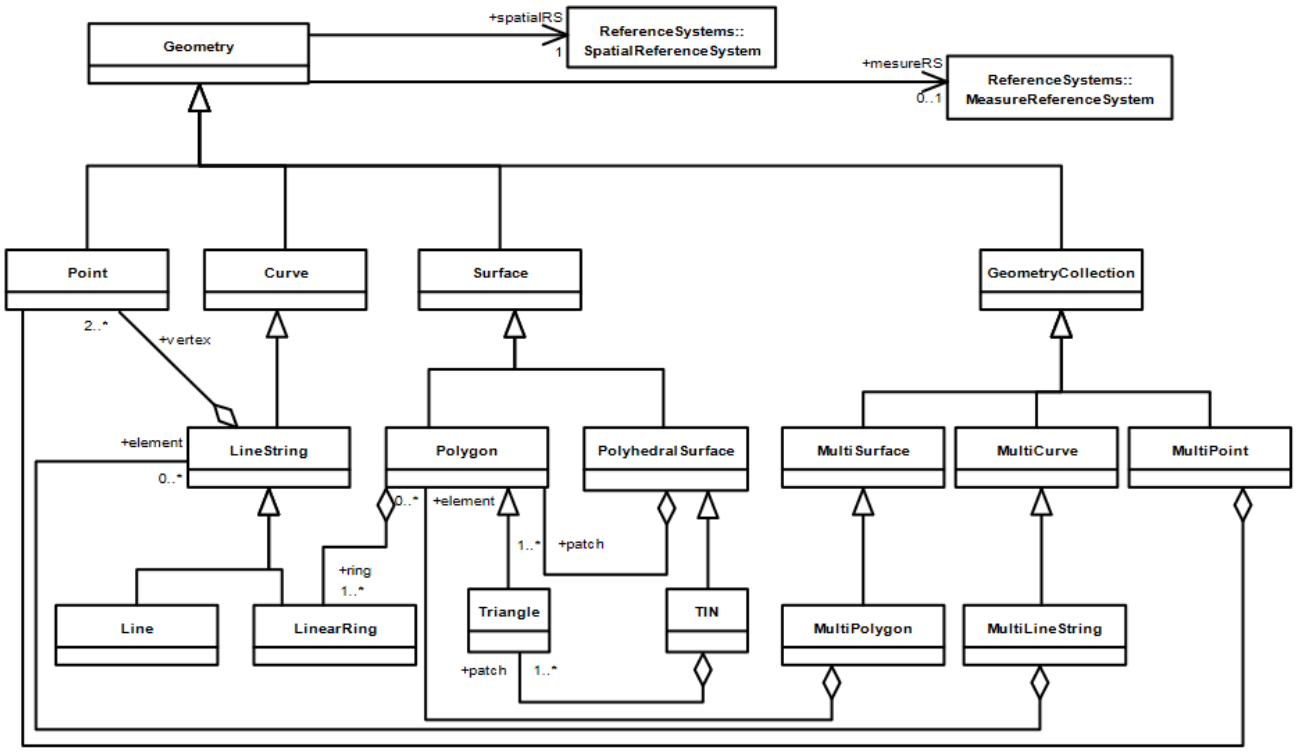
## 7.6. Geometry

This clause specifies the CDB 2.0 and later Core Geometry Requirements Module.

The OGC and ISO Implementation Standard for Geographic information - Simple feature access - Part 1: Common architecture defines a geometry model. This geometry model is the foundation for the implementation of the Simple Features (SF) Standard in numerous spatial databases including PostGIS, Oracle Spatial, and DB2. Further, a number of other international standards, such as GeoPackage and GeoJSON, comply with the SF Geometry Model. Finally, any Shapefule encoding is consistent with the ISO/OGC Geometry model.

### 7.6.1. The Geometry Model

The following is the Simple Features Geometry Class Hierarchy.




Geometry is the root class of the hierarchy. Geometry is an abstract (non-instantiable) class. The instantiable subclasses of Geometry defined in this Standard are restricted to 0, 1 and 2-dimensional geometric objects that exist in 2, 3 or 4-dimensional coordinate space (□2, □3 or □4). Geometry values in R2 have points with coordinate values for x and y. Geometry values in R3 have points with coordinate values for x, y and z or for x, y and m. Geometry values in R4 have points with coordinate values for x, y, z and m where "m" is a measure which could be date and time. (Source: Simple feature access - Part 1: Common architecture)

Geometry types specified in the model that can be used in a CDB 2.0 or later data store are defined in the following table. Please note that an CDB application profile can extend the allowed set of Geometry Types. Please also note that A GeometryCollection is a geometric object that is a collection of some number of geometric objects. All the elements in a GeometryCollection shall be in the same Spatial Reference System. This is also the Spatial Reference System (or Coordinate Reference System) for the GeometryCollection.

**NOTE** The Geometry type codes specified in this requirements module are consistent with the Simple Features Standard as well as the GeoPackage Standard. However, the codes above the number 7 are NOT consistent with the codes used in CDB datastores conformant with the CDB 1.X series of Standards.

| Code | CDB Geometry type | Description |
|---|---|---|
| 0 | Geometry | A geometry |
| 1 | Point | topological 0-dimensional geometric primitive , representing a position usually encoded as an X, Y, Lat/Long, or Lomg/Lat coordinate. |

| Code | CDB Geometry type | Description |
|---|---|---|
| 2 | Linestring | A LineString is a Curve with linear interpolation between Points. Each consecutive pair of Points defines a Line segment.<br>Note: A Line is a LineString with exactly 2 Points.<br>Note: A LinearRing is a LineString that is both closed and simple. |
| 3 | Polygon | A Polygon is a planar Surface defined by 1 exterior boundary and 0 or more interior boundaries. Each interior boundary defines a hole in the Polygon.<br>Note: A Triangle is a polygon with 3 distinct, non-collinear vertices and no interior boundary. |
| 4 | MultiPoint | A MultiPoint is a 0-dimensional GeometryCollection. The elements of a MultiPoint are restricted to Points. |
| 5 | MultiLinestring | A MultiLineString is a MultiCurve whose elements are LineStrings.<br>Note: A MultiCurve is a 1-dimensional GeometryCollection whose elements are Curves. Note: topological 1-dimensional geometric primitive, representing the continuous image of a line. |

| Code | CDB Geometry type | Description |
|---|---|---|
| 6 | MultiPolygon | A MultiPolygon is a MultiSurface whose elements are Polygons.<br>Note: A MultiSurface is a 2-dimensional GeometryCollection whose elements are Surfaces, all using coordinates from the same coordinate reference system. The geometric interiors of any two Surfaces in a MultiSurface may not intersect in the full coordinate system. |
| 7 | GeometryCollection | A GeometryCollection is a geometric object that is a collection of some number of geometric objects.<br>Note: Subclasses of GeometryCollection may restrict membership based on dimension and may also place other constraints on the degree of spatial overlap between elements, |

Table Geometry 1: Core Geometry Types

The following table is the set of core geometries that may have a Z element or coordinate associated with the geometry coordinates. Typically the Z value is the height above or below the elipsoid but could also be the height of a building or the depth of a well. The z coordinate value traditionally represents the third dimension (i.e. 3D). For example: A map might have point identifying the position of a mountain peak by its location on the earth, with the x and y coordinate values, and the height of the mountain, with the z coordinate value.

| | | |
|---|---|---|
| 1001 | Point Z | Point geometry with a Z value. |
| 1002 | Linestring Z | A LineString geometry in which every coordinate in the LineString has an associated Z value. |
| 1003 | Polygon Z | A Polygon geometry in which every coordinate in the Polygon has an associated Z value. |

| | | |
|---|---|---|
| 1004 | Multipoint Z | A MultiPoint geometry in which every coordinate in the Point geometry collection has an associated `Z` value. |

The following table is the set of core geometries that may have a `M` element associated with the geometry. `M` stands for a measure, such as precipitation or soil accidity.

| | | |
|---|---|---|
| 2001 | Point M | Point geometry with a `m` value. |
| 2002 | Linestring M | A LineString geometry in which every coordinate in the LineString has an associated `m` value. |
| 2003 | Polygon M | A Polygon geometry in which every coordinate in the Polygon has an associated `m` value. |
| 2004 | MultiPoint M | A MultiPoint geometry in which every coordinate in the Point geometry collection has an associated `m` value. |

NOTE 1: The Name of the geometry type as used in CDB 2.0 is consistent with the OGC Simple Features, GeoPackage, GeoJSON and other standards that reference Simple Features as normative.

NOTE 2: The codes are consistent with those specified in GeoPackage.

The above geometry types are considered to be components of the core. The following geometry types may also be specified for use as extensions in any CDB 2.0 application profile.

| **Code** | **Name** | Description |
|---|---|---|
| 11 | MULTICURVE | A MultiCurve is a 1-dimensional GeometryCollection whose elements are Curves |
| 12 | MULTISURFACE | A MultiSurface is a 2-dimensional GeometryCollection whose elements are Surfaces, all using coordinates from the same coordinate reference system. |

| | | |
|---|---|---|
| 13 | CURVE | A Curve is a 1-dimensional geometric object usually stored as a sequence of Points, with the subtype of Curve specifying the form of the interpolation between Points. This standard defines only one subclass of Curve, LineString, which uses linear interpolation between Points. |
| 14 | SURFACE | A Surface is a 2-dimensional geometric object. |

### 7.6.2. Geometry Requirements Class

The following is the Geometry Requirements Class.

| **Requirements Class - Geometry** | |
|---|---|
| /req/core/geometry | |
| Target type | Operations |
| Dependency | Simple Features geometry model |
| Dependency | CRS UoM or Global UoM (CRS Requirement CRS6) |
| Requirement Geom1 | /req/core/geometry-model |
| Requirement Geom2 | /req/core/geometry-types |
| Requirement Geom3 | /req/core/geometry-zvalue |
| Requirement Geom4 | /req/core/geometry-mvalue |
| Requirement Geom5 | /req/core/geometry-coordinates |
| Requirement Geom6 | /req/core/geometry-collection-srs |

#### 7.6.2.1. Geometry Model

Geometry in a compliant CDB Datastore must be consistent with and conform to Simple Feature Access.

| | |
|---|---|
| **Requirement Geom1** | **/req/core/geometry-model**<br>Any CDB implementation, profile, extension, or application profile specifying and/or implementing geometry *SHALL* be consistent with and conform to the terms and definitions and logical models as defined in OGC Implementation Standard for Geographic information - Simple feature access - Part 1: Common architecture (ISO 19111:2019) AND |
| A | *SHALL* comply with requirements Geom2 through Geom6 as stated below. |

#### 7.6.2.2. Geometry types

The following specifies the types of Geometry in a CDB datastore. Please note that the list above is extensible.

| | |
|---|---|
| **Requirement Geom2** | **/req/core/geometry-types**<br>Any CDB implementation, profile, extension, or application profile specifying and/or implementing geometry *SHALL* implement one or more of the geometry types as specified in the Simple Features model and specified in the above table of Geometry Types.<br><br>NOTE: Application profiles and extensions may specify additional geometry types. However, implementers should be aware that such extensions to the geometry model may impact interoperabiity. |

### 7.6.3. Geometry with a Z coordinate.

| | |
|---|---|
| **Requirement Geom3** | req/core/geometry-zcoordinate<br>Any CDB implementation, profile, extension, or application profile specifying and/or implementing geometry with a `z` coordinate *SHALL* specify the the units of measure, such as meters, for the `z` values in the CDB Global Metadata Units of Measure (UoM).<br><br>NOTE: Typically the `z` value is the height above or below the elipsoid but could also be the height of a building or the depth of a well. |

| | |
|---|---|
| **Requirement Geom4** | **/req/core/geometry-mvalue**<br>Any CDB implementation, profile, extension, or application profile specifying and/or implementing geometry with a `m` coordinate *SHALL* specify the`m` value in the units as specified in the metadata for a given CDB dataset.<br><br>NOTE: The m coordinate represents a measurement. |

| | |
|---|---|
| **Requirement Geom5** | **/req/core/geometry-coordinates**<br>For CDB implementation, profile, extension, or application profile specifying and/or implementing geometry:<br>. Each coordinate *SHALL* be unambiguously associated to a coordinate reference system either directly or through its containing geometry.<br>. All coordinates within a geometry object *SHALL* be in the same coordinate reference system.<br><br>NOTE: The m coordinate represents a measurement. |

### 7.6.4. Consistent SRS/CRS in a Geometry Collection

| | |
|---|---|
| **Requirement Geom6** | **/req/core/geometry-collection-srs**<br>For CDB implementation, profile, extension, or application profile specifying and/or implementing geometry collection: |
| A | Every Geometry in the Geometry Collection *SHALL* be unambiguously associated to a coordinate reference system either directly or through its containing geometry. |
| B | All Geometries in a Geometry Collection *SHALL* be in the same coordinate reference system. |

All Geometry classes described in this Standard are defined so that instances of Geometry are topologically closed. In other words, all represented geometries include their boundary as point sets.

## 7.7. Links

Links are used to describe a relationship with another resource. The resource may be internal to a CDB datastore or may be an external reference such as to an authoritative controlled vocabulary.

| **Requirements Class - links** | |
|---|---|
| /req/core/links | |
| Target type | Operations |
| Dependency | URL formatting requirements |
| Requirement Link1 | /req/core/link-href |
| Requirement Link2 | /req/core/link-rel |
| Recommendation Link3 | /req/core/link-media |
| Recommendation Link4 | /req/core/link-title |

All links are encoded as a URL (Uniform Resource Locator) string.

| **Requirement link1** | http://www.opengis.net/spec/CDB/2.0/core/link-href<br><br>The actual link *SHALL* be encoded in the format of an URL. Relative and absolute links are both allowed. |
|---|---|

The following table summarizes the requirements and recommendations for the `links` requirements class.

This object describes a relationship with another entity.

| Field Name | Type | Description | Example | M/R |
|---|---|---|---|---|
| **href** | string | The actual link in the format of an URL. Relative and absolute links are both allowed. | http://data.example.com/buildings/123 | M |
| **rel** | string | The type or semantics of the relation. | alternate | M |
| type | string | A hint of the [Media type](#media-types) of the referenced entity. | application/geo+json | R |
| title | string | A human readable title to be used in rendered displays of the link. | Trierer Strasse 70, 53115 Bonn | R |

JSON media types that would typically be used in a server that supports JSON are:

- `application/geo+json` for feature collections and features, and
- `application/json` for all other resources.

XML media types that would typically occur in a server that supports XML are:

- `application/xml` for all other resources.

The typical HTML media type for all "web pages" in a server would be `text/html`.

Files whose media type is image contain image data. The subtype specifies which specific image file format the data represents. Possible image media types used in a CDB datastore include:

- `image/jpeg` - Joint Photographic Expert Group image (JPEG)
- `image/png` - Portable Network Graphics (PNG)
- `image/tiff` -

### 7.7.1. Note on templated links

Templated links may used used in a CDB data store. Templated linksare used to bind to resources that may require additional information in order to access the resources. For example, simply knowing the URL of a legacy OGC service such as WMS or WFS is not sufficient to access resources offered by those services. Additional information in the form of request parameters and values are required in order to have a complete link to a resources. A templated link with variables can provide this additional context. In a CDB datastore, a typicaly use for a templated link would be to access additional information related to an entry in the attribute model.

## 7.8. Media Types

One of the best ways to help inform client/service applications about the content in a link is to use a common Media Type in the `type` field. The use of media type is quite useful for browsers to better determine what to render and display to users searching and browsing a CDB datastore. Media types are often referred to by the now deprecated term "MIME types".

### 7.8.1. Media Type for CDB Datastores

Metadata and controlled vocabularies in a CDB datastore are encoded as XML or JSON. Additionally, resources may be encoded as GeoPackages, GeoJSON, Shapefiles, OpenFlight models, glTF, and so on.

The following table lists the Media Types to use for CDB metadata and datasets. This table can be extended as required.

| Media Type | Description |
|---|---|
| --- | --- |
| `model/flt` | A model dataset is an OpenFlight encoding. |
| `application/geo+json` | A vector dataset is encoded as GeoJSON. |
| `application/geopackage+sqlite3` | A dataset is encoded as a GeoPackage |
| `image/tiff; application=geotiff` | A dataset encoded as GeoTIFF with standardized georeferencing metadata. |
| `application/gltf-buffer` | A dataset is a glTF - Buffer encoding. |
| `model/gltf-binary` | A model dataset is a glTF binary encoding. |
| `model/gltf+json` | A model dataset is a glTF JSON encoding. |
| `application/gml+xml` | A dataset is encoded as GML. |
| `image/jp2` | A dataset is encoded as JPEG2000. |
| `application/json` | A dataset, usually metadata, is encoded as JSON. |
| `image/png` | An image (coverage) is encoded as png. |
| `application/vnd.shp` | A vector dataset is encodes as a Shapefile. |
| `image/tiff` | An image is encode as TIFF. |

| | |
|---|---|
| `application/xml` | A dataset, usually metadata, is encoded as XML. |

# 7.9. CDB Global and Local Resource/Dataset Metadata

This CDB Metadata Requirements Module specifies requirements and recommendations for global and local (dataset) metadata in a CDB compliant datastore.

## 7.9.1. What is metadata?

Metadata is data that provides information about other data. In the geospatial community and the rapidly emerging spatial web community, metadata is critical to operations such as discovery and determining whether a resource is “fit for purpose”. Three distinct types of metadata exist: **descriptive metadata**, **structural metadata**, and **administrative metadata** (National Information Standards Organization (NISO)).

- Descriptive metadata describes a resource for purposes such as discovery and identification. It can include elements such as title, abstract, author, and keywords.
- Structural metadata is metadata about containers of metadata and indicates how compound objects are put together, for example, how pages are ordered to form chapters.
- Administrative metadata provides information to help manage a resource, such as when and how it was created, file type and other technical information, and who can access it. <end Wikipedia>

The geospatial community has a long and extensive history in defining and using metadata for geospatial resources. Without metadata, discovery of required resources and determination of whether a resource is “Fit for Purpose” becomes difficult if not impossible. The geospatial community makes use of all three types of metadata, although the first and third are more critical. The CDB use of metadata focuses on use cases 1 and 3.

## 7.9.2. CDB and Metadata

While much of the CDB Core requirements, such as the CRS requirements, are related to metadata in one form or another, this sestion specifies requirements targeted at 1.) Global Metadata for a CDB datastore and 2.) more "traditional" metadata for location enabled resources and datasets. There are a number of metadata and controlled vocabularies, such as Lights and Version, that may be used as part of a CDB datastore implementation. The global metadata properties and controlled vocabulary elements are described with requirements in their own Clauses in the core CDB 2.0 Standard.

The CDB Core Standard provides the requirements for:

- CDB Global metadata - those metadata elements applicable to an entire CDB datastore instance.
- Resource Metadata (formerly local metadata in CDB 1.x) - those metadata elements specific to a given local resource, such as a GeoTypical Model or vector dataset, in a CDB datastore.

| | |
|---|---|
| **NOTE** | Should - for purposes of CDB - this be termed `dataset` metadata (dataset is part of the DCAT superclass Resource which also includes the super-class of dcat:Dataset, |

dcat:DataService, dcat:Catalog and any other member of a dcat:Catalog)?

**NOTE** This standard does not specifiy which metadata standard shall be used. Instead, for flexibility and in recogniztion that different communities use different metadata standards or profiles of metadata standards, the CDB standard provides a list of allowed metadata standards and profiles of those standards. The caveat is that the same metadata standard (or profile of that standard) is used consistently throughout the datastore.

More specifically, `Global metadata`, including vocabularies, are data such as CDB version or Lights are global to an entire CDB data store instance. Individual file sets, tiles, and other resources in a CDB data store may also have metadata. These resources are termed “resource” metadata.

`Resource metadata` refers to metadata specific to a given dataset within a CDB data store. This level of metadata is the minimum core for geospatial datasets regardless of the specific geospatial subject (i.e., vector, raster (imagery), sensor, or model). Metadata at this level is designed to assist in discovery, retrieval and semantic description of the data. Dataset level metadata is typically stored at the model or tile level.

In CDB 1.x the majority of CDB controlled vocabularies, enumerations, and metadata files are encoded using eXtended Markup Language (XML) files. In CDB 1.x the XML schemas can be found in the \CDB\Metadata\Schema\ folder delivered with the CDB standard. In CDB 1.x the exceptions are the global and local geospatial metadata files which may be XML schema, JSON, JSON schema, or some other internationally recognized encoding. In CDB 2.0 and later, any metadata may be encoded using XML, JSON or stored in a database table. However, the encoding chosen shall be consistent throughout a given CDB datstore instance.

**NOTE** In CDB 2.0 and later, the location of the global metadata is specified using a link as specified in the CDB Core Links Requirements Class.

Resource Metadata may also be stored in a GeoPackage using the GeoPackage Metadata Extension.

**NOTE** Please note that domain or application specific metadata, such as for satellite imagery or weather sensor observations are specified as extensions to this core metadata module and may be mandatory requirements classes in a CDB application profile.

### 7.9.3. CDB Global Datastore and Resource/Dataset Metadata

The following requirements class specifies the required metadata elements applicable to an entire CDB datastore.

| **Requirements Class - Global-Metadata** | |
|---|---|
| /req/core/metadata- | |
| Target type | Operations |
| Dependency | Specified metadata standard or profile of said standard. |

| | |
|---|---|
| Requirement Metadata1 | /req/core/metadata-repository |
| Requirement Metadata2 | /req/core/metadata-standard |
| Requirement Metadata3 | /req/core/metadata-global |
| Requirement Metadata4 | /req/core/metadata-language |
| Requirement Metadata5 | /req/core/metadata-encoding |
| Requirement Metadata6 | /req/core/metadata-datetime |
| Requirement Metadata7 | /req/core/metadata-temporal-interval |
| Requirement Metadata8 | /req/core/metadata-uom-measure |

Each of the fundamaental metadata requirements are now defined.

#### 7.9.3.1. Global Metadata

| **Requirement Metadata1** | **/req/core/metadata-repository** |
|---|---|
| A | Any CDB implementation, profile, extension, or application profile datastore *SHALL* have a global metadata file stored in the Global_Metadata folder in a CDB datastore. Global_Metadata is metadata applicable to all resources in a CDB datastore. |
| B | The Global_Metadata *SHALL* always be referenced/available in a subfolder in the CDB root folder (See Requirements 2 and 3 in CDB Core File Structure Requirements |

#### 7.9.3.2. Specify the metadata standard being used.

| **Requirement Metadata2** | **/req/core/metadata-standard** |
|---|---|

| | |
|---|---|
| A | Any CDB datastore *SHALL* use one of the following metadata standards for any metadata provided in a CDB datastore (keyword, description):<br>ISO-19115:2019 - Geographic information — Metadata — Part 1: Fundamentals<br>ISO-19115:2003 - Geographic information — Metadata<br>DDMS-5.0 - DoD Discovery Metadata Specification DDMS-4.1 - DoD Discovery Metadata Specification<br>DCAT - Data Catalog Vocabulary<br>DCAT-AP - DCAT Application Profile<br>GeoDCAT-AP - An extension of DCAT-AP for describing geospatial datasets, dataset series, and services.<br>NGCMP - National System for Geospatial Intelligence (NSG) Geospatial Core Metadata Profile<br>FG3D - NGA Foundation for GEOINT 3D<br>*NoMetadata* |

#### 7.9.3.3. Link/Path to Global Metadata

| **Requirement Metadata3** | **/req/core/metadata-global** |
|---|---|
| A | Any CDB datastore *SHALL* specify a Global_Metadata path or link to the physical location of the global metadata. |

#### 7.9.3.4. Language for metadata

| **Requirement Metadata4** | **/req/core/metadata-language** |
|---|---|
| A | Any CDB implementation, profile, extension, or application profile datastore *SHALL* use the same language. The language *SHALL* be consistent with IETF BCP 47. |
| B | The language *SHALL* be specified using the `language` metadata element as specified in the global metadata table (below). |

| **NOTE** | An IETF BCP 47 language tag is a standardized code or tag that is used to identify human languages in the Internet. The tag structure has been standardized by the Internet Engineering Task Force (IETF) in Best Current Practice (BCP) 47; the subtags are maintained by the IANA Language Subtag Registry. The actual Best Practice language is captured in RFC 5646. |
|---|---|

#### 7.9.3.5. Metadata encoding consistency

| **Requirement Metadata5** | **/req/core/metadata-encoding** |
|---|---|

| | |
|---|---|
| A | Any CDB implementation, profile, extension, or application profile datastore *SHALL* use the same encoding for all metadata instances in a CDB datastore. Possible values are `xml`, `json`, or `gpkg`. |

#### 7.9.3.6. Date and Time Metadata

The following is the requirement for expressing date and time in any CDB metadata - whether at the global or resouorce level. This requirement is consistent with various OGC and ISO standards as well as with STAC.

| **Requirement Metadata6** | **/req/core/metadata-datetime** |
|---|---|
| A | Searchable date and time. This may be a time instance or time period. The specification of date and time in any CDB metadata *SHALL* be specified in UTC. |
| B | Date and Time *SHALL* be formatted according to RFC 3339, section 5.6. RFC 3339 is a profile of the ISO 8601 standard for representation of dates and times using the Gregorian calendar. |

#### 7.9.3.7. Temporal Intervals

The following requirement defines how temporal (time) intervals are specified in a given CDB datastore. A given CDB Application Profile may further restrict how temporal intervals are specified in a CDB datastore. This requirement is exactly the same as specified in the (draft) OGC API - Records standard.

| **Requirement Metadata7** | **/req/core/metadata-temporal-interval** |
|---|---|
| A | Temporal geometries are either a date-time value or a time interval. The parameter value *SHALL* conform to the following syntax (using ABNF):<br>interval-bounded = date-time "/" date-time<br>interval-half-bounded-start = [".."] "/" date-time<br>interval-half-bounded-end = date-time "/" [".."]<br>interval = interval-closed / interval-half-bounded-start / interval-half-bounded-end<br>datetime = date-time / interval |

| **NOTE** | Half-bounded time intervals are supported using a double-dot (..) or an empty string for the start/end. |
|---|---|

### 7.9.4. Global metadata for UoM

| **Requirement Metadata8** | **/req/core/metadata-uom-measure** |
|---|---|

| | |
|---|---|
| A | Any CDB implementation, profile, extension, or application profile datastore *SHALL* use the same units of measure for measurements for the entire CDB datastore. Possible values are:<br>M = Meters<br>FT = Feet<br>K = Kilometer<br>MI = Miles |
| B | The metadata element name for encoding the UoM value *SHALL* be `uom` |

#### 7.9.4.1. Global Metadata elements

The following are the metadata elements that describe an entire CDB datastore instance. In the following table, the first element name is from DCAT and the second element name is from ISO 19115: Metadata. In CDB 1.3 and earlier, global metadata were stored and accessible from /CDB/metadata. If using a file systems approach for a CDB datastore, then global metadata in CDB 2.0 and later may still be stored at /CDB/metadata. In this case, the Global_Metadata path or link to the physical location of the global metadata would be /CDB/metadata (See Metadata3 Requirement).

| Metadata element Name | DCAT and 19115 Names | Description | Mandatory or Optional |
|---|---|---|---|
| ID | `dct:identifier`, MD_Metadata.metadata Identifier | A unique identifier for the entire CDB data store instance. This identifier is persistent and is considered global metadata. For example, this could be a Digital Object Identifier (DOI). | Mandatory |
| title | `dct:title`, CI_Citation.title) | Title by which the datastore is known. For global metadata describing a CDB data store, this would be a name given to the entire data store. For example, “Yemen demonstration CDB data store.” | Mandatory |
| description | `dct:description` [1], MD_DataIdentification. abstract | Detailed multi-line description to fully explain the datastore. | Mandatory |

| | | | |
|---|---|---|---|
| contactPoint | `dcat:contactPoint`, MD_Metadata.contact/CI_ResponsibleParty | Name of the person, position, or organization responsible for the resource. This is a text string. An example of a point of contact could be “Flight Safety” or “CAE.” | Mandatory |
| created | `dcterms:created` or `dcterms:issued`, CI_Citation.date | A date which is used to help identify the datastore. For global metadata, this is the date that the CDB data store was created or issued. See Requirement Metadata6 above | Mandatory |
| language | `dct:language`, PT_Locale | The language and character set used for the datastore (if a language is used). NOTE: See Metadata requirement 4 above. | Mandatory |
| update | `dct:issued`, MD_Metadata.dateInfo | Date of last update to the CDB datastore. Note: Date gives values for year, month and day. Character encoding of a date is a string that shall follow the format for date as specified in Requirement Metadata6 above. | Optional |
| temporal | `dct:temporal`, EX_TemporalExtent | The temporal extent of the datastore. For a CDB datastore, this would be the temporal range of when the data store was initially created to the point where the most recent content was created. | Optional |

| | | | |
|---|---|---|---|
| accessRights | `dct:accessRights`, MD_SecurityConstraints | Security restrictions on the access and use of the datastore. These would be constraints for an entire CDB data store. This could be information necessary to generate an EDH compliant encoding. | Optional |
| license | `dct:license`, MD_LegalConstraints | A sub-class of all access constraints. These legal constraints include copyright, patent, patent pending, trademark, license, Intellectual Property Rights, restricted, and other. At the global level, these are legal constraints applicable to an entire CDB data store. | Optional |

#### 7.9.4.2. Resource Metadata (aka dataset metadata)

| | |
|---|---|
| **NOTE** | Should - for purposes of CDB - this be termed dataset metadata (dataset is part of the DCAT superclass Resource which also includes the super-class of dcat:Dataset, dcat:DataService, dcat:Catalog and any other member of a dcat:Catalog)? |

The following are the mandatory and optional metadata elements that describe an individual CDB dataset (resource). These elements are consistent with the STAC and OGC API Records dataset elements.

| Field Name | Description | DCAT and ISO 19115 alignment | Mandatory or Optionsal |
|---|---|---|---|
| ID | A unique identifier for the resource. This resource identifier is persistent. | `dcat:Identifier`, `MD_Metadata.metadataIdentifier` | Mandatory |
| type | Denotes the resource type of the resource. For the dataset this is **required** to be `dataset`. | `dcat:Dataset` | Mandatory |

| | | | |
|---|---|---|---|
| title | A short descriptive human-readable one-line title for the Resource. | `dcterms:title` | Mandatory |
| description | Detailed multi-line description to fully explain the Resource. | `dcterms:description` | Mandatory |
| keywords | List of keywords describing the Resource. | `dcat:keyword` (free-text - repeat if necessary) `dcat:theme` | Conditional |
| keywordsCodespace | A reference to a controlled vocabulary used for the keywords property. | `dcat:themeTaxonomy` | Optional (defaults to XXX) |
| externalId | Identifier for the resource that is unique across the provider. | `dcterms:identifier` | Optional |
| publisher | The entity making the resource available. | `dcterms:publisher` | Conditional |
| created | The date and time the collection represented by this record was created, formatted as specifed in Requirement Metadata5 above. | `dcterms:created` or `dcterms:issued` | Conditional |
| updated | The date and time this resource was updated, formatted to RFC 3339. | `dcterms:modified` | Optional |
| themes | A knowledge organization system used to classify the resource. | `dcat:themeTaxonomy` | Optional |
| formats | A list of available distributions for the resource. | `dcat:distribution` (pointer to description of serialized form) (repeat as required) | Optional |
| contactPoint | An entity to contact about the resource. | `dcat:contactPoint` | Conditional |
| license | A legal document under which the resource is made available. | `dcterms:license` | Conditional |

| | | | |
|---|---|---|---|
| rights | A statement that concerns all rights not addressed by the license such as a copyright statement. | `dcterms:rights` or `dcterms:accessRights` | Optional |
| extent | Spatial and temporal extents. | `dcterms:spatial` | Conditional |
| associations | A list of links for accessing the resource, links to other resources associated with this resource, etc. | `dcat:qualifiedRelation` `dcterms:relation` `dcat:accessURL` and a few other specific properties and property-paths | Optional |
| CharacterSetCode | A set of character coding standards that may be individually used by the text of a resource. May be one of utf8 or utf16 | MD_CharacterSetCode from ISO 19115-1:2014 | Optional. Default is utf8 |

**NOTE** Conditional and optional metadata elements may be stated as mandatory in other requirements modules or in application profiles.

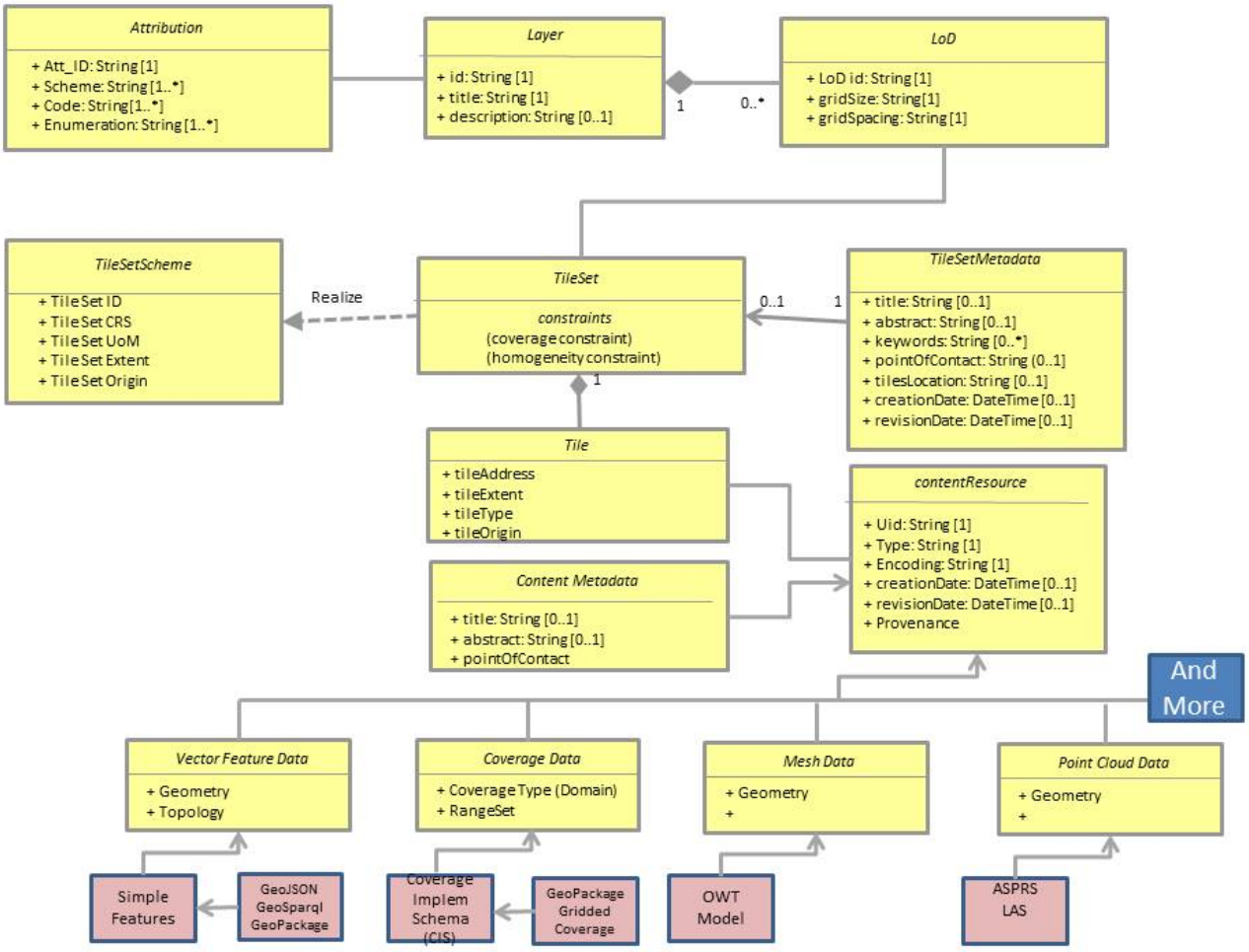
# 7.10. Tiling Requirements Module (Abstract)

At an abstract level, this core Tiling Requirements Module specifies the mandatory Tiling Requirements for any CDB data store that is to be tiled. If a CDB data set is to be tiled, then this Abstract Requirements module is applicable.

The requirements specified in this clause have their foundation in the OGC Abstract Specification Topic 22 - Core Tiling Conceptual and Logical Models for 2D Euclidean Space. This Abstract Specification consists of two Parts: A General Tiling Conceptual Model and, based on the Conceptual Model, a Logical Model for the Tessellation (Tiling) of 2D Euclidean Space. Tiling of 2D Euclidean space is the most commonly known approach to partitioning space in traditional geospatial technology. The requirements specified in this clause are data type agnostic and are applicable to any geospatial dta type as well as geospecific and geotypical models.

## 7.10.1. Tiling, LoD, and Layers Logical Model

The following figure, based on the Tiling Abstract Specification Logical Model, shows the properties by class (concept) and the relationships between the classes.




## 7.10.2. CDB Core Tiling Requirements Class and Requirements.

The CDB Abstract Tiling Requirements module is optional. However, if tiling is to be implemented in a CDB data store, then this is a mandatory requirements class. Any tiling implementation, profile, extension, or application profile shall conform with this requirements class and related conformance class.

**NOTE**

While the same tiling structure is used for any and all tile sets (layers) in a CDB data store, the TileSet metadata is provided for each individual TileSet (layer) in the data store. This is because each tile set has a unique identifier (ID), point of contact, date of creation, and so forth. Please see the examples related to requirement Tiling10. As such, Requirements Tiling1 through Tiling8 can be thought of "global" parameters stored once in the global metadata.

| **Requirements Class - Tiling-Abstract** | |
|---|---|
| /req/core/geometry- | |
| Target type | Operations |
| Dependency | OGC Abstract Specifcation Topic 22 - Tiling |
| Dependency | Core CRS Requirements |
| Dependency | Core Metadata Requirements |

| | |
|---|---|
| Requirement Tiling1 | /req/core/tiling-rule |
| Requirement Tiling2 | /req/core/tiling-model |
| Requirement Tiling3 | /req/core/tiling-conformance |
| Requirement Tiling4 | /req/core/tiling-tilingscheme-consistent |
| Requirement Tiling5 | /req/core/tiling-tilingscheme-crs |
| Requirement Tiling6 | /req/core/tiling-tilingscheme-uom |
| Requirement Tiling7 | /req/core/tiling-tilingscheme-extent |
| Requirement Tiling8 | /req/core/tiling-tilingscheme-definition |
| Requirement Tiling9 | req/core/tiling-tileset-metadata-standard |
| Requirement Tiling10 | /req/core/tiling-tileset-metadata-elements |
| Recommendation Tiling1 | /req/core/tiling-extension |

| | |
|---|---|
| **NOTE** | Need a requirement that states that the same tiling structure is used for any and all tiled content (layers). |

#### 7.10.2.1. When to tile rule

| **Requirement Tiling1** | **/req/core/tiling-rule** |
|---|---|
| A | Geospatial content, including vector data, coverages (such as elevation), and location specific models that are to be tiled *SHALL* be tiled according to the requirements specified in this abstract tiling requirements module. |

| | |
|---|---|
| **NOTE** | There are data types in CDB where data is not inherently geospatial, such as a library of models, that can be used/placed in different places in the world, or a library of model textures that are used in more than one model. These data types typically are not tiled. |

#### 7.10.2.2. Consistent with Topic 22

| Requirement Tiling2 | /req/core/tiling-topic22 |
|---|---|
| A | Any CDB implementation, profile, extension, or application profile that implements or references tiling (tiled data stores) *SHALL* be consistent with and conform to the terms and definitions and logical models as defined in OGC Abstract Specification Topic 22: OGC Abstract Specification Topic 22] - Core Tiling Conceptual and Logical Models for 2D Euclidean Space. |

#### 7.10.2.3. Conformance with this Clause

| Requirement Tiling3 | /req/core/tiling-conformance |
|---|---|
| A | Any CDB implementation, profile, extension, or application profile that implements or references tiling (tiled data stores) *SHALL* be consistent with and conform to all other requirements specified in this Tiling Requirements Module. |

#### 7.10.2.4. Tiling Scheme Restrictions

The following requirements restrict the Tiling Abstract Model.

##### 7.10.2.4.1. Tiling scheme definition consistent across entire CDB datastore.

| Requirement Tiling4 | /req/core/tiling-tilingscheme-consistent |
|---|---|
| A | A tiling in a CDB *SHALL* conform to the same TilingScheme definition (consistent tiling throughout a CDB datastore) as defined by the following requirements and in any tiling extension such as CDBGlobalGrid and GNOSISGlobalGrid. |

| NOTE | As per OGC Abstract Topic Volme 22, a TilingScheme is defined by the content defined in the TilingScheme class. |
|---|---|

##### 7.10.2.4.2. TilingScheme CRS restriction

This requirement restricts Topic 22 Requirement 3.

| Requirement Tiling5 | /req/core/tiling-tilingscheme-crs |
|---|---|
| A | Any CDB tiling scheme *SHALL* have a TilingScheme CRS that is the same as, or compatible with, the CRS specified in req/core/crsStorage. |

#### 7.10.2.4.3. TilingScheme UoM Restriction

This requirement restricts Topic 22 Requirement 4.

| Requirement Tiling6 | /req/core/tiling-tilingscheme-uom |
|---|---|
| A | Any CDB tiling scheme *SHALL* have a TilingScheme Units of Measure (UoM) that conforms to the UoM requirement as specified in /req/core/uom. This is the UoM as specified in the CRS metadata for a given CRS. For example, EPSG:4326 (2D WGS 84) specifies a UoM of decimal degrees. As such, a tiling scheme based on EPSG:4326 would have all coordinates and the tiling scheme definition, expressed in decimal degrees.<br><br>NOTE: This UoM is not for mensuration, heights, or depths. |

#### 7.10.2.4.4. TilingScheme Extent Restriction

This requirement restricts Topic 22 Requirement 5.

| Requirement Tiling7 | /req/core/tiling-tilingscheme-extent |
|---|---|
| A | Any CDB tiling scheme *SHALL* have a TilingScheme extent that cover the entire earth with no gaps. |

### 7.10.2.5. TilingScheme Definition

The following requirement focuses on the core metadata. The first states that any metadata encoded in a CDB datastore will use the same semantics and elements names as specified in a named metadata standard.

| NOTE | The approach is similar to what was implemented in CDB version 1.1 when the concepts of global and local metadata were specified. |
|---|---|

| Requirement Tiling8 | /req/core/tiling-tilingscheme-definition |
|---|---|
| A | Any CDB tiling scheme definition *SHALL* be included in the global CDB metadata that conforms with the metadata standard as specified in the req/core/metadata-global property specified in the global CDB metadata. |

### 7.10.2.6. TileSet Metadata

The following requirement focuses on the core metadata. The first states that any metadata encoded in a CDB datastore will use the same semantics and elements names as specified in a named metadata standard.

| NOTE | The approach is similar to what was implemented in CDB version 1.1 when the concepts of global and local metadata were specified. |
|---|---|

| Requirement Tiling9 | /req/core/tiling-tileset-metadata-standard |
|---|---|
| A | Any CDB tileset *SHALL* have metadata that conforms with the metadata standard as specified in the req/core/metadata-global property specified in the global CDB metadata. |

The following requirement specifies the mandatory minimal metadata element set for a CDB tileset.

| Requirement Tiling10 | /req/core/tiling-tileset-metadata-elements |
|---|---|
| A | Any CDB tileset *SHALL* have at a minimum TileSet metadata that provides the following Metadata elements:<br>* ID: A TileSet Identifier.<br>* Title: Title of this TileSet, normally used for display for a human.<br>* Description (Abstract): Brief narrative description of this TileSet, normally available for display for a human.<br>* Keywords: Unordered list of one or more commonly used or formalized word(s) or phrase(s) used to describe this dataset.<br>Note: The tiling scheme metadata and tile metadata SHALL comply with the dataset (resource) metadata scheme as specified in the Core Metadata Requirements Class. |

An example of the mandatory TileSet metadata elements is:

ID: 1
Title: RoadNetwork
Description: This TileSet (layer) contains the CDB road and highway network.
Keywords: roads, streets, highways, trails.

#### 7.10.2.7. Tiling/Lod Recommended Approach

To meet a stated design goal of interoperability and ease of transformations between CDB data store implementations and profiles, the CDB core standard recommends implementation of and conformance with one of the specified CDB Core Tiling Extensions, both based on the OGC Two-dimensional TileMatrixSet and TileSet metadata standard 2.0.

| Recommendation Tiling1 | */rec/core/tiling-extension |
|---|---|
| A | Any CDB tiling scheme *SHOULD* comply with one of the following tiling extensions:<br>- CDB CDB1GlobalGrid Tiling Extension<br>- CDB GNOSISGlobalGrid Tiling Extension |

# 7.11. Tiling Extension (TE) - CDB Global Grid

The CDB Global Grid Requirements Module defines the requirements for implementing a global tiling scheme hereafter termed `CDB1GlobalGrid`. This is a recommended extension to the CDB Core Tiling Requirements Module. This extension is applicable for any tiled CDB layer contained in a CDB data store. Of special note is that this tiling extension is backwards compatible with the current tiling structure as specified in the CDB 1.x series of standards. However, other tiling extensions to the Core can be defined using the same model as described below.

The requirements specified in this extension are based on the OGC Two Dimensional Tile Matrix Set and Tile Set Metadata Standard (2DTMS). Any developer wishing to work with and implement the CDB tiling structure should read Clause 6 of 2DTMS.

| | |
|---|---|
| **NOTE** | While this Tiling extension is optional, any implementations or application profiles using this extension must comply with the requirements stated in the Requirements Class and the related Conformance Class. |
| **NOTE** | For the purposes of this extension, a tile matrix is equivalent to the definition of a tile set as defined in Topic 22. Further, a Tile Matrix Set is equivalent to a series of tile sets at different but related Levels of Detail (LoDs) that comprise a tile pyramid (see figure below). In general, all layers structured as tile sets/tile matrix sets have identical tiling schemes but may have unique metadata, such as point of contact, tile set ID, and date of creation. Also note that each tiled layer, such as railroads or elevations, is defined as a unique tile set/tile matrix set. |

[Pyramid] | *../../images/Pyramid.jpg*

An example of a tile matrix set (aka pyramid): Source, OSGEO GDAL 2022

## 7.11.1. CDB Tiling and the OGC Two Dimensional Tile Matrix Set and Tile Set Metadata Standard (Informative)

Version 1 of OGC Two Dimensional Tile Matrix Set (2DTMS) defines the rules and requirements for a tile matrix set as a way to index space based on a set of regular grids defining a domain (tile matrix). Version 1 provided a limited list of scales in a Coordinate Reference System (CRS) as defined in [OGC 08-015r2] Abstract Specification Topic 2: Spatial Referencing by Coordinates. The latest revision of 2DTMS titled OGC Two Dimensional Tile Matrix Set and Tile Set Metadata provides numerous changes including detailed guidance and examples for specifying a tiled CDB datastore.

A key concept in 2DTMS that is relevant to the recommended CDB tiling scheme (CDB1GlobalGrid) is Variable width tile matrices. Many tiling schemes assume that matrixWidth is constant for all tile rows. This is common usage for projections that do not distort the Earth too much. However, when using any regular tessellation, the distortion increases for tiles closer to the poles. In the extreme, the upper row of the upper tile (the one representing the North Pole) contains a list of repeated values that represents almost the same position in the space.

The solution consists of reducing the number of tiles (matrixWidth) in the high latitude rows and generating those tiles with a compressed scale in the i dimension (see Figure 5). To allow this

solution, the tile model must be extended to specify coalescence coefficients (c) that reduce the number of tiles in the width direction by aggregating *c* horizontal tiles but keeping the tileWidth (and tileHeight). The coalescence coefficient is not applied next to the Equator but is used in medium and high latitudes (the higher the latitude the larger the coefficient).

### 7.11.2. CDB 2.0 CDB1GlobalGridTiling Extension overview (Informative)

`CDB1GlobalGrid` The CDB1 Global Grid Tile Matrix Set defines tiles in the Equirectangular Plate Carrée projection (EPSG:4326 CRS) for the whole world, based on the level of details and zones defined by the OGC CDB 1.x Standards, which can be encoded following this standard with the use of variable widths. For the tile matrices with negative identifiers of the CDB1GlobalGrid, the tiles' geographic extents remain the same as those of tile matrix 0, but the tile size in cells is reduced. For the CDB1GlobalGrid, the polar adjustment zones corresponding to coalescence factors are the same (at a given latitude) for all tile matrices of the set.

Specifically, for the definitions and examples of a CDB1GlobalGrid, see:

- Annex E2: CDBGlobalGrid Tile Matrix Set definition
- Annex H: Example encodings for TileSetMetadata (Informative).
- Annex G2: CDB Global Grid JSON and XML encoding examples.

Further, Annex E of 2DTMS includes definitions for TileMatrixSets utilizing the variable width capability. This capability allows for the coalescence of tiles in specific rows so that the overall width of the tile matrix is divided among fewer tiles. This is particularly well suited to define global grids in an Equirectangular Plate Carrée projection, where tiles closer to the pole cover a smaller physical area than tiles at the equator.

At each successive zoom level, each lower level tile is split into four new tiles at the next level of detail (zoom level). This subdivision can continue until the zoom level is high enough to accommodate the highest resolution data that is to be stored within a CDB X data store.

Annex G.2 of the 2DTMS Standard provides JSON and XML examples for the CDBGlobalGrid. Using the approach defined in Annexes E and G, one can define an arbitrary number of zoom levels. In the examples, 3 zoom levels are illustrated.

These are the JSON and XML definitions of the CDB1GlobalGrid tile matrix set (see CDB1GlobalGrid) that can be reproduced by other standards needing to define a tile matrix set. Not all TileMatrix elements need to be included and including other TileMatrices for more detailed scales is possible if they follow the same pattern.

The following figures depict the CDB 1.x zones, determining the coalescence coefficients, as well as the Levels of Detail, determining the tile matrices for the CDB1GlobalGrid.

[cdb zones] | *../../images/cdb-zones.jpg*

[cdb lod] | *../../images/cdb-lod.png*

### 7.11.3. Tiling Extension Requirements Class

The requirements specified in this Requirements Module are consistent with the guidance provided for the `CDB1GlobalGrid` as defined in Annexes E and G of the OGC Two Dimensional Tile Matrix Set and Tile Set Metadata Standard.

**NOTE** This CDB1GlobalGrid Tile Matrix Set defines tiles in the Equirectangular Plate Carrée projection (EPSG:4326 CRS) for the whole world, based on the level of details and zones defined by the OGC CDB 1.x standards, which can be encoded following requirements defined in this extension with the use of variable widths as defined in 2DTMS.

| **Requirements Class - Tiling Extension** | |
|---|---|
| /req/core/geometry- | |
| Target type | Operations |
| Dependency | OGC Abstract Specification Topic 22 - Tiling |
| Dependency | Core CRS Requirements |
| Dependency | Core Metadata Requirements |
| Dependency | Core Tiling Class |
| Dependency | 2DTMS Annex E: Variable grids - CDBGlobalGrid |
| Dependency | 2DTMS Annex G: CDBGlobalGrid encodings |
| Requirement TCE1 | /req/core/tiling-extension-conformance |
| Requirement TCE2 | /req/core/tiling-extension-tms |
| Requirement TCE3 | /req/core/tiling-extension-crs |
| Requirement TCE4 | /req/core/tiling-extension-uom |
| Requirement TCE5 | /req/core/tiling-extension-metadata |
| Requirement TCE6 | /req/core/tiling-extension-lod-0 |
| Requirement TCE7 | /req/core/tiling-extension-tile-tessellate |

#### 7.11.3.1. Conformance with this Clause

| **Requirement TCE1** | **req/core/tiling-extension-conformance** |
|---|---|

| | |
|---|---|
| A | Any CDB implementation, profile, extension, or application profile that requires tiled data stores that are backwards compatible with CDB 1.x tiling *SHALL* be consistent with and conform to all requirements specified in this Tiling Extension Requirements Module. This Tiling Extension requirements module is a restriction (profile) of the Core Tiling Requirements Module (Abstract). |

#### 7.11.3.2. Conformance with OGC Tile Matrix Standard

| **Requirement TCE2** | **req/core/tiling-extension-tms** |
|---|---|
| A | Any CDB implementation, profile, extension, or application profile implementing this extension *SHALL* be consistent with and conform to all requirements in this CDB1GlobalGrid Tiling Extension Requirements Module by application of specific 2DTMS requirements as provided below and as defined in the CDB1GlobalGrid specified in Annex H. |

#### 7.11.3.3. Conformance with EPSG:4326 Coordinate Reference System

| **Requirement TCE3** | **req/core/tiling-extension-crs** |
|---|---|
| A | Any CDB implementation, profile, extension, or application profile that requires tiled data stores *SHALL* specify the use of EPSG:4326 as defined at "http://www.opengis.net/def/crs/EPSG/0/4326" with axis order latitude, longitude. |
| B | Further, any implementation, application profile, or extension *SHALL* be consistent with and conform to the CDB Core CRS Requirements Module. |

#### 7.11.3.4. Units of measure rule for tile orgins, tile boundaries, and so forth

| **Requirement TCE4** | **req/core/tiling-extension-uom** |
|---|---|
| | Any CDB implementation, profile, extension, or application profile that requires tiled data stores *SHALL* specify any tile origins, tile extents, tileset extents, and any other bounding box metadata in decimal degrees. |

#### 7.11.3.5. Level of Detail (LoD) level 0 rule

| **Requirement TCE6** | **req/core/tiling-extension-lod-0** |
|---|---|
| A | The Level 0 (LoD) tiling of a CDB data store *SHALL* be a set of tiles with each tile being 1 degree in longitude and latitude in extent. In the CDB 1.x standard this is known as `Geocell`. |
| B | This *SHALL* be referred to as LoD 0 (aka zoom level 0 and tile matrix identifier "0"). |

| | |
|---|---|
| C | As per the CDB 1.x standard's requirements, at LoD 0 the matrix width SHALL be 360 and the matrix height SHALL be 180. |

#### 7.11.3.6. LoD Hierarchy/Tessellation rule

| **Requirement TCE7** | **req/core/tiling-extension-tile-tessellate** |
|---|---|
| A | A CDB datastore tiled using the CDB1GlobalGrid TilingScheme *SHALL* have an LoD range from -10 to 23. |
| B | A tile from LOD –10 to LOD 0 SHALL occupy the area of exactly one CDB Geocell. This is true for all CDB Geocells from all CDB Zones. Starting with LOD 1 and up, this area SHALL recursively be subdivided into smaller tiles, every level corresponding to a finer representation of the previous level allowing for multiple levels of detail. |
| C | LoDs of level 0 and higher *SHALL* have Tile sizes of 1024 × 1024. |

# 7.12. Tiling Extension (TCE) - GNOSISGlobalGrid

The Gnosis Global Grid Requirements Module defines the requirements for implementing the GNOSISGlobalGrid. This is a recommended extension to the CDB Core Tiling Requirements Module. This extension is applicable for any tiled CDB layer contained in a CDB data store. Of special note is that this tiling extension IS NOT compatible with the OGC CDB 1.x tiling structure.

The requirements specified in this extension are based on the OGC Two Dimensional Tile Matrix Set and Tile Set Metadata Standard (2DTMS). Any developer wishing to work with and implement the CDB tiling structure should read Clause 6 of the 2DTMS Standard.

| | |
|---|---|
| **NOTE** | While this Tiling extension is optional, any implementations or application profiles using this extension must comply with the requirements stated in the Requirements Class and the related Conformance Class. |

| | |
|---|---|
| **NOTE** | For the purposes of this extension, a tile matrix is equivalent to the definition of a tile set as defined in OGC Abstract Specification Topic 22. Further, a Tile Matrix Set (TMS) is equivalent to a series of tile sets at different but related Levels of Detail (LoDs) that comprise a tile pyramid (See figure below) |

[Pyramid] | *../../images/Pyramid.jpg*

An example of a tile matrix set (aka pyramid): Source, OSGEO GDAL 2022

## 7.12.1. CDB Tiling and the OGC Two Dimensional Tile Matrix Set and Tile Set Metadata Standard (Informative)

The OGC Two Dimensional Tile Matrix Set and Tileset Metadata (2DTMS) Standard defines the rules and requirements for a tile matrix set as a way to index space based on a set of regular grids (tile matrices) defining a domain.

A key concept in 2DTMS that is relevant to tiling CDB data is variable width tile matrices. Many tiling schemes assume that the tile width is constant for all tile rows. This is common usage for projections that do not distort the Earth too much. However, when using Equirectangular Plate Carrée projection the distortion increases for tiles closer to the poles. In the extreme, the upper row of the upper tile (the one representing the North Pole) contains a list of repeated values that represents almost the same position in the space.

The solution consists of reducing the number of tiles (the matrixWidth property in the 2DTMS Standard) in the high latitude rows and generating those tiles with a compressed scale in the i dimension. To support this solution, the tile model must be extended to specify coalescence coefficients (c) that reduce the number of tiles in the width direction by aggregating *c* horizontal tiles but keeping the tileWidth (and tileHeight). The coalescence coefficient is not applied next to the Equator but is used in medium and high latitudes (the higher the latitude the larger the coefficient).

The latest revision of 2DTMS (see release notes) provides detailed description of variable width TileMatrixSets suitable for tiling CDB data stores, such as the GNOSIS Global Grid.

### 7.12.2. CDB 2.0 GNOSISGlobalGrid Tiling Extension overview (Informative)

The CDB GNOSISGlbalGrid Tiling Extension Requirements Module is based on the GNOSISGlobalGrid as specified in the OGC TileMatrixSet registry, and documented in informative annexes of the OGC Two Dimensional Tile Matrix Set and Tile Set Metadata[OGC 17-083r4] Standard (2022). Specifically, for the definitions and examples of a GNOSISGlobalGrid, see:

- Annex E1: GNOSISGlobalGrid Tile Matrix Set definition
- Annex H: Example encodings for TileSetMetadata (Informative).
- Annex G1: GNOSIS Global Grid JSON and XML encoding examples.

The GNOSISGlobalGrid Tile Matrix Set defines tiles in the Equirectangular Plate Carrée projection (EPSG:4326 CRS) for the whole world while still attempting to approximate equal area tiles, even near the poles, by leveraging the 2DTMS variable width capability supporting the coalescence of tiles in specific rows so that the overall width of the tile matrix is divided among fewer tiles.

This is particularly well suited to define global grids in an Equirectangular Plate Carrée projection, where tiles closer to the pole cover a smaller physical area than tiles at the equator.

Starting at the tile matrix with identifier 1, a variable matrix coalescence factor is applied for polar tiles to tend towards equal area.

Using the approach defined in Annexes E and G, one can define an arbitrary number of zoom levels if they follow the same pattern. In the examples, 3 zoom levels are illustrated.

Levels 0 to 28 are officially defined, reaching a 2:1 scale while the matrix, row and column identifiers can still fit together within a single 64-bit key.

This extension is easy to implement procedurally: It is a quad-tree starting with eight 90 x 90 degree tiles at level 0, with the exception that tiles touching a pole are split into 3 tiles rather than 4 — the part of the tile touching the pole is not split longitude-wise. Therefore, there will always be only 4

tiles touching each pole.

Note: The GNOSIS Global Grid scales match the GoogleCRS84Quad Well Known Scale Set, but starts at the third scale of the set.

Note: Aside from the variable width considerations, the only other difference with the WorldCRS84Quad TileMatrixSet is the fact that the latter starts at the second scale of the WKSS — its TileMatrix identifier 1 is equivalent to the GNOSISGlobalGrid TileMatrix identifier 0.

The following figure depicts the tiling configuration for Level 2 of the `GNOSISGlobalGrid` as defined in this extension.

[cdbxGGGTMS] | *../../images/cdbxGGGTMS.jpg*

### 7.12.3. Tiling Extension Requirements Class

The requirements specified in this Requirements Module are consistent with the guidance provided for the `GNOSIS Global Grid` as defined in Annexes E and G of the OGC Two Dimensional Tile Matrix Set and Tile Set Metadata Standard.

| **Requirements Class - Tiling Extension** | |
|---|---|
| /req/core/geometry- | |
| Target type | Operations |
| Dependency | OGC Abstract Specification Topic 22 - Tiling |
| Dependency | Core CRS Requirements |
| Dependency | Core Metadata Requirements |
| Dependency | Core Tiling Class |
| Dependency | 2DTMS TileMatrixSet |
| Dependency | 2DTMS VariableWidth |
| Dependency | OGC 2DTMS Registry: GNOSISGlobalGrid |
| Requirement TCE1 | /req/core/tiling-extension-conformance |
| Requirement TCE2 | /req/core/tiling-extension-tms |
| Requirement TCE3 | /req/core/tiling-extension-crs |
| Requirement TCE4 | /req/core/tiling-extension-uom |
| Requirement TCE5 | /req/core/tiling-extension-metadata |
| Requirement TCE6 | /req/core/tiling-extension-start-lod |

#### 7.12.3.1. Conformance with this Clause

| Requirement TCE1 | req/core/tiling-extension-conformance |
|---|---|
| | Any CDB implementation, profile, extension, or application profile that implements or specifies the use of the GNOSISGlobalGrid tiling scheme *SHALL* be consistent with and conform to all requirements specified in this Tiling Extension Requirements Module. This Tiling Extension requirements module is a restriction (profile) of the Core Tiling Requirements Module (Abstract) and the Two Dimensional Tile Matrix Set and Tile Set Metadata Standard (2DTMS). |

#### 7.12.3.2. Conformance with OGC Tile Matrix Standard and GNOSISGlobalGrid definition

| Requirement TCE2 | req/core/tiling-extension-tms |
|---|---|
| A | Any CDB implementation, profile, extension, or application profile implementing this extension *SHALL* be consistent with and conform to all requirements in the Tiling Extension Requirements Module by application of the TileMatrixSet and VariableMatrixWidth requirements as defined in the 2DTMS Standard. |
| B | Any CDB implementation, profile, extension, or application profile implementing this extension *SHALL* be consistent with and conform to the TileMatrixSet definition of the GNOSISGlobalGrid as specified in the OGC TileMatrixSet registry and documented in the informative annexes of the 2DTMS Standard. |

#### 7.12.3.3. Conformance with EPSG:4326 Coordinate Reference System

| Requirement TCE3 | req/core/tiling-extension-crs |
|---|---|
| A | Any CDB implementation, profile, extension, or application profile that specifies or implements this extension *SHALL* specify the use of EPSG:4326 as defined at "http://www.opengis.net/def/crs/EPSG/0/4326" with axis order latitude, longitude. |
| B | Further, any implementation, application profile, or extension *SHALL* be consistent with and conform to the CDB Core CRS Requirements Module. |

#### 7.12.3.4. Units of measure rule for tile orgins, tile boundaries, and so forth

| Requirement TCE4 | req/core/tiling-extension-uom |
|---|---|

| | |
|---|---|
| | Any CDB implementation, profile, extension, or application profile that specifies or implements this extension *SHALL* specify any tile origins, tile extents, tileset extents, and any other bounding box metadata in decimal degrees. |

#### 7.12.3.5. Level of Detail (LoD) level 0 rule

| **Requirement TCE6** | **req/core/tiling-extension-start-lod** |
|---|---|
| A | The CDB GNOSISGlobalGrid tiling scheme *SHALL* start with a 2x4 grid of tiles with each tile being 90 degrees in longitude and latitude in extent. |
| B | This *SHALL* be referred to as zoom level 0 (or tile matrix identifier "0"). |
| C | Tile splitting rules SHALL_ follow those specied in the 2DTMS GNOSISGlobalGrid annexes. |

### 7.12.4. Additional Information

The levels 0 to 5 are shown in 2DTMS Annex E.1. Unlike the CDBGlobalGrid, for the GNOSISGlobalGrid, the coalescence factors for the polar zones are re-adjusted at each tile matrix of the set. Levels 0 to 3 are shown on the following picture.

[ggg 0 3] | *../../images/ggg-0-3.png*

## 7.13. Topology Core Requirements Module

The CDB Topology Requirements Module specifies requirements and recommendations for using topology for vector data in a CDB datastore.

Topology is the branch of mathematics describing the properties of objects which are invariant under continuous deformation. For example, a circle is topologically equivalent to an ellipse because one can be transformed into the other by stretching. In geographic modelling, the foremost use of topology is in accelerating computational geometry. The constructs of topology allow characterization of the spatial relationships between objects using simple combinatorial or algebraic algorithms. Topology, realized by the appropriate geometry, also allows a compact and unambiguous mechanism for expressing shared geometry among geographic features. [GML, 2007]

Traditionally in GIS, Topology operations are used to manage shared geometry, define and enforce data integrity rules, support topological relationship queries and navigation, and build more complex shapes such as polygons from primitive ones such as lines. The use of topology also supports the detection and correction of digitizing errors and performing network analysis computations.

### 7.13.1. Topology concepts and terminology

In CDB 1.x, the concepts of topology, geometry, and features are "mixed" together and not treated as separate abstract classes. In reality, topology can be defined as a graph with no geometry and no

feature type designations. Therefore, in CDB 2.0 and later, the following note is relevant in understanding the topology requirements module:

| | |
|---|---|
| **NOTE** | Topological primitives are equivalent to but are not subclasses of geometric primitives. This is consistent with the view that topological complexes are constructed to optimize computational geometry procedures by the use of combinatorial algorithms. This also permits the creation of structures that ignore geometric constraints by using a topological complex that is not realized by a geometric complex. [ISO 19107:2017] |
| **NOTE** | A reading of the CDB 1.x Topology clauses and requirements suggests that CDB 1.x requirements are not in conflict with ISO 19107:2017 or the requirements as specified in this CDB 2.0 Topology Requirements Module. |

[topology class diagram]

**Figure:** Topology Class Diagram. Source: ISO 19107:2017

The following are key definitions used in the CDB Core Topology Requirements Module. These definitions are extracted from ISO 19107:2017.

`direct position`
Position described by a single set of coordinates within a coordinate reference system (CRS).

`directed edge`
Directed topological object that represents an association between an edge and one of its orientations.

Note 1 to entry: A directed edge that is in agreement with the orientation of the edge has a + orientation, otherwise, it has the opposite (–) orientation. Directed edge is used in topology to distinguish the right side (–) from the left side () of the same edge and the start node (–) and end node () of the same edge and in computational topology to represent these concepts.

`directed face`
Directed topological object that represents an association between a face and one of its orientations.

Note 1 to entry: The orientation of the directed edges that compose the exterior boundary of a directed face will appear positive from the direction of this vector; the orientation of a directed face that bounds a topological solid will point away from the topological solid. Adjacent solids would use different orientations for their shared boundary, consistent with the same sort of association between adjacent faces and their shared edges. directed faces are used in the coboundary relation to maintain the spatial association between face and edge.

`directed node`
Directed topological object that represents an association between a node and one of its orientations.

Note 1 to entry: Directed nodes are used in the coboundary relation to maintain the spatial

association between edge and node. The orientation of a node is with respect to an edge, "+" for end node, "–" for start node. This is consistent with the vector notion of "result = end - start".

`edge`
1-dimensional topological primitive.

Note 1 to entry: The geometric realization of an edge is a curve. The boundary of an edge is the set of one or two nodes associated with the edge within a topological complex.

`edge-node graph`
Graph embedded within a topological complex composed of all of the edges and connected nodes within that complex.

Note 1 to entry: The edge-node graph is a subcomplex of the complex within which it is embedded.

`end node`
Node in the boundary of an edge that corresponds to the end point of that edge.

`face`
2-dimensional topological primitive.

Note 1 to entry: The geometric realization of a face is a surface. The boundary of a face is the set of directed edges within the same topological complex that are associated with the face via the boundary relations. These can be organized as rings (aka polygons).

interior Set of all direct positions that are on a geometric object but which are not on its boundary. In traditional GIS terminology, an interior is an island or hole contained within a polygon.

`isolated node`
Node not related to any edge. In traditional GIS, an isolated node with a `direct position` property is known as a point geometry.

`start node`
Node in the boundary of an edge that corresponds to the start point of that edge as a curve.

[topology elements]

**Figure:** Example of Topology Primitives and related geometry elements.

In the above figure, the topological and related geometries are depicted. The tables are provided as examples of how the topological information could be organized. However, specific table organization is beyond the scope of the CDB Core Topology Module.

In the CDB 2.x context:

A **node primitive** is a 0 dimensional topological primitive. The node may have or not have additional properties such as height or depth and other attributes, such as a feature code or type. A node may or may not be connected to a network. An isolated node typically is a point feature with an associated coordinate (`direct position`). An example might be a lighthouse or fire hydrant. A node that has one or more associated edges is a directedNode and represents a node in a directed graph. A directed node differentiates between edges that start at the node and those adjacent edges

that terminate at the node.

An **edge** is a one dimensional topological primitive defined by a set of linestring (curve) primitives that define the geometry connecting two nodes. A directedEdge begins at a start node and terminates at an end node. These start and end nodes are defined as directedNodes: The node is with respect to an edge, "+" for end node, "–" for start node. It is possible for two nodes to be connected with no geometry. An example is a straight line road segment between two road intersections. In this case, the line geometry is defined by the start and end nodes.

A **face** is a two dimensional topological primitive. The geometric realization of a face is a surface. The boundary of a face is the set of directed edges within the same topological complex that are associated to the face via the boundary relations. These can be organized as rings or as they are more traditionally known, polygons.

At the fundamental level, topology is an edge-node graph of primitive concepts with no geometry or attributes other than identifiers. Edge-node graphs are very useful in many applications. However, in the geospatial world the majority of topologically structured networks exist in relation to the surface of the earth and have geometries. The geometries specified for use in CDB 2.x are defined in the CDB Core Geometry Model Requirements Module.

### 7.13.2. Topology, Geometry, and Features

Topology is represented as a graph. A "pure" topology graph does not have a coordinate system nor is there the concept of geometry or coordinates. The addition of spatial coordinates ties the graph to some coordinate system. For CDB all coordinates, bounding boxes, and so forth are expressed in the coordinate reference system as defined in the CDB Core CRS Requirements Module. The addition of coordinates to the model enables expressing nodes as points, edges as linestrings, and faces as polygons. The definition of any geometry in a CDB data store is defined in the CDB Core Geometry Requirements Module. While topology and geometry are useful in many applications, the majority of user facing GIS applications also have attributes associated with a geometry and may also have metadata. At this point, such entities are typically described as features.

### 7.13.3. Crosswalk with 1.x Terminology

The following table provides a terminology crosswalk between what is defined in the CDB 2.x Standard and what is defined in the Topology Clause of the CDB 1.x Standard. While there is not a one to one mapping, the concepts are closely aligned. The exception is the case for using Junction ID (JID) where the JID (node) is for relating two different feature types, such as a road intersecting a river. The other JID use case in 1.x is for a change of feature code within a network, such as a road changing from 2nd class to 3rd class. In 2.x, the case would just be a node primitive and not have a special ID.

| CDB 2.x | CDB 1.x |
|---|---|
| Start Node | Start Junction ID (SJID) |
| End Node | End Junction ID (EJID) |

| | |
|---|---|
| Isolated Node | When no ID specified (i.e., blank), the feature is not connected to any other features. A point geometry. |
| Edge | Series of points connected together using lineal and polygon features. |

## 7.13.4. Topology Core Requirements

The following specifies the set of requirements and recommendations that comprise the CDB Core Topology Requirements Module

| **Requirements Module - Topology.** | |
|---|---|
| /req/core/topology | |
| Target type | Operations |
| Dependency | OGC Abstract Specification Topic 1: Spatial Schema (ISO 19107:2017) |
| Requirement 1 | /req/core/topology-topic1 |
| Requirement 2 | /req/core/topology-nodeid |
| Requirement 3 | /req/core/topology-edgeid |
| Requirement 4 | /req/core/topology-node-dir |
| Requirement 5 | /req/core/topology-edge-dir |
| Requirement 6 | /req/core/topology-edge-clip |

Each of the core topology requirements is now described.

#### 7.13.4.1. Conformant with ISO 19107:2017 (aka OGC Abstract Specification Topic 1)

| | |
|---|---|
| **Topology Requirement 1** | **/req/core/topology-topic1**<br>Any CDB implementation, profile, extension, or application profile *SHALL* be consistent with and conform to the terms and definitions and logical models as defined in OGC Abstract Specification Topic 1: Spatial Schema (ISO 19101:2017) AND *SHALL* comply with all other requirements specified in the Topology Requirements Module. |

#### 7.13.4.2. Unique Node Identifiers

In order to uniquely identify any node in graph, each node shall have a unique identifier. This core standard does not specify how the identifier is to be structured. However, examples could be an integer number or a combination of a tile identifier and an integer number. In CDB version 1.x, the node identifiers were knowns as the SJID, EJID or JID attributes.

| | |
|---|---|
| **Topology Requirement 2** | **/req/core/topology-nodeID**<br>Any node in a topologically structured vector dataset *SHALL* have a unique identifier. |

#### 7.13.4.3. Unique Edge Identifiers

In order to uniquely identify any edge in graph, each node shall have a unique identifier. This core standard does not specify how the identifier is to be structured. However, examples could be an integer number or a combination of a tile identifier and an integer number.

| | |
|---|---|
| **Topology Requirement 3** | **/req/core/topology-edgeID**<br>Any edge in a topologically structured vector dataset *SHALL* have a unique identifier. |

#### 7.13.4.4. Directed Nodes

The following requirement is for any node that has one or more edge elements associated with that node. Any node with no connected edges is termed an `isolated node` and as such this requirement does not apply. A `directed node` has edges "entering" the node and "leaving" the node. This information needs to be part of the vector dataset.

| | |
|---|---|
| **Topology Requirement 4** | **/req/core/topology-node-dir**<br>Any node in a topologically structured vector dataset that has associated edges *SHALL* be termed a `directed node` and shall mark the edge ID's with a "-" sign if the edge is leaving the node and a "+" if the edge enters the node. |

#### 7.13.4.5. Directed Edges

The following requirement is for any edge that has two elements associated with that edge. This includes artificial (virtual) nodes generated by processes that clip edges to tile boundaries.

| | |
|---|---|
| **Topology Requirement 5** | **/req/core/topology-edge-dir**<br>Any edge in a topologically structured vector dataset that has associated nodes *SHALL* be termed a `directed edge` and *SHALL* include the node ID's of the start node and the end node. |

#### 7.13.4.6. Faces - Recommendation

Faces are a two dimensional topological primitive that is generated by traversing a set of edges to generate an exterior boundary (aka polygon). Generation of faces is not required unless the application profile specifies the generation of polygon coverages as defined in ISO 19123:2005 - Schema for coverage geometry and functions. The Core Topology Requirements module does not specify winding order for the face. Further, the CDB Core does not specify how topology traversal is accomplished nor how the face "tables" are structured. These are considerations for CDB Application Profiles and specific implementations.

| | |
|---|---|
| **Topology Recommendation 1** | **/rec/core/topology-face**<br>If faces are generated, then the face primitive *SHALL* be defined as a list of `directed nodes` and `directed edges`. |

#### 7.13.4.7. Clipping edges at tile boundaries.

A CDB Topological network is broken into tiles. Therefore, any edge crossing a tile boundary will need to be clipped and a new node generated for the directed point generated by the clip operation.

| | |
|---|---|
| **NOTE** | Clipping sounds trivial and the mathematics is trivial. However, there are numerous "corner" cases such as an edge that ends right at the corner of a tile. There are also issues of precision and accuracy that may cause the location of the node's direct position to be slightly different from one tile to the next. These are implementation considerations and as such are not considered in the Core Topology Module. |

| | |
|---|---|
| **Topology Requirement 6** | **/req/core/topology-clip**<br>Any edge that crosses a tile boundary *SHALL* be clipped against tile boundaries. To ensure the connectivity between tile boundaries of any lineal or polygonal feature, the resulting node generated by the clip operation *SHALL* share the same node identifier in both tiles |

### 7.13.5. Face Topology Primitive - Optional

Faces are a two dimensional topological primitive that is generated by traversing a set of edges to generate an exterior boundary (aka polygon). Generation of faces is not required unless the application profile specifies the generation of polygon coverages as defined in ISO 19123:2005 - Schema for coverage geometry and functions. The Core Topology Requirements module does not specify winding order for the face. Further, this core module does not specify how topology traversal is accomplished nor how the face "tables" are structured. These are considerations for CDB Application Profiles and specific implementations.

The generation of faces is an optional requirements class. Traversal of the topology graph to form faces in order to create (when combined with geometry) polygons requires conformance to this class. Traversal of the graph to detect topological consistency errors does not require conformance to this class.

#### 7.13.5.1. Relation of topological face to a polygon.

The geometric realization of a face is a surface. The boundary of a face is the set of directed edges within the same topological complex that are associated to the face via the boundary relations. The term `boundary` is most commonly used in the context of geometry, where the set is a collection of points or a collection of objects that represent those points. The extent of a polygon in 2D is totally defined by its boundary.

In more simple terms [Source: 1-Spatial]:

Faces exist in planar topology only and represent the interior of an area formed by enclosing edges. A polygon is represented by one or more faces.

The edges describe the outer and inner boundaries of the face geometry and any edges contained by, but not bounding, the face.

To ensure that faces never overlap, edges cannot cross edges. If an edge does cross another edge, a node is created at the junction and each edge is split into two edges.

Adjacent faces share an edge, but the edge is used in opposite directions when defining such entities as a polygon complex.

| **Requirements Module - Topology Faces.** | |
|---|---|
| **/req/core/topology-face** | |
| Target type | Operations |
| Dependency | OGC Abstract Specification Topic 1: Spatial Schema (ISO 19107:2017) |
| Requirement Face1 | /req/core/topology-topic1 |
| Requirement Face2 | /req/core/topology-faceid |
| Requirement Face3 | /req/core/topology-face-structure |
| Requirement Face4 | /req/core/topology-winding |

#### 7.13.5.2. Conformance with ISO 19107:2017

As with all other topological primitives, if faces are to be generated, then the application profile and/or implementation has to be consistent with ISO 19107:2017

| | |
|---|---|
| **Face Topology Requirement 1** | **/req/core/topology-topic1**<br>Any CDB implementation, profile, extension, or application profile that implements faces *SHALL* be consistent with and conform to the terms and definitions and logical models as defined in OGC Abstract Specification Topic 1: Spatial Schema (ISO 19101:2017) AND *SHALL* comply with all other requirements specified in the Topology Requirements Module. |

#### 7.13.5.3. Unique Face ID

As with Node and Edge primitives, every Face has a unique identifier.

In order to uniquely identify any face in graph, each face shall have a unique identifier. This core standard does not specify how the identifier is to be structured. However, examples could be an integer number or a combination of a tile identifier and an integer number.

| | |
|---|---|
| **Face Topology Requirement 2** | **/req/core/topology-faceID**<br>Any face in a topologically structured vector dataset *SHALL* have a unique identifier. |

#### 7.13.5.4. Structure of a Face primitive

| | |
|---|---|
| **Face Topology Requirement 3** | **/rec/core/topology-face-structure**<br>If faces are generated, then the face primitive *SHALL* be defined as a list of `directed nodes` and `directed edges`. |

#### 7.13.5.5. Face winding order

Winding order refers to the "direction" used to generate a Face and create the subsequent list of node IDs and Edge IDs that define the face. Winding order is either clockwise or counterclockwise.

| | |
|---|---|
| **Face Topology Requirement 4** | **/rec/core/topology-face-winding**<br>If faces are generated, then the winding order *SHALL* be defined and documented in the metadata for the topologically structured data set. |

#### 7.13.5.6. Islands/Holes

Quite often, a polygon geometry as defined by the face definition contains islands or holes. A typical example would be a lake geometry containing one or more "real" islands. In ISO 19107 terms, islands are interior boundaries. While the ability to define, store, and process islands is a critical feature for any GIS, this CDB Core requirements module does not specify how islands are associated with the parent polygon geometry (exterior boundary). If required, such associations should be defined in an CDB application profile.

### 7.13.6. An example logical model for topologically structured vector data

The following diagram is extracted from the 1984 Map Overlay and Statistical System Topology Implementation Study funded by the USGS. This diagram represents are fairly typical logical model for structuring topological relationships and content for use in a datastore. There are other similar examples from the late 1970's through the 1980's.

[topo database example]

**NOTE** Due to the requirement that a typical GIS needs to support both geometry and features, this diagram is more extensive than just showing pure topology. This is why such concepts as cartographic symbolization and labeling are shown. Also note that the "core" concept is `feature`.

## 7.14. Versioning

Versioning is used extensively in fielded CDB 1.x implementations. A primary principal of the CDB 1.x approach is the direct replacement of a file in a parent CDB datastore by an identically named file in a linked child dataset. Anyone at any level in the operation of the site of the simulators is able to assume the files are 'packaged' identically in every CDB dataset, whether it was produced locally, or produced as a product purchased, or acquired through sharing, and so forth. Versioning is used to distribute a small change very rapidly. The versioning capability remains a critical component of the CDB 2.0 Standard. The Versioning Requirements Class specifies the core abstract requirements

for versioning of a CDB datastore.

In CDB 2.0 and later, the concept `versioning` represents a collection of one or more changes to content in the datastore, including modifications, additions, and deletions. A key aspect of versioning in CDB 2.0 is the provision of metadata that describes the change(s). A key global metadata element is the date and time of the collection of modification(s). Having such metadata enables the ability to view changes over time and/or rollback to previous versions of the entire datastore or a given asset, such as a specific building, in the datastore.

| NOTE | `Versioning` is not the CDB Version number. The CDB Version number is the version of the CDB Standard that a datastore is based on. These include 3.0, 3.1, 3.2, OGC 1.0, OGC 1.1, OGC 1.2, and OGC 1.3. Implementation of the CDB 2.0 Standard would extend this list to a CDB Version number 2.0 |
|---|---|

### 7.14.1. Versioning Requirements Class

| **Requirements Class - Versioning** | |
|---|---|
| /req/core/versioning | |
| Target type | Operations |
| Dependency | Target storage system. |
| Dependency | Core Metadata Requirements |
| Dependency | Resource File Conventions |
| Requirement Versioning1 | /req/core/versioning |
| Requirement Versioning2 | /req/core/version-collection |
| Requirement Versioning3 | /req/core/versioning-metadata |
| Requirement Versioning4 | /req/core/versioning-functions |
| Requirement Versioning5 | /req/core/versioning-file-replacement |
| Requirement Versioning6 | /req/core/versioning-transitory |

### 7.14.2. A CDB datastore implementation shall support versioning

Any CDB datastore implementation needs to provide some mechanism for applying and tracking changes to the assets maintained in a CDB datastore.

| **Requirement Versioning1** | **req/core/versioning** |
|---|---|

| | |
|---|---|
| A | Any CDB datastore implementation *SHALL* support the ability to apply and track any changes to a CDB datastore These include adding, deleting, and updating assets in the datastore (CRUD). |

### 7.14.3. Collection of changes

A revision to a CDB datastore may include one to many changes to one or more assets in the datastore. These changes include modifications to geometry, attributes or both, updates to models, deletions, and additions and more. The set of changes is referred to as a `versioning collection`.

| **Requirement Versioning2** | **req/core/versioning-collection** |
|---|---|
| A | Any CDB datastore implementation *SHALL* support the ability to apply and track any changes to a CDB datastore. |

### 7.14.4. Versioning metadata

| **Requirement Versioning3** | **req/core/versioning-metadata** |
|---|---|
| A | Anytime a versioning collection of changes is applied to a CDB datastore, the following metadata updates *SHALL* be implemented: |
| B | The CDB Global Metadata Element `Update` *SHALL* be updated to reflect the application of a versioning collection of changes. |
| C | The CDB Local Resource Metadata Element `Updated` *SHALL* be updated to document the date and time of the change to the asset. |

Need words on how to link the resource to additional information about the chnage to the asset, such as who did it and a description.

### 7.14.5. CRUD

Any CDB implementation needs to support the traditional CRUD functions include adding, deleting, and updating assets in the datastore.

| **Requirement Versioning4** | **req/core/versioning-functions** |
|---|---|
| A | Any CDB Implementation *SHALL* support creating (adding) assets in a CDB datastore. |
| B | Any CDB Implementation *SHALL* support deleting assets in a CDB datastore. |
| C | Any CDB Implementation *SHALL* support updating assets in a CDB datastore. |

### 7.14.6. Minimum update capability - File (Asset) replacement

For update at a minimum a CDB implementation needs to support a file/table replacement mechanism. A CDB file replacement allows content in a CDB datastore to be easily updated. Examples are textures, models, or terrain datasets that have been updated and/or edited.

| **Requirement Versioning5** | **req/core/versioning-file-replacement** |
|---|---|
| A | Any CDB datastore implementation *SHALL* at a minimum support the ability to replace a file (or table) in a CDB datastore. |

### 7.14.7. Transitory (ephemeral) Assets

Quite often an asset in a CDB datastore will change state. For example, a road may be temporarily closed or a bridge damaged by a flood. These transitory events need to be reflected in a CDB datastore so that end user clients or simulators can use that information to modify a visualization, an analysis, or a simulation.

| **Requirement Versioning6** | **req/core/versioning-transitory** |
|---|---|
| A | Any CDB datastore implementation *SHALL* at a minimum support the ability capture a change of state for any geospatial asset in a CDB datastore |

[1] DCAT does not have a concept “abstract”. Use description instead.

# Annex A: Abstract Test Suite (Normative)

## A.1. Introduction

The following is the Abstract Test Suite for determining if a CDB 2.x extension, profile, or implementation is conformant withe the CDB 2.0 or later core requirements modules.

## A.2. Conformance Class Core

| **Abstract Test 1** | **/conf/minimal-core** |
|---|---|
| Test Purpose | Validate that any CDB 2.0 and later datastore implements the minimum set of mandatory conformance classes as specified below. |
| Requirements Classes | CRS<br>File Naming<br>File Structure<br>Links<br>Metadata |
| Test Method | 1. Visual inspection of the application profile document Annex A - Abstract Test Suite |

# Annex B: Annex CDB Design Principles

The following are the major design principles that guided the development and documentation of the CDB 2.0 Core Standard. These principles are based on implementation experience, OGC SWG discussions, and results of experimentation in the SOFWERX CDB Sprint.

- Performance: Potential impacts on performance *SHALL* be a design consideration for decisions on core requirements.
- Deterministic: Requirements in the core *SHALL* support the definition of deterministic structures in any CDB application profile.
- Geometry model: All geometry for any vector feature data *SHALL* be compliant with the OGC/ISO Simple Features Geometry Model.
- Core as mandatory: There is a core set of requirements and conformance classes that *SHALL* be implemented in any implementation instance of a CDB data store.
- Tiling: The core *SHALL* specify an optional requirements class for tiling. This requirements class *SHALL* be consistent with the requirements as stated in OGC Abstract Specification Topic 22 - Core Tiling Conceptual and Logical Models for 2D Euclidean Space. Further, the optional tiling requirements class *SHALL* support the tiling of any layer as does the current CDB standard.
- LoD: The core *SHALL* specify an optional requirements class for Levels of Detail (LoD).
- Storage Independence: The core requirements *SHALL* be implementable in any storage structure ranging from a file system (as today) to container technology such as GeoPackage to the cloud.
- Attribute model: The CDB core should not mandate any particular data dictionary or content
  - Instead provide the conceptual and logical metamodel for describing any ISO 19109 compliant application schema to the maximum extent practical.
  - There should be no technical reason why one could not develop an extension profile for CDB for any particular data dictionary that complies with ISO 19109.
  - Delegate entity and attribute physical encoding choices to vector and 3D model containers instead of specifying globally.
- Holistic foundation: Any CDB application profile structures *SHALL* comply with the data structures, requirements and conformance classes defined in the core.
- 3D Model Independence: CDB 2.0 and later *SHALL* support 3D model encoding beyond OpenFlight. Requirements for using glTF *SHALL* be specified as an optional requirements class.
- Metadata: CDB and later *SHALL* specify mandatory and optional metadata elements including support for provenance. Any application profiles *SHALL* clearly specify metadata elements/requirements for that profile.
- The ability to version the contents of a CDB datastore is mandatory.
- Significant Size: Storing ‘significant size’ on model instancing point features can significantly improve the model retrieval scheme, rather than storing models in the significant size related folder scheme. Storing and evaluating significant size on instancing points can make visual content and performance tuning much more practical.

# B.1. Detailed Design Principles and Objectives

The following are more fine grained design principles and objectives for key aspects of the CDB-2.0 and later Core.

## B.1.1. Design Objectives for CDB X Tiling, LoDs and Layers

The following are the design principles for a CDB 2.0 and later tiling, levels of detail, and layering. These design principals were discussed and documented during Phase 1 discussions as well as during the experimentation. Again, for the purposes of expediting experimentation, GeoPackage was the choice for a standard container.

- Use metadata for whole datasets that describe how data layers are regrouped (LOD grouping and data layer grouping).
- Provide for fewer top level tiles than in OGC CDB 1.x.
- Store imagery in a container according to the tiling scheme specified below.
- Store coverage data in the container according to the tiling scheme specified below (need anchor). Suggested coverage types based on current CDB standard and user specified requirements: (Ellipsoidal body surface): Elevation models, visual and non-visual (multi-spectral) imagery, terrain light maps (emissive at night), and raster materials (for non-visual sensors). Should be extensible to support other types.
- Spatial filters appear to easily mimic the existing CDB tiling scheme. Therefore, the CDB-2.0 and later core standard should specify requirements that support the ability to 1.) Specify such filters and 2.) Provide highly performant queries.
- Any tiled data store should have tileset metadata for each tileset. This should include moving the minimum and maximum elevation values for the gridded elevation coverage contained in a tile to the tile metadata.
- Elevation min-max - Move the minimum and maximum elevation values for the gridded elevation coverage contained in a tile to the tile metadata.
- Need to keep the various encodings that CDB 1.x already has:
  - Elevation compression encodings (use of scaled integers vs floating point for smaller data sizes).
  - Elevation grids that are adjustable (better terrain fidelity at lower LODs/zoom levels)
  - Raster Material encodings using multiple coverage layers.
  - Imagery Compression (Imagery is typically the largest layer in CDB by disk storage).
- Must enable a "relatively" easy migration path from the CDB 1.1/1.2 tiling/LoD structure into the CDB 2.0 and later structure.

# B.2. Design Objecctices for Metadata and Provenance

- Metadata and provenance content should be self-describing.
- Keep the core set of mandatory metadata elements limited and simple.

- Define an extensible CDB metadata model that allows for easily incorporating additional metadata elements for specific data types, domains or applications.
- Discuss and agree on element names for the mandatory elements.
- Every CDB dataset should have its own metadata that describes the content of that specific dataset.

## B.3. Design principles from SOFWERX Sprint - CDB 2.0 Design Goals and Objectives

- Any profiles based on the core shall have the following requirements:
  - Correlation Requirement: CDB-T, CDB-S, CDB-G and CDB-A profiles shall correlate to the data defined in CDB-F.
  - Derivative and Optional structures: Application profiles shall encode rules and mechanisms for optimization in a self-describing form.

Keep stuff here until ready to delete.

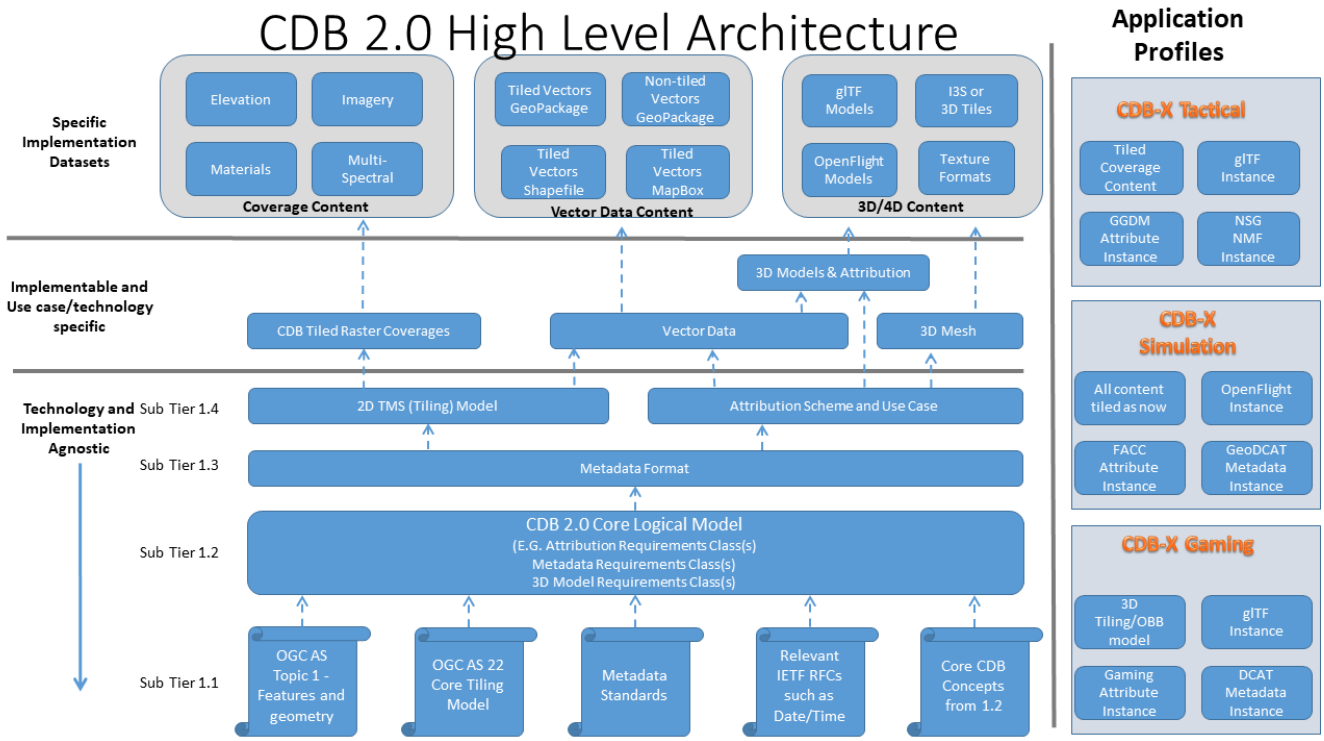
- Performance is a key design objective for CDB 2.0. Structured-data enables run-time performance. Be as performant as possible in terms of 1.) Speed of access and 2.) Impact on hardware resources.

## B.4. Additional 2.0 Design Requirements

- Adopt the latest version of OpenFlight, leveraging the extended materials, hot spots, and other important constructs provided in the latest version.
- Consider a Model indexing scheme that includes the following: Model LoDs, Interior and exteriors, Navigation and collision mesh.
- Consider reducing the current number of files in CDB 1.x, a recommendation is to group each model data into a zip (all LODs, textures etc…) while keeping the xml (index file) separate. A quick access to the index file would allow identifying what is required to be loaded into each ZIP. See Model Indexing recommentation. This is a possible CDB 1.3 enhancement.

# Annex C: Annex C Description of the High Level Architecture

The following figure presents a high level CDB 2.0 conceptual model.




## C.1. Three tiered architecture

The CDB 2.0 architecture is comprised of three primary tiers.

- Tier 1 - Technology and Implementation Agnostic. This tier contains the components of the model that are technology, language, platform, vendor, and implementation agnostic. This tier is also heavily vested in a number of existing OGC, ISO, W3C, and IETF Standards.
- Tier 2 - Implementable and Use case/technology specific. This tier documents the components of the architecture that can be implemented.
- Tier 3 - Implementation specific details based on content (resource) type. This tier is comprised of model components that describe in detail the requirements and properties for utilizing specific content/resources in a CDB data store.

### C.1.1. Tier 1

#### C.1.1.1. Foundation Conceptual Models and Abstract Foundation Standards

Tier 1 is abstract. At the base level Tier 1 is a foundation consisting of a number of OGC Abstract Specifications, ISO TC 211 91xxx series International Standards, and standards from other Standards Development Organization such as the W3C and the IETF.

The OGC Abstract Specification and ISO 19xxx Standards provide the conceptual foundation for

OGC Standards development activities. They also provide a conceptual (abstract) foundation for many other geospatial standards development activities in the IETF, W3C, OASIS, and a large number of government organizations. Open interfaces and protocols can be designed, built, and referenced against the Abstract Specification, thus enabling interoperability between different platforms, applications, and different spatial processing systems. The Abstract Specification provides a reference model for the development of standards that can be implemented, such as CDB 2.0 Core.

The following OGC and ISO Abstract Standards provide key components of the foundation for the CDB 2.0 Core conceptual architecture, logical models, and subsequent detailed implementation requirements and conformance classes.

1. Topic 2: Referencing by coordinates (aka ISO/DIS 19111:2018): This document defines the conceptual schema for the description of referencing by coordinates. It describes the minimum data required to define coordinate reference systems. All Coordinate and Coordinate Reference System requirements and terms and definitions specified in CDB 2.0 are based on this OGC Abstract Specification.
2. Topic 1: "Features and geometry – Part 1: Feature models" describes how geographic information in datasets and databases using a "feature model" are structured, created, stored, queried, and manipulated. Topic 1 (an earlier version) is the basis for the OGC Simple Features Standard. Simple features is an OGC Implementation Standard that is the basis for many other standards, such as GeoJSON and the GeoPackage vector storage model.
3. Topic 6: Topic 6 - Schema for coverage geometry and functions (aka ISO 19123) defines a conceptual schema for the spatial characteristics of coverages. Coverages support mapping from a spatial, temporal or spatiotemporal domain to feature attribute values where feature attribute types are common to all geographic positions within the domain. A coverage domain consists of a collection of direct positions in a coordinate space that may be defined in terms of up to three spatial dimensions as well as a temporal dimension.
4. Topic 22: OGC Abstract Specification Topic 22 - Core Tiling Conceptual and Logical Models for 2D Euclidean Space Describes 1.) a conceptual model for tiling space in any dimension and 2) A logical model for 2D tiled structures and by extension tiling. The logical model is based on the conceptual model.

#### C.1.1.2. Tier 1 Logical Model

The CDB Logical Model describes the core elements and required implementation rules for the CDB Standard. Terms and definitions, semantics, concepts and other elements of the CDB logical model are primarily based on the conceptual models defined in the abstract standards described above.

For example, the *OGC Topic 2: Referencing by coordinates* Standard provides an extensive vocabulary for terms and definitions used in referencing. These terms and definitions are included by reference as normative in the CDB Standard. Topic 2 also defines:

1. Two classes of conformance for relating coordinates to coordinate metadata; and
2. Twenty six classes of conformance for the definition of a coordinate reference system (CRS) or of a coordinate operation.

These conformance classes provide the basis for all Coordinate Reference System (CRS)

requirements and conformance clauses specified in the CDB Core Standard. These classes along with the requirements specified in the WKT for CRS Standard are the basis for all CRS requirements specified in every OGC Standard that specifies the use of a CRS.

#### C.1.1.3. Tier 1 CDB Metadata, Controlled Vocabularies, and Metadata Files

A critical aspect of developing, curating, maintaining, and using a CDB data store in a well known, interoperable manner is the definition and use of metadata. In CDB, the term *metadata* is often used in a much broader context than just "data about data". Metadata in a CDB data store is comprised by "traditional" metadata such as the author of a dataset, controlled vocabularies such as country names and codes, and enumerations such as months of the year.

In the CDB Core Standard, a single metadata standard is not specified as mandatory for encoding of metadata. The CDB 2.0 Standard allows for a number of metadata standards to be specified. However, the CDB 2.0 Core Standard does specifiy that all data sets shall have a minimum set of metadata.

The three main types of metadata utilized in a CDB data store are now defined.

### C.1.2. Metadata

Metadata is data that provides information about other data. In the geospatial community and the rapidly emerging spatial web community, metadata is critical to operations such as discovery and determining whether a resource is “fit for purpose”. Three distinct types of metadata exist: **descriptive metadata**, **structural metadata**, and **administrative metadata** (National Information Standards Organization (NISO)).

- Descriptive metadata describes a resource for purposes such as discovery and identification. It can include elements such as title, abstract, author, and keywords.
- Structural metadata is metadata about containers of metadata and indicates how compound objects are put together, for example, how pages are ordered to form chapters.
- Administrative metadata provides information to help manage a resource, such as when and how it was created, file type and other technical information, and who can access it. <end Wikipedia>

The geospatial community has a long and extensive history in defining and using metadata for geospatial resources. Without metadata, discovery of required resources and determination of whether a resource is “Fit for Purpose” becomes difficult if not impossible. The geospatial community makes use of all three types of metadata, although the first and third are more critical. The CDB use of metadata focuses on use cases 1 and 3.

#### C.1.2.1. Tier 1 Implementation Standards

Tier 1 may also reference Implementation Standards

1. Elements of existing OGC Implementation Standards, such as the Two Dimensional Tile Matrix Set Standard are specified as components of the CDB 2.0 Core Standard. In other words, such as with the case of tiling, the rules specified in the Core are applicable to all use cases, extensions, information domains, or profiles of the Core.

2. Specific guidance and requirements are specified for characteristics such as attribution that are not currently specified in an existing Implementation Standard. These characteristics are independent of use cases, domains, or profiles. In other words, the attribution rules specified in the Core are applicable to all use cases and information communities.

An important note is that even though specific standards are specified, the CDB Core architecture remains agnostic as to data formats, operating platforms, encodings, physical models, and so forth.

## C.1.3. Tier 2 - Implementable requirements and conformance classes

In Tier 2 of the architecture, specific implementable requirements are specified as part of the CDB Core. An example of implementable requirements are those that specifically describe the CDB Tiling structure. The Tiling structure in the CDB 1.x series of standards is well defined and widely implemented in the CDB community. However, the CDB tiling structure is not specifically tied to or consistent with international standards. Further, a number of issues have been documented over the years of CDB use. At the same time as recommended in the SOFWERX CDB Sprint, the CDB 2.0 tiling scheme must enable a "relatively" easy migration path from the CDB 1.1/1.2 tiling/LoD structure into the CDB 2.0 structure.

### C.1.3.1. Tier Two Tiling Example

Continuing with the Tiling example, a number of design criteria were identified in the SOFWERX CDB Sprint.

- Use metadata for whole datasets that describe basic information about the tile scheme as well as how data layers are regrouped (LOD grouping and data layer grouping).
- Provide for fewer top level tiles than in OGC CDB 1.x.
- Store imagery in a container according to the tiling scheme specified.
- Store coverage data in the container according to the tiling scheme specified in CDB-X. Suggested coverage types based on current CDB standard and user specified requirements: (Ellipsoidal body surface): Elevation models, visual and non-visual (multi-spectral) imagery, terrain light maps (emissive at night), and raster materials (for non-visual sensors). Should be extensible to support other types.
- Coordinate Reference System is WGS 84 with epoch encoding (same as current CDB Standard except for epoch).
- Need to keep the various encodings that CDB 1.x already has:
    - Elevation compression encodings (use of scaled integers vs floating point for smaller data sizes).
    - Elevation grids that are adjustable (better terrain fidelity at lower LODs/zoom levels)
    - Raster Material encodings using multiple coverage layers.
    - Use Imagery Compression (Imagery is typically the largest layer in CDB by disk storage).

More specifically, an example of a requirement based on the above design requirements and experience in the CDB Sprint might be:

| **Requirement 8** | **/req/core/variablematrixwidth/model** |
|---|---|
| Dependency | WGS-84 2d definition |
| Dependency | OGC Two Dimensional Tile Matrix Set Standard |
| Req | When a tiled resource or dataset has variable width tiles, the resource or dataset shall define the variable matrix width in a tile matrix set 2D using a variablematrixwidth data structure. This will result in the following UML model shown in Figure <TBD> and model description in Table Example. |

The complete set of tiling requirements would be captured and documented in a tiling requirements class. There would also then be an associated conformance class documented in the Abstract Test Suite (Annex A of any OGC Standard). The Tiling Conformance class would be specified as a Core conformance class which means that any CDB conformant implementation shall implement either the Tiling Conformance Class as defined or a profile of that Conformance Class.

## C.1.4. Tier 3

In this Tier, requirements related to specific categories of data content are specified. Many of the data content requirements specified in the current CDB 1.x Standards could be moved directly to CDB-X. Another characteristic of Tier 3 requirements is that they may or may not be in the core. This is because there is currently no requirement that a CDB data store implement any of the data content types detailed in the CDB 1.x Standard. An example is the current Volume 6: OGC CDB Rules for Encoding Data using OpenFlight

That said, there are also data content requirements that are specified in the Core. These are requirements for such elements as file naming. An example is provided below.

Another characteristic of the CDB 2.0 Tier 3 is the role of metadata. Unlike the current CDB 1.X Standard, there is no mandatory requirement for metadata at the data content storage level. In CDB 2.0, metadata is only recommended. In CDB 2.0, a minimal set of metadata is mandatory for any dataset in a CDB data store.

### C.1.4.1. Tier 3 Content Requirement Example

The following requirement is extracted from the current CDB 1.2 Standard (Volume 1). This requirement may or may not exist in a CDB 2.0 Application Profile. However, requirements such as this (and related conformance classes) mayl be specified in extensions to the CDB Core.

| **Requirement 107 Vector Type** | **req/vector-type-rule**<br><br>All instances of the feature *SHALL* be of the same vector data type. The CDB Standard requires a maximum of one vector data type for point features, a maximum of one type for lineal features and a maximum of one type for polygon features for each tile (for a maximum of 3 feature vector files per tile). |
|---|---|

**NOTE** The above is an example only!!

## C.2. CDB Application Profiles

**NOTE** The use of the term "Profile" here should probably be changed as this usage is not consistent with the OGC/ISO definition. This should be a discussion topic. I would suggest a better term is Application Profile or Domain Profile.

The SOFWERX/SOCOM CDB Sprint participants identified four application areas: CDB-Tactical, CDB-Simulation, CDB-Gaming and CDB-Analytics. These 4 application areas, as stated in the CDB-X Engineering Report, *shall* correlate to the data defined in CDB-Foundation. The CDB-Foundation conceptis the CDB 2.0 Core as defined in this document.

The CDB 2.0 Core and Content requirements do not specify what conformance classes (or profiles) and/or what extensions are required to meet the use cases and needs for a specific application domain or information community. An application domain is the segment of reality for which a standard(s) and related software system is developed. The definition of an application domain is the starting point for the actual-state analysis and the creation of a domain model. The domain model is a conceptual model of the domain that incorporates both behavior and data. In ontology engineering, a domain model is a formal representation of a knowledge domain with concepts, roles, datatypes, individuals, and rules, typically grounded in description logic.

In the CDB 2.0 context, this means that each application area (aka Profile in the Sprint Engineering Report):

1. Clearly specifies which Core requirements (and conformance classes) or profiles thereof are mandatory for that Application Profile;
2. Clearly states which data content requirements (and conformance classes) or profiles thereof are mandatory for that Application Profile; and
3. Clearly defines and documents any required extensions necessary to meet the requirements of a particular application domain.

**NOTE** This note clarifies how specifying a core enhances interoperability. First all application profiles *shall* implement the Core or profiles of the Core. The Core specifies core requirements/conformance classes that shall be implemented in ALL application profiles. An Application Profile may restrict a given requirements and related conformance class. For example, the CDB standard may specify a requirement that OGC WKT for CRS shall be used to specify any CRS information. A profile could restrict that requirement by stating that only WGS 84 2D as specified in EPSG code 4326 shall be used. (Of course the SWG may decide that only WGS 84 shall be used in any CDB data store!). Also remember that the Core not only specifies implementation requirements such as for Tiling, the Core also specifies requirements - as done the current CDB standard - for both characteristics of data types as well as for specific data themes. An example of a specific data theme is Navigation. An example of requirements for the characteristics of a general data type are the requirements and conformance classes for Tiled Elevation Data as

specified in the current CDB 1.x standard. As such, for example, any Application Profile that implements elevation data *shall* do so based on the specified requirements. The net result is interoperability regardless of the Application Profile used.

# Annex D: Revision History

| Date | Release | Editor | Primary clauses modified | Description |
|---|---|---|---|---|
| 2021-08-11 | 2.0 | C. Reed | all | Initial version |
| 2021-12-15 | 2.0 | C. Reed | CRS | Initial full draft for SWG approval |
| 2022-02-10 | 2.0 | C. Reed | Tiling | Abstract and Extension - initial full drafts for SWG review |
| 2023-12-28 | 2.0 | C. Reed | All | Get docs ready for publication |
| 2023-12-28 | 2.0 | C. Reed | All | Restructure documents to accommodate current OGC publication practices. |

# Annex E: Bibliography

| This table lists the documentation referenced throughout this document. | | |
|---|---|---|
| **Ref** | **Title** | **Description** |
| 1 | ARINC Standard 424-16 | Navigation System Data Base, Aeronautical Radio Inc., August 30, 2002. |
| 2 | ASTARS-04 CDB | Systems Requirements |
| 3 | Digital Geographic Information Exchange Standard (DIGEST), Standard V2.1 | The document can be obtained at the following address:<br><br>**http://www.digest.org/** |
| 4 | Enumeration and Bit Encoded Values for use with Protocols for Distributed Interactive Simulation Applications. | This is document SISO-REF-010. It accompanies IEEE Std 1278.1-1995 and can be obtained from the *Simulation Interoperability Standards Organization* at **http://www.sisostds.org/** |
| 5 | Extensible Markup Language (XML) 1.0 (Third Edition) | Bray, Tim, et al.<br><br>**http://www.w3.org/TR/2004/REC-xml-20040204/**<br><br>W3C Recommendation, February 04, 2004. |
| 6 | Guide - PD6777, BSI's _Guide to the practical implementation of JPEG 2000 _ | The document can be found at:<br><br>**http://www.jpeg.org/**<br><br>Other useful sites include:<br><br>**http://en.wikipedia.org/wiki/SRGB_color_space**<br><br>The document is targeted at managers; application software developers and end-users who want to know more about JPEG 2000. |
| 7 | IEEE Standard for Distributed Interactive Simulation - Application Protocols | IEEE Std 1278.1-1995 |

| | | |
|---|---|---|
| 8 | JPEG 2000: Image Compression Fundamentals, Standards and Practice | Kluwer International Series in Engineering and Computer Science, Secs 642, by David S. Taubman and Michael W. Marcellin |
| 9 | MIL-STD-2411 Raster Product Format Specification | The Raster Product Format (RPF) is a standard data structure for geospatial databases composed of rectangular arrays of pixel values (e.g., in digitized maps or images) in compressed or uncompressed form. RPF defines a common format for the interchange of raster-formatted digital geospatial data among DoD Components.<br><br>Department of Defense Information Technology Standards Registry Baseline Release 04-2.0.<br><br>http://earth-info.nga.mil/publications/specs/printed/2411/2411_RPF.pdf |
| 10 | MIL-C-89041 Controlled Image Base Specification | Controlled Image Base (CIB). This Specification provides requirements for the preparation and use of the RPF CIB data. CIB is a dataset of orthophotos, made from rectified grayscale aerial images.<br><br>http://www.fas.org/irp/program/core/mil-c-89041-cib.htm |
| 11 | OpenFlight Scene Description Database Standard, Version 16.0, Revision A, November 2004, Presagis Inc | The original document has been annotated by CAE to create the CDB-annotated OpenFlight Standard. |
| 12 | Product Standard for the Digital Aeronautical Flight Information File (DAFIF), Eight Edition, Doc. # PS/1FD/086 | National Imagery and Mapping Agency (NIMA), April 2003. |

| | | |
|---|---|---|
| 13 | SEDRIS™ - Synthetic Environment Data Representation Interchange Specification | The Source for Synthetic environment Representation and Interchange.<br><br>http://www.sedris.org |
| 14 | Shapefile Technical Description - an ESRI White Paper—July 1998 | The original document has been annotated by CAE Inc to create the CDB-annotated Shapefile Technical Description. |
| 15 | The SGI Image File Format, Version 1.00, Paul Haeberli, Silicon Graphics Computer Systems | This specification can be found at:<br><br>http://paulbourke.net/dataformats/sgirgb/sgiversion.html |
| 16 | TIFF rev 6.0 Adobe Developers Association, Adobe Systems Incorporated, 1585 Charleston Road and P.O. Box 7900Mountain View, CA 94039-7900 | A copy of this original standard can be found at:<br><br>http://partners.adobe.com/public/developer/en/tiff/TIFF6.pdf<br><br>and at:<br><br>ftp://ftp.adobe.com/pub/adobe/DeveloperSupport/TechNotes/PDFfiles<br><br>The original document has been annotated by CAE Inc to create the CDB-annotated TIFF Standard. |
| 17 | XML Schema Part 0: Primer Second Edition | Fallside, David, Priscilla Walmsley.<br><br>**http://www.w3.org/TR/xmlschema-0/**<br><br>W3C Recommendation, October 28, 2004. |
| 18 | XML Schema Part 1: Structures Second Edition | Thompson, Henry S., et al.<br><br>**http://www.w3.org/TR/xmlschema-1/**<br><br>W3C Recommendation, October 28, 2004. |

| | | |
|---|---|---|
| 19 | XML Schema Part 2: Datatypes Second Edition | Biron, Paul V., Ashok Malhotra.<br><br>**http://www.w3.org/TR/xmlschema-2/**<br><br>W3C Recommendation, October 28, 2004. |
| 20 | ICAO Airline Designator | List of ICAO Airline Codes,<br><br>**http://en.wikipedia.org/wiki/Airline_codes** |
| 21 | Radar Signatures and Relations to Radar Cross-Section. Mr. P E R Galloway, Roke Manor Research Ltd, Romsey, Hampshire, United Kingdom. | This document can be obtained at the following Internet address:<br><br>**http://aircraftdesign.nuaa.edu.cn/lo/Ref/General%20Topics/radar_signatures_and_relations_to_rcs.pdf** |
| 22 | AN/APA to AN/APD - Equipment Listing. | This document can be obtained at the following Internet address:<br><br>**http://www.designation-systems.net/usmilav/jetds/an-apa2apd.html#_APA** |
| 23 | Radar Polarimetry - Fundamentals of Remote Sensing.<br><br>National Resources Canada. | This document can be obtained at the following Internet address:<br><br>**https://www.nrcan.gc.ca/earth-sciences/geomatics/satellite-imagery-air-photos/satellite-imagery-products/educational-resources/9275** |
| 24 | RCS in Radar Range Calculations for Maritime Targets, by Ingo Harre. Bremen,<br><br>Germany. (V2.0-20040206). | This document can be obtained at the following Internet address:<br><br>**http://www.mar-it.de/Radar/RCS/RCS_xx.pdf** |

| | | |
|---|---|---|
| 25 | Decibels relative to a square meter – dBsm. By Zhao Shengyun. | This document can be obtained at the following Internet address:<br><br>**http://radarproblems.com/chapters/ch06.dir/ch06pr.dir/c06p11.dir/c06p11.htm** |
| 26 | MIL-C-89041 | Controlled Image Base (CIB) |
| 27 | MIL-STD-2411 | Defense Mapping Agency, Military Standard, Raster Product Format (RPF) |
| 28 | MIL-STD-2411-1 | Defense Mapping Agency, Registered Data Values for Raster Product Format |
| 29 | MIL-STD-2411-2 | Defense Mapping Agency, Incorporation of Raster Product Format (RPF) Data in National Imagery Transmission Format (NITF). |
| 30 | IEEE Std 1516-2000 | IEEE Standard for Modeling and Simulation (M&S) High Level Architecture (HLA) |
| 31 | RPR-FOM Version 2 Draft 17 | Real-time Platform Reference (RPR) Federation Object Model (FOM)<br><br>This RPR-FOM maps the DIS standard to the HLA standard.<br><br>The document can be obtained from the *Simulation Interoperability Standards Organization* at the following address:<br><br>**http://www.sisostds.org/** |
| 32 | MIL-PRF-89039 Amendment 2 | Performance Specification Vector Smart Map (VMAP Level 0), 28 September 1999<br><br>**http://en.wikipedia.org/wiki/Vector_Map http://earth-info.nga.mil/publications/specs/printed/VMAP0/vmap0.html** |

| | | |
|---|---|---|
| 33 | MIL-PRF-89033 Amendment 1 | Performance Specification Vector Smart Map (VMAP Level 1), 27 May 1998 |
| 34 | MIL-PRF-89035A | Urban Vector Map (UVMap), 1st August, 2002 |
| 35 | OneSAF Ultra High Resolution Building (UHRB) Object Model | OneSAF UHRB Object Model (Version 2.2, Document Revision E, March 7th, 2008, Contract #: N61339-00-D-0710, Task Order: 28.) *http://www.onesaf.net/community/systemdocuments/v.3.0/MaintenanceManual/erc/UHRB_2_Object_Model.pdf * |
| 36 | OneSAF Ultra High Resolution Building (UHRB) On-Disk Format | OneSAF UHRB On-Disk Format Model (Version 2.2, Document Revision E, March 7th, 2008, Contract #: N61339-00-D-0710, Task Order: 28.) *http://www.onesaf.net/community/systemdocuments/v.3.0/MaintenanceManual/erc/UHRB_2_On_Disk_Format.pdf * |
| 37 | U.S. Department of Transportation - Federal Aviation Administration – Advisory Circular 150/5340-1J | Standards for Airport Markings, AC- 150/5340-1J, dated 4/29/2005 |
| 38 | Federal Aviation Administration – Aeronautical Information Manual | Official Guide to Basic Flight Information and ATC Procedures, dated 14th February, 2008 |